MB86290 Series Graphics Driver V02
Users Manual
Rev. 0.1

FUJITSU LIMITED

Revision History		
Date	Rev.	Changes of contents
May. 30, 2003	0.1	The provisional version

Introduction

The purpose and target readers

This document explains mechanism of MB86290 Series Graphics Driver V02 (the “Graphics Driver”) and Application Program Interface (API).

This document is written for an engineer developing a graphics application using the “Graphics Driver”.

A description of this document has premised the reader who has understood specification of MB86290 Series Graphics Controller (the Graphics Controller) and technology about graphics.

If needed refer to the specification of Graphics Controller, or graphics-related books.

Necessary knowledge about graphics for understanding this document

To understand this document, following knowledge is needed.

Device coordinate system, Object coordinate system, Coordinate transformation, Conversion matrix, Clipping, Shading, Z-buffer method, Hidden surface elimination, Texture mapping, Tilling, Anti-aliasing, Alpha blending, Chrome-key composition, Palette color, etc.

Specifications of the Graphics Controller

For hardware specifications of the Graphics Controller and programming, refer to the following documents.

- Graphics Controller Specifications

- Application Note

These documents are prepared for every Graphics Controller. Each Graphics Controller and document names are described in the Table 1.

Table 1. List of documents

Graphics Controller	Document title
MB86290A	MB86290A Graphics Controller Hardware Specifications
	Cremson Application Note
MB86291/86291S	MB86291 <SCARLET> Graphics Controller Specifications
	MB86291 <SCARLET> Application Note
	I^2C Interface Specification
MB86291A	MB86291A <SCARLET2> Graphics Controller Specifications
	MB86291 <SCARLET> Application Note
MB86292/86292S	MB86292 <ORCHID> Graphics Controller Specifications
	MB86292 <ORCHID> Application Note
	I^2C Interface Specification
MB86293	MB86293 <CORAL-LQ> Graphics Controller Specifications
MB86294/86294S	MB86294 <CORAL-LB> Graphics Controller Specifications
	I^2C Interface Specification

A specific Graphics Controller group may be shown as follows in this document.

MB86291 or later : MB86291/86291S/86291A/86292/86292S/86293/86294/86294S Graphics Controller

MB86291/86292 : MB86291/86291S/86291A/86292/86292S Graphics Controller

MB86293 or later : MB86293/86294 Graphics Controller or later

Contents

1 MB86290 Series Graphics Driver Overview

This section describes abstract of the MB86290 Series Graphics Driver.

1.1 Overview

MB86290 Series Graphics Driver (the “Graphics Driver”) is a set of commands to assist development of graphics application programs utilizing the MB86290 Series Graphics Controller (the Graphics Controller). Each command to use the “Graphics Driver” from a graphics application program is called Driver Command. Abstract of the “Graphics Driver” is shown by Figure 1.1.

By using this “Graphics Driver”, graphics application programs can be made without the code concerned with hardware mechanism such as accessing to hardware registers and managing Display List (refer to “1.2.4 Display List”).

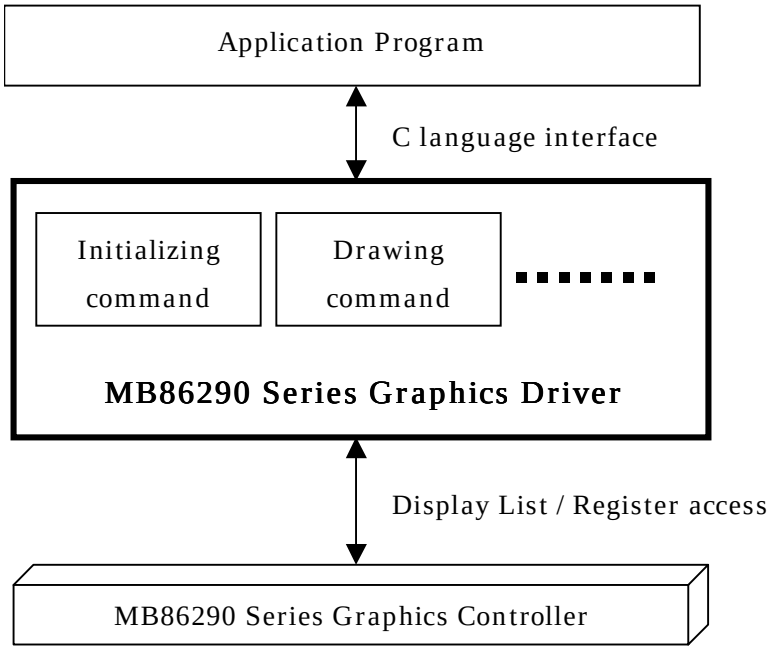


Figure 1.1 Overview of the “Graphics Driver”

1.2 Component parts of the “Graphics Driver”

This section describes the component parts of the “Graphics Driver”.

1.2.1 Component parts

The “Graphics Driver” consists of the following parts. Figure 1.2.1 shows the relationship of each part.

(1) Driver Command

(2) Context

(3) Display List

(4) System Dependent Command

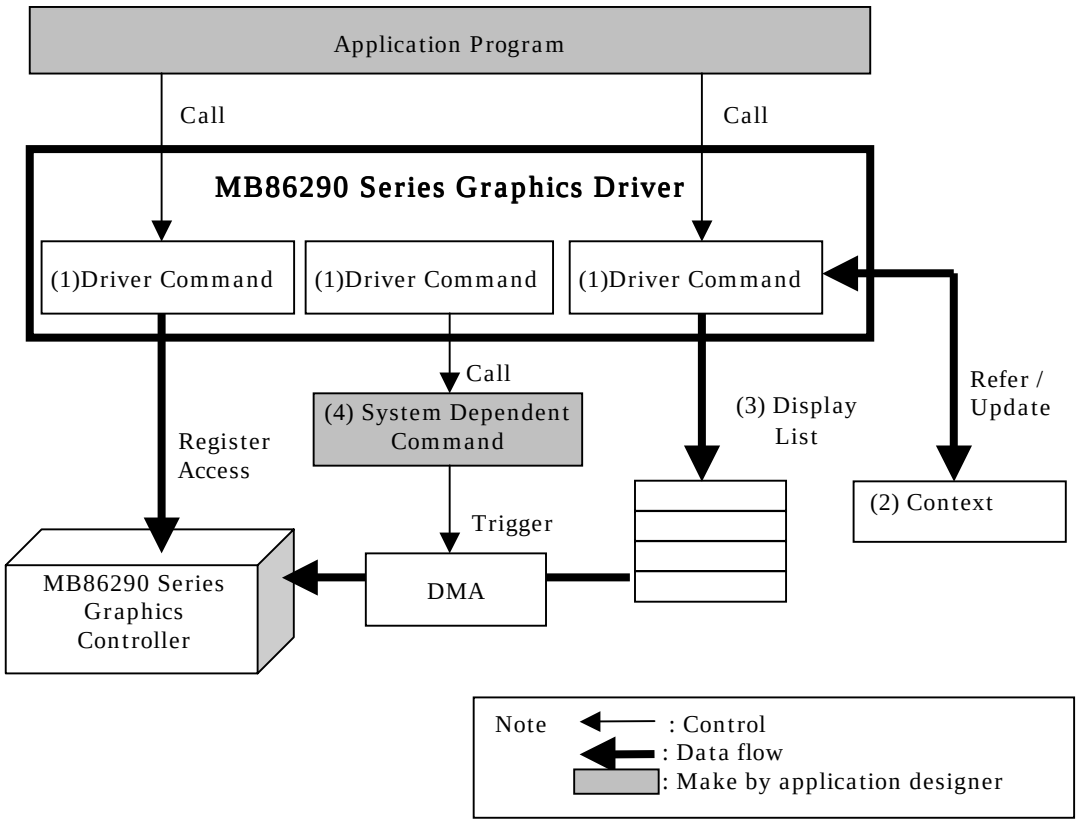


Figure 1.2.1 Component parts of the “Graphics Driver”

1.2.2 Driver Command

The Driver Command is an interface function used by application program. The Driver Command consists of “drawing command”, “display controlling command” and etc. By calling the Driver Command, the application is able to use the function of the Graphics Controller. The type of the Driver Command is described in section “2.1 The type of Driver Command”.

1.2.3 Context

The “Context” is the data which stores the status and rendering attribute of the “Graphics Driver”.

The “Context” has to be made by the **XGdcCreateContext** command at first. (Please refer section “4.2.1 **XGdcCreateContext** [Create a context]” for the detail of the **XGdcCreateContext command**)

The application has to allocate the memory area for the “Context”.

When creating two or more tasks which perform a drawing, by generating the “Context” individually by each task, drawing processing of these tasks does not need exclusion control, but can be performed simultaneously.

1.2.4 Display List

The Display List consists of the various drawing commands and the parameters. The Display List is made by drawing command of the Driver Command and transferred to the Graphics Controller. The application have to manage the store of the Display List and the transfer them to Graphics Controller. Regarding the management of the Display List, please refer “2.2 Management of the Display List”.

1.2.5 System Dependent Command

The System Dependent Command is a command to process procedure such as DMA transfer that depends on a target system and a graphics application program in the “Graphics Driver”. The System Dependent Command must be designed by each application designer according to the command interface specified by the “Graphics Driver”.

The details of this command is described in section “2.3 The type of System Dependent Command”.

2 The details of the “Graphics Driver”

This section describes the details for the “Graphics Driver”.

2.1 The type of the Driver Command

2.1.1 The components of the Driver Command

The Driver Commands are able to be categorized by the follows. Figure 2.1.1 shows the block diagram of each command.

(1) System control command

(2) Display control command

(3) Drawing control command

(4) Display List control command

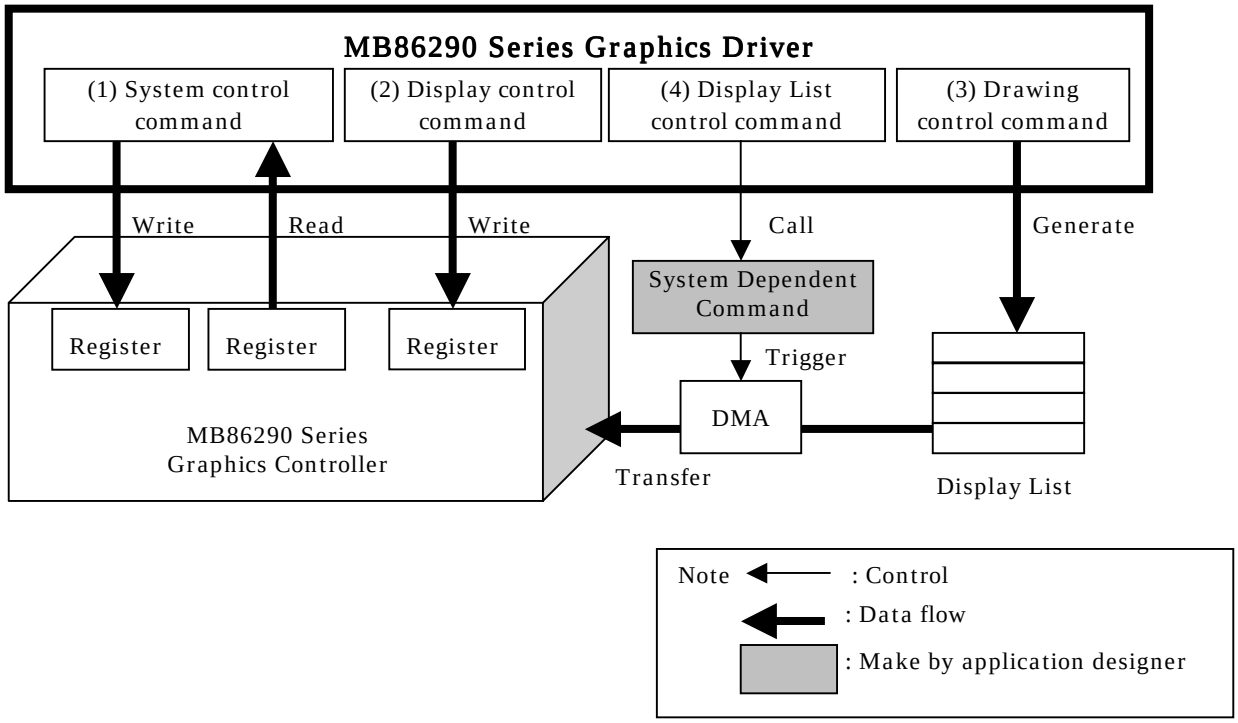


Figure 2.1.1 Work of the Driver Command

2.1.2 System control commands

The system control commands are the commands to initialize the driver, set the mode and status of the Graphics Controller, generate the context, get the error code and etc.

The system control commands are categorized as follows.

- System control commands (Refer to the section “4.1”)
- Context control commands (Refer to the section “4.2”)
- Error Control commands (Refer to the section”4.3”)

2.1.3 Display control command

The display control commands are commands for controlling display such as setting layers and displaying cursor. These functions operate resisters of Graphics Controller for above control.

The display control commands are categorized as follows.

- Display commands (Refer to the section “4.4”)
- Color control commands (Refer to the section “4.5”)
- Cursor control commands (Refer to the section “4.6”)
- Video Capture commands (Refer to the section “4.18”)
- I^2C control commands (Refer to the section “4.19”)

2.1.4 Drawing control command

The drawing control commands are the commands to generate the Display List which draws the graphics on a frame buffer. The drawing control commands are categorized as follows.

- Drawing frame control commands (Refer to the section “4.8”)
- Primitive drawing commands for device coordinate system (Refer to the section “4.9”)
- Primitive drawing commands for objects coordinate system (Refer to the section “4.10”)
- Drawing attribute control commands (Refer to the section “4.11”)
- Attribute control commands for object coordinate (Refer to the section “4.12”)
- Texture pattern control commands (Refer to the section “4.13”)
- Binary pattern drawing commands (Refer to the section “4.14”)
- BLT commands (Refer to the section “4.15”)
- Execution control commands (Refer to the section “4.16”)
- Drawing attribute restore command (Refer to the section “4.17”)

2.1.5 Display List control command

Display List control commands are the command to flush the Display List, cancel the Display List and etc. The details refer to the section “4.7”.

2.2 Management of Display List

2.2.1 Store the Display List

The “Graphics Driver” stores the Display List to the Display List buffer (DL Buffer). The DL Buffer is the area which is in the main memory or the Graphics Memory. This area has to be allocated by the application.

2.2.2 Organization of DL Buffer

The DL Buffer consists of 4 bytes alignment data area and is able to use plural numbers. Regarding the size of the DL Buffer is described in “2.2.3 Condition of the size and addressing of DL Buffer”.

The DL Buffer is related to the context by the **XGdcCreateContext** command. (Refer to “4.2.1 **XGdcCreateContext** [Creates a context]”)

Preparing the multiple number of the DL Buffer, both the making the Display List and the transfer the Display List are able to execute simultaneously. The Display List is able to save to re-use them.

The each DL Buffer is managed by the serial number from 0. This DL Buffer number is allocated to all of the contexts. Figure 2.2.2 shows the example of three DL Buffer.

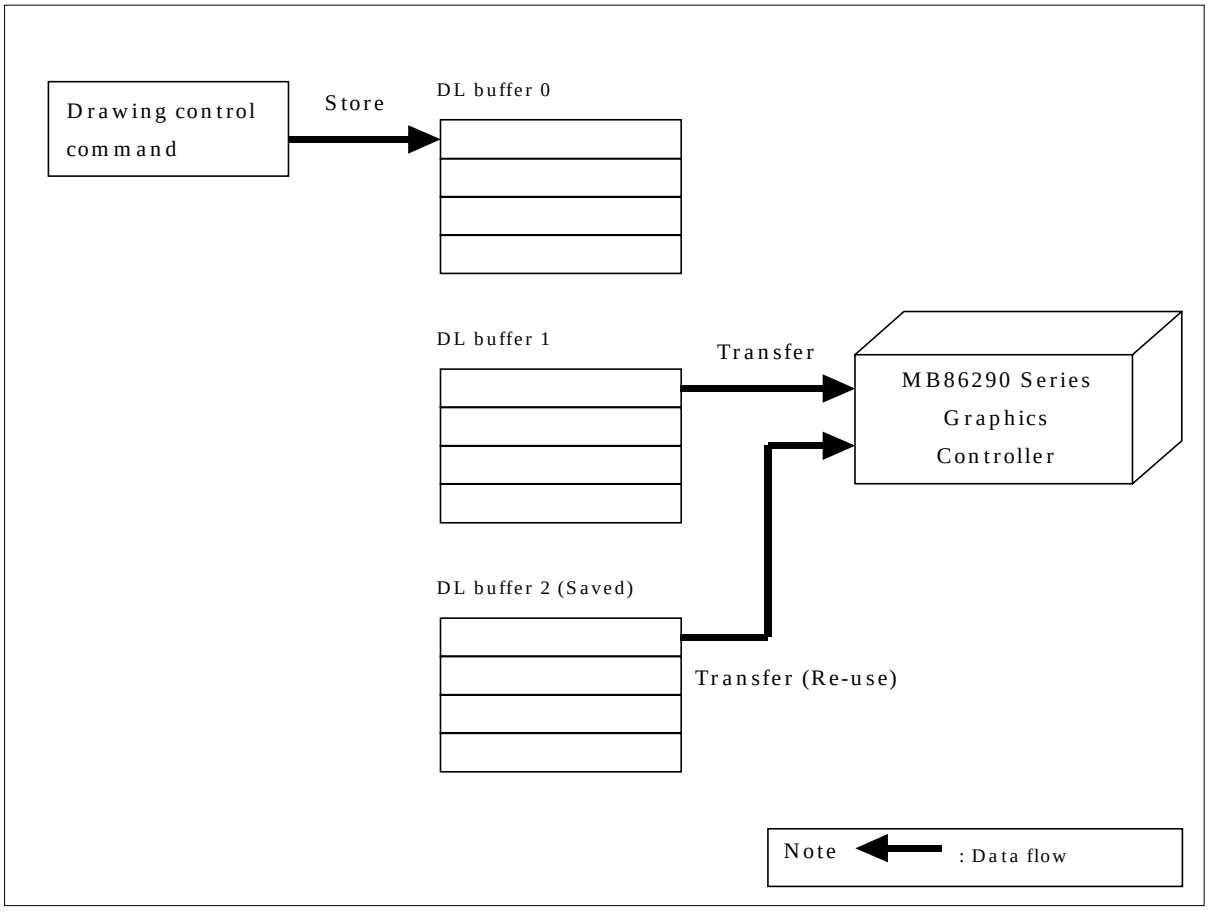


Figure 2.2.2 Example of using three DL Buffers

2.2.3 Condition of the size and addressing of the DL Buffer

The size of a DL Buffer should be equal or bigger than 16416 bytes and be in multiples of 32 bytes.

The size of a DL Buffer is recommended the size of Display List for drawing one frame.

The addressing of a DL Buffer should be set to four bytes boundary.

If the size and addressing do not obey the above condition in the **XGdcCreateContext** command, this command returns an error code. In case of it, if execute drawing, the system may happen hang-up. Perform management when an error occurs in the application which an error may generate.

The details of the **XGdcCreateContext** command shows in the section “4.2.1 **XGdcCreateContext** [Creates a context]”.

2.2.4 Select the DL Buffer

The Display List generated by drawing control commands is stored to “current DL Buffer”. “Current DL Buffer” is selected by the **XGdcSetDLBuf** command. (The details of the **XGdcSetDLBuf** command, refer to the section “4.7.1 **XGdcSetDLBuf** [Sets current DL Buffer]”) The default number of the DL Buffer is zero.

If the **XGdcSetDLBuf** command is called, the contents of selected DL Buffer are canceled and the Display List is stored from beginning of the DL Buffer. The Display List stored the DL Buffer is saved until called the **XGdcSetDLBuf** command with the same number.

Figure 2.2.4 shows the image of the selecting the DL Buffer.

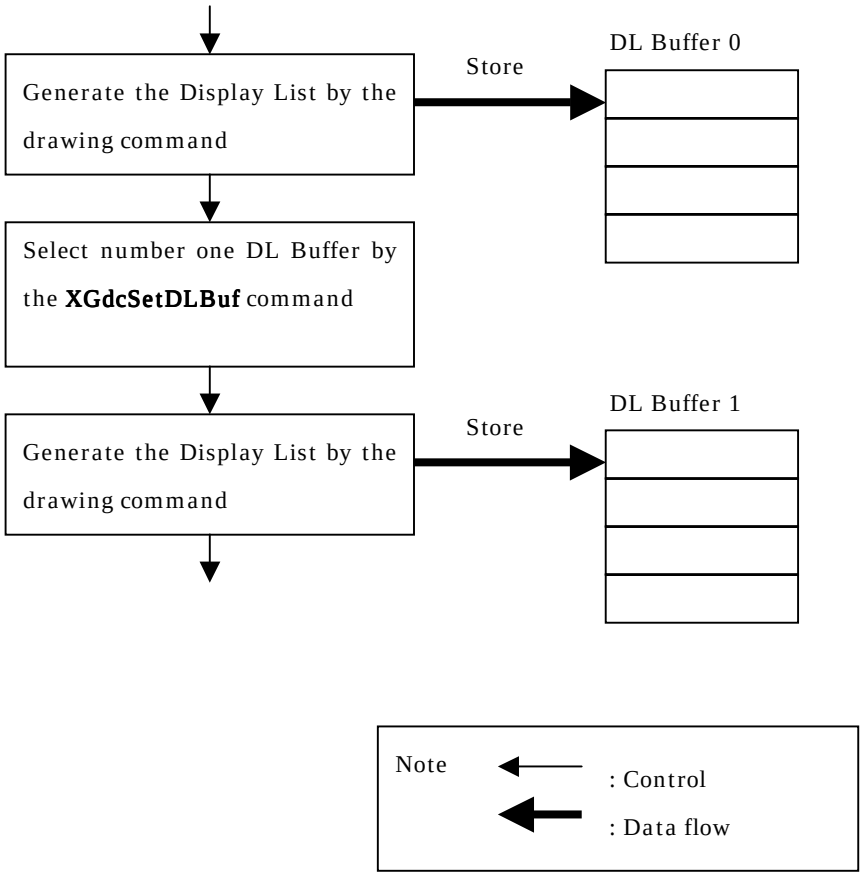


Figure 2.2.4 Image of selecting the DL Buffer

2.2.5 In case of inadequate DL Buffer size

If the DL Buffer is not allocated enough size, DL Buffer may be insufficient in generating the Display List. In this case, to continue the generating the Display List, switch the DL Buffer from current buffer to other buffer by the **XGdcSwitchDLBuf** command which is one of the System Dependent Commands. (Refer to the section "2.3 The type of System Dependent Command" and "6. System Dependent Command interface")

Figure 2.2.5 shows the case of inadequate DL Buffer size.

Note: The "Graphics Driver" does not support the management of order of the DL Buffer if the Display List is stored to multiple DL Buffers by the **XGdcSwitchDLBuf** command. Therefore this management should be programmed in the **XGdcSwitchDLBuf** by the application designer.

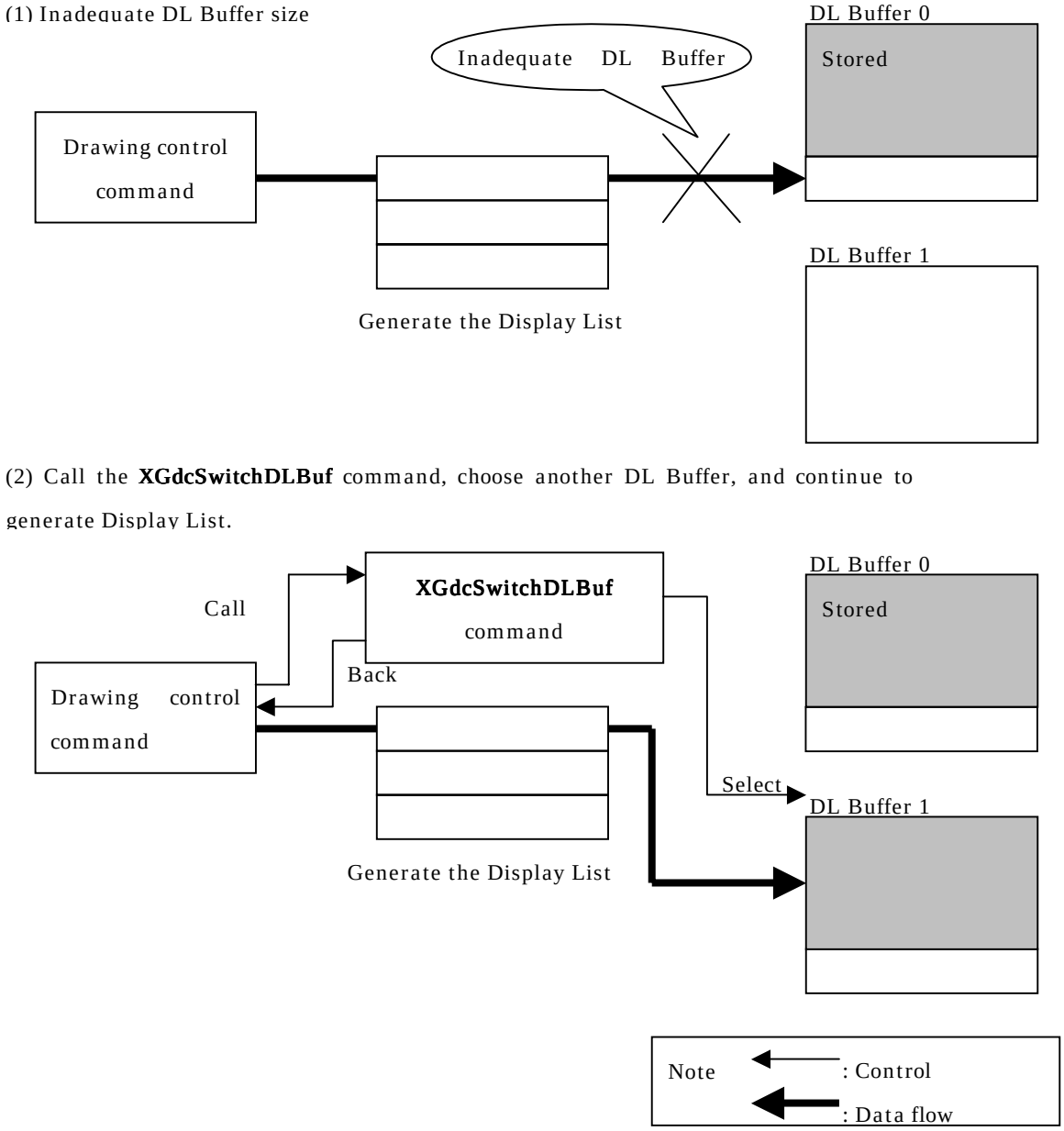


Figure 2.2.5 Inadequate DL Buffer size

2.2.6 Method of Display List

For the Display List transfer, the following three options are available. Depends on the target system configuration, each application designer should choose the most appropriate options:

The method of transfer is selected by the **GdcFlushDisplayList** command which is one of the System Dependent Commands. The details, refer to the section “2.3 The type of System Dependent Command” and “6. System Dependent Command interface”.

- DMA transfer
- Transfer of Local Display List (Read the Display List from the Graphics Memory by the Graphics Controller)
- Display List write by host CPU

2.3 The type of System Dependent Command

The System Dependent Commands are the commands to do the system or application dependent works like a DMA transfer. Figure 2.3 shows the relationship between System Dependent Command and the “Graphics Driver”.

The System Dependent Command should be programmed by the application designer according to the “Graphics Driver” interface. There are the following three commands in System Dependent Commands. The details refer to the section “6. System Dependent Command interface”.

(1) **GdcWait** command (Wait process)

This command is called for waiting for the time which initializing the Graphics Controller in the “Graphics Driver” initializing command (**GdcInitialize** command).

(2) **XGdcSwitchDLBuf** command (Switch the DL Buffer)

This command is called when switching the DL Buffer in case of inadequate the DL Buffer in drawing control command.

(3) **GdcFlushDisplayList** command (Transfer the Display List)

This command is called when flushing the Display List. By “2.2.6 Method of the Display List”, the Display List is transferred to the Graphics Controller.

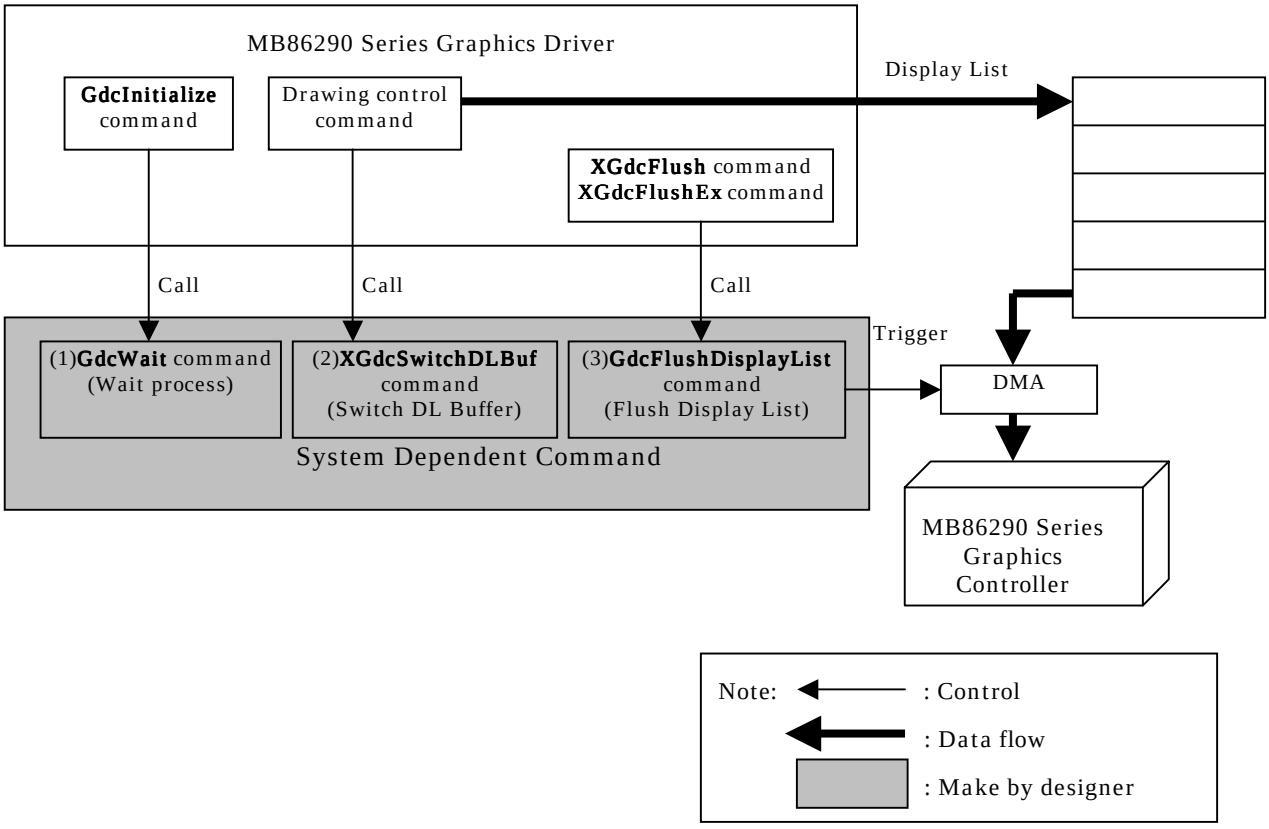


Figure 2.3 The relationship between the System Dependent Command and the “Graphics Driver”

2.4 Error process

If the parameter is out of order in the Driver Commands which order the parameter range, the Driver Commands set the error code. (In this case, the driver returns the **GDC_FALSE**)

The error code is able to get by the **XGdcGetErrCode** command. (The details of the **XGdcGetErrCode**, refer to the section "4.3.1 **XGdcGetErrCode** [Gets an error code]").

If the Driver Commands complete normally (in this case, the Driver Commands returns the **GDC_TRUE**), the error code does not set and save the latest error code.

To clear the error code, set the **GDC_ERR_NOERROR** by the **XGdcSetErrCode** command.

The details of the **XGdcSetErrCode** command, refer to the section "4.3.2 **XGdcSetErrCode** [Sets an error code]"

3 Data Format

This section describes the data types and data structures specified by the “Graphics Driver”.

3.1 Data Type

Data types to define in the “Graphics Driver” is shown in the Table **3.1**.

Table 3.1 Data type list

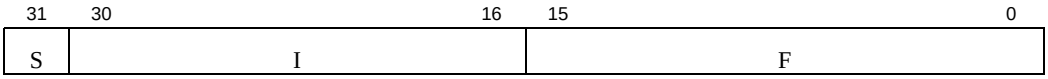
Format	Description
GDC_UCHAR	8 bits signed integer
GDC_SHORT	16 bits signed integer
GDC_USHORT	16 bits unsigned integer
GDC_LONG	32 bits signed integer
GDC_ULONG	32 bits unsigned integer
GDC_SFLOAT	32 bits single precision float (IEEE754 compliant)
GDC_BOOL	True/false (value of **GDC_TRUE** / **GDC_FALSE**) [Note] It is not used as a type of the return value of the Driver Command
GDC_FIXED32	32 bits signed fixed point (1 bits sign, 15 bits integer and 16 bits fraction)
GDC_FIXED_SCALE	16 bits unsigned fixed point for Capture Scale (5 bits integer and 11 bits fraction)
GDC_COLOR32	32 bits unsigned integer (32 bits color format)
GDC_COL32	32 bits unsigned integer (palette color format)
GDC_COL24	24 bits unsigned integer (24 bits color format)
GDC_COL16	16 bits unsigned integer (16 bits color format)
GDC_COL8	8 bits unsigned integer (8 bits color format)
GDC_BINIMAGE	32 bits unsigned integer data (binary pattern data)
GDC_VERTEX	Vertex data structure
GDC_INITPARAM	Initialize Parameter Table
GDC_DLBUF_STRUCT	Information of Display List buffer struct
GDC_CTX	Context

3.2 Data Structure

Data structures at to define in the “Graphics Driver” is shown in the following.

3.2.1 GDC_FIXED32 [32 bits fixed point]

A fixed-point data with sign described in sign 1 bit, integer 15 bits, and fraction 16 bits.



S: Sign (1 bit)

0:Positive number or zero

1:Negative number

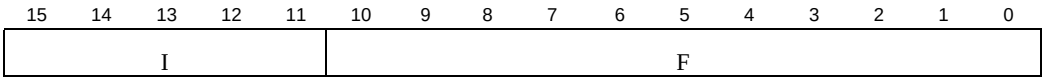
I: Integer (15 bits)

F: Fraction (16 bits)

Figure 3.2.1 GDC_FIXED32 format

3.2.2 GDC_FIXED_SCALE [Capture scale]

A capture scale data described in integer 5 bits, fraction 11 bits.
It used by the **GdcCapSetVideoCaptureScale** command.



I: Integer (5 bits)

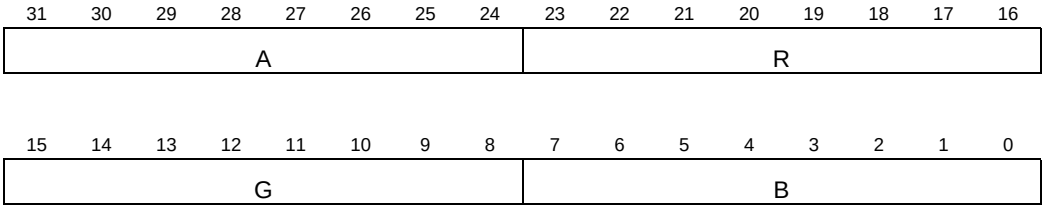
F: Fraction (11 bits)

Figure 3.2.2 GDC_FIXED_SCALE format

3.2.3 GDC_COLOR32 [32 bits color]

[16/24 bits color mode]

A color data described in 8 bits per A, R, G and B respectively.



A:Alpha bit (*1) (8 bits)

Sets blend ratio of vertex

(*1) An alpha bit here is the blend coefficient referred to when using the alpha shading function at triangle drawing.

R,G,B:

Color bit (8 bits)

Sets vertex color, each value range is 0 to 255.

Figure 3.2.3a GDC_COLOR32 format

[8 bits color mode]

A 8 bits color value is data expressed by the bit16-23.

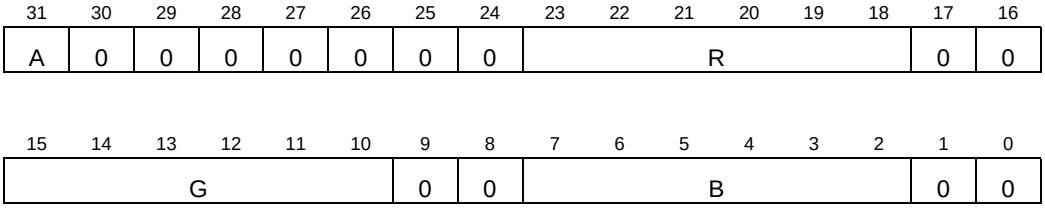
color: 8 bits color data (8 bits)

Other: All 0

Figure 3.2.3b GDC_COLOR32 format

3.2.4 GDC_COL32 [Palette color]

A color data described in 6 bits per R, G and B respectively. For C layer palette, bit 31 is an alpha bit.



A:Alpha bit (1 bit)

When blend mode is available, sets mode of blend

0:Blending

1:Not blending

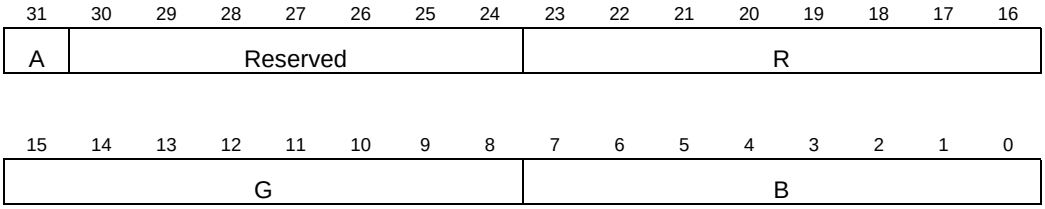
R,G,B:

Color bit (6 bits)

Figure 3.2.4 GDC_COL32 format

3.2.5 GDC_COL24 [24 bits color]

A color data described in 5 bits per R, B and G respectively. When this color data format is applied to texture, bit 31 is used as an alpha bit.



A:Alpha bit (1 bit)

When blend mode is available, sets mode of blend or stencil processing

0:Blending or stencil processing

1:Not blending or stencil processing

R,G,B:

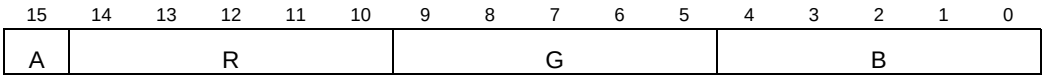
Color bit (8 bits)

Sets image data's color, each value range is 0 to 255.

Figure 3.2.5 GDC_COL24 format

3.2.6 GDC_COL16 [16 bits color]

A color data described in 5 bits per R, B and G respectively. When this color data format is applied to texture, bit 15 is used as an alpha bit.



A:Alpha bit (1 bit)

When blend mode is available, sets mode of blend or stencil processing

0:Blending or stencil processing

1:Not blending or stencil processing

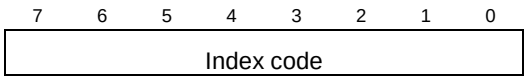
R,G,B:

Color bit (5 bits)

Figure 3.2.6 GDC_COL16 format

3.2.7 GDC_COL8 [8 bits color]

A index code to refer to color palette in 8 bits.



Index code: Index code (8 bits)

The index code in the case of referring to a color palette is specified, range 0 to 255.

Figure 3.2.7 GDC_COL8 format

3.2.8 GDC_VERTEX [GDC_SFLOAT format vertex data structure]

A structure data is packed in vertex coordinates, texture coordinates and RGB value.

It used by the **GdcDrawPrimitive** command.

GDC_VERTEX structure is shown in the following.

GDC_VERTEX structure

```
typedef struct {
    GDC_SFLOAT      x, y, z;    /* x,y,z coordinates of vertex for device coordinates */
    GDC_SFLOAT      r, g, b;    /* r,g,b value of vertex color */
    GDC_SFLOAT      u,v;        /* u,v texture coordinates of vertex */
    GDC_SFLOAT      rw;         /* Reciprocal w texture coordinates of vertex */
    long            work;       /*Reserved */
} GDC_VERTEX;
```

3.2.9 GDC_INITPARAM [Initialize Parameter Table]

An Initialize-Parameter Table is a table for specifying the various parameters used by initialization processing of the “Graphics Driver”, and is used with the **GdcInitialize** command.

The contents of a setting of the data structure of The Initialize-Parameter Table and each parameter are shown in the following.

GDC_INITPARAM structure

```
typedef struct {
    GDC_ULONG       cputype;        /* CPU type                          */
    GDC_ULONG       gdctype;        /* Graphics Controller type          */
    GDC_ULONG       gdcbase;        /* Graphics Controller base address  */
    GDC_ULONG       geoclock;       /* Geometry engine clock             */
    GDC_ULONG       otherclock;     /* Other clock                       */
    GDC_ULONG       locate;         /* Register location                 */
    GDC_ULONG       memorymode;     /* Memory interface mode             */
    struct {
        GDC_ULONG   mode;           /* Display clock mode                */
        GDC_ULONG   htp;            /* Horizontal pixel                  */
        GDC_ULONG   hsp;            /* Horizontal pulse position         */
        GDC_ULONG   hsw;            /* Horizontal sync width             */
        GDC_ULONG   hdp;            /* Horizontal display pixel          */
        GDC_ULONG   vtr;            /* Vertical total raster             */
        GDC_ULONG   vsp;            /* Vertical sync pulse position      */
        GDC_ULONG   vsw;            /* Vertical sync pulse width         */
        GDC_ULONG   vdp;            /* Vertical display raster           */
    } disp;
} GDC_INITPARAM;
```

[Parameters]

cputype

CPU to be used is specified on one of the following macros.

[Macro]	[Meaning]
GDC_CPU_SH	SH-3 or SH-4
GDC_CPU_WINDOWS	PC
GDC_CPU_V832	V832
GDC_CPU_OTHER	Other

gdctype

Graphics Controller to be used is specified on one of the following macros.

[Macro]	[Meaning]
GDC_TYPE_MB86290A	MB86290A
GDC_TYPE_MB86291	MB86291 or MB86291S
GDC_TYPE_MB86291A	MB86291A
GDC_TYPE_MB86292	MB86292 or MB86292S
GDC_TYPE_MB86293	MB86293
GDC_TYPE_MB86294	MB86294 or MB86294S

gdcbase

The address which mapped the Graphics Controller is specified.

Specify the return value of the Get_FrameAddress command of the map driver in PC.

geoclock (*1)

The clock of geometry engine is specified by following either.

This parameter is used only when GDC_TYPE_MB86293 or GDC_TYPE_MB86294 is specified to be gdctype, and in the case of others, it is ignored.

[Macro]	[Meaning]
GDC_CLOCK_166MHZ	166MHz
GDC_CLOCK_133MHZ	133MHz
GDC_CLOCK_100MHZ	100MHz

(*1) Combine geoclock and otherclock according to Table 3.2.9.

Table 3.2.9 Internal operation frequency combination

		geoclock		
		166MHz	133MHz	100MHz
otherclock	133MHz	Y	Y	N
	100MHz	Y	Y	Y

Y: Can be combination

N: Cannot be combination (Not supported)

otherclock (*1)

Clock of operation other than geometry engine, such as pixel engine, is specified by following either.

This parameter is used only when GDC_TYPE_MB86293 or GDC_TYPE_MB86294 is specified to be gdctype, and in the case of others, it is ignored.

[Macro]	[Meaning]
GDC_CLOCK_133MHZ	133MHz
GDC_CLOCK_100MHZ	100MHz

locate

The mapping location of the register area of the Graphics Controller is specified by following either.

It is used, only when this parameter specifies GDC_CPU_SH to be cputype and specifies GDC_TYPE_MB86293 or GDC_TYPE_MB86294 to be specification and gdctype, and in the case of others, it is ignored.

[Macro]	[Meaning]
GDC_REG_LOCATE_CENTER	Locate from H'01FC0000
GDC_REG_LOCATE_BOTTOM	Locate from H'03FC0000

memorymode

A memory interface mode value (setting value to MMR register) is specified.

Refer to the hardware specifications of the Graphics Controller of use about MMR register.

disp

The parameter about the display is specified as follows.

[Member variable]	[Meaning]
mode	Display clock mode (DCM or setting value to DCEM register) (*2)
htp	Horizontal total pixel
hsp	Horizontal pulse position
hsw	Horizontal sync width
hdp	Horizontal display pixel
vtr	Vertical total raster
vsp	Vertical sync pulse position
vsw	Vertical sync pulse width
vdp	Vertical display raster

(*2) Refer to the hardware specifications of the Graphics Controller of use about DCM or DCEM register.

3.2.10 GDC_DLBUF_STRUCT [DL Buffer structure information]

DL Buffer structure information is used with the **XGdcCreateContext** command, in order to register the structure of DL Buffer into a context.

The contents of a setting of the data structure of DL Buffer structure information and each parameter are shown below.

GDC_DLBUF_STRUCT structure

```
typedef struct{
    GDC_ULONG       *top;           /* Top address of DL Buffer          */
    GDC_ULONG       *bottom;        /* Last address of DL Buffer + 1     */
    GDC_ULONG       *cur;           /* Display List writing address      */
    GDC_ULONG       ext_data;       /* Extend data area                  */
} GDC_DLBUF_STRUCT;
```

[Parameters]

top

The top address of DL Buffer is specified.

bottom

The address of the last address +1 of DL Buffer is specified.

cur

The address which next writes in a Display List is set up by the Driver Command.

For this reason, o not changes the value of this area by application.

ext_data

Set up the value of this area by application if needed. Since the “Graphics Driver” does not perform reference or setup of this area, user can use this area freely.

3.2.11 GDC_CTX [Context]

A context is the area where the execution state and drawing attribute of the “Graphics Driver” are stored, and secures a area by the application side. The **XGdcCreateContext** command generates a context. Since the command for system control, the command for drawing control, and the command for Display List control use a context, do not change the value of this area by application (since the “Graphics Driver” manages this area, explanation of the data structure is omitted).

4 Driver Command reference

This chapter explains Driver Command API in the form of the following.

Interface

Function Interface.

Arguments

Description of arguments.

The "default" in explanation expresses the value set up at the initialization of the “Graphics Driver”.

Return value

The return values and the description of them.

Error code

Error code at when a Driver Command returns abnormally.

This item is omitted if the Driver Command has no return value.

Notes: Some of Driver Commands has a return value though always return normally. In these Driver Commands, this item does not exist.

Target GDC

Target Graphics Controllers of the Driver Command.

Description

Description of the Driver Command.

Notes

Notes.

4.1 System Control Commands

4.1.1 GdcInitialize [Initialization of the “Graphics Driver”]

Interface

GDC_BOOL **GdcInitialize** (const GDC_INITPARAM *_initparam_, int _flag_)

Arguments

initparam	Pointer to Initialize Parameter Table	
flag	Specify any of the following	
	GDC_INIT_START	Executes initialization
	GDC_INIT_RESET	Executes Software Reset

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_CPU_TYPE	Invalid CPU type is specified
GDC_ERR_INVALID_GDC_TYPE	Invalid Graphics Controller is specified
GDC_ERR_INVALID_FLAG	Value of *flag* is invalid

Target GDC

All

Description

Initializes the Graphics Controller.

After the system starts, call this command only once first before a calling other Driver Commands. And at this time, specify **GDC_INIT_START** for *flag*.

Also specify address of the Initialize Parameter Table for *initparam*. It is necessary to allocate memory space for the Initialize Parameter Table and to set each parameter before the calling this command. Refer to 3.2.9 GDC_INITPARAM [Initialize Parameter Table for specification of the Initialize Parameter Table.

To work a Software Reset, specify **GDC_INIT_RESET** for *flag*.

4.1.2 GdcSetBurstMode [Sets burst transfer mode of drawing]

Interface

void **GdcSetBurstMode** (GDC_ULONG *mode*)

Arguments

mode Burst transfer mode, specify any of the following.

GDC_ENABLE	Enable
GDC_DISABLE	Disable

Return value

None

Target GDC

MB86293 or later

Description

Sets burst transfer mode of drawing.

If the bus occupation rate of drawing becomes high, all the 6 layers may not be displayed. In this case, sets burst transfer mode to disable.

4.1.3 GdcSetDMAMode [Sets DMA Mode]

Interface

GDC_BOOL **GdcSetDMAMode** (int *tran_unit*, int *dma_request*,
int *address_mode*, int *ack_mode*)

Arguments

tran_unit Unit of DMA transfer.

GDC_DMA_TRANUNIT_4 4 bytes

GDC_DMA_TRANUNIT_32 32 bytes

dma_request DMA request. Specify any of the following.

GDC_DMA_REQUEST_NEGATE

During transferring, when the Graphics Controller cannot receive data, DMA request is invalid (negate), and if it will be in the state where data is receivable, it will be valid (assert)

GDC_DMA_REQUEST_ NO_NEGATE

Not negate while DMA is transferring Display List

address_mode Address mode of external DMA request. Specify any of the following.

GDC_DMA_ADDRMODE_DUAL Dual address mode

GDC_DMA_ADDRMODE_SINGLE Single address mode

ack_mode ACK mode. Specify any of the following.

GDC_DMA_ACKMODE

Uses ACK(detect DMA request at a low level signal)

GDC_DMA_NO_ACKMODE

Not use ACK(detect DMA request at an edge)

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_INVALID_PARAMETER Invalid parameter has specified

Target GDC

All

Description

Sets DMA transfer mode of DSU (DMA Set Up) register.

This command is available for all Graphics Controllers. However, *ack_mode* is available only for the MB86293 or later. In MB86290A/291/292, Specifies **GDC_DMA_ACKMODE** (uses ACK) for *ack_mode*.

4.1.4 GdcSetInterruptMask [Sets interrupt mask for MB86290A]

Interface

void **GdcSetInterruptMask** (GDC_UCHAR *mask*)

Arguments

mask Mask pattern (shown below)

bit 7	. . .	4	3	2	1	0
0	. . .	x	x	x	x	x

bit 0: Command error interrupt Mask=1, Enable=0
bit 1: Command complete interrupt Mask=1, Enable=0
bit 2: VSYNC interrupt Mask=1, Enable=0
bit 3: Frame sync interrupt Mask=1, Enable=0
bit 4: External sync error interrupt Mask=1, Enable=0
All other bits:0

Figure 4.1.4 Mask pattern format

Return value

None

Target GDC

MB86290A

Description

Sets interrupt mask pattern of IMASK (Interrupt MASK) register to disable interrupt requests generated by the respective events.

Notes

In MB86291 or later, use **GdcGeoSetInterruptMask** command instead of **GdcSetInterrupt-Mask** command.

4.1.5 GdcGeoSetInterruptMask [Sets interrupt mask for MB86291 or later]

Interface

void **GdcGeoSetInterruptMask** (GDC_ULONG *mask*)

Arguments

mask Mask pattern (shown below)

bit 31	...	4	3	2	1	0
0	...	x	x	x	x	x

bit 0: Command error interrupt Mask=0, Enable=1
bit 1: Command complete interrupt Mask=0, Enable=1
bit 2: VSYNC interrupt Mask=0, Enable=1
bit 3: Frame sync interrupt Mask=0, Enable=1
bit 4: External sync error interrupt Mask=0, Enable=1
All other bits:0

Figure 4.1.5 Mask pattern format

Return value

None

Target GDC

MB86291 or later

Description

Sets interrupt mask pattern of IMASK(Interrupt MASK) to disable interrupt requests generated by the respective events.

Notes

In MB86290A, use **GdcSetInterruptMask** command instead of **GdcGeoSetInterruptMask** command.

4.1.6 GdcGetInterruptStatus [Gets interrupt status for MB86290A]

Interface

GDC_UCHAR **GdcGetInterruptStatus** (void)

Arguments

None

Return value

Interrupts status (IST register value) in the following format:

bit 7	...	4	3	2	1	0
0	...	x	x	x	x	x

bit 0: Command error interrupt Yes=1, No=0
bit 1: Command complete interrupt Yes=1, No=0
bit 2: VSYNC interrupt Yes=1, No=0
bit 3: Frame sync interrupt Yes=1, No=0
bit 4: External sync error interrupt Yes=1, No=0
All other bits:0

Figure 4.1.6 Interrupt status

Target GDC

MB86290A

Description

Reads IST (Interrupt Status Register) and returns interrupt status.

Notes

In MB86291 or later, use **GdcGeoGetInterruptStatus** command instead of **GdcGetInterruptStatus** command.

4.1.7 GdcGeoGetInterruptStatus [Gets interrupt status for MB862901 or later]

Interface

GDC_ULONG **GdcGeoGetInterruptStatus** (void)

Arguments

None

Return value

Interrupts status (IST register value) in the following format:

bit 31	. . .	4	3	2	1	0
0	. . .	x	x	x	x	x

bit 0: Command error interrupt Yes=1, No=0
bit 1: Command complete interrupt Yes=1, No=0
bit 2: VSYNC interrupt Yes=1, No=0
bit 3: Frame sync interrupt Yes=1, No=0
bit 4: External sync error interrupt Yes=1, No=0
All other bits:0

Figure 4.1.7 Interrupt status

Target GDC

MB86291 or later

Description

Reads IST (Interrupt Status Register) and returns interrupt status.

Notes

In MB86290A, use **GdcGetInterruptStatus** command instead of **GdcGeoGetInterruptStatus**.

4.1.8 GdcClearInterruptStatus [Clears interrupt status for MB86290A]

Interface

void **GdcClearInterruptStatus** (GDC_UCHAR *clear*)

Arguments

clear Clear pattern (shown below)

bit 7	...	4	3	2	1	0
1	...	x	x	x	x	x

bit 0: Command error interrupt Clear=0, Hold=1
bit 1: Command complete interrupt Clear=0, Hold=1
bit 2: VSYNC interrupt Clear=0, Hold=1
bit 3: Frame sync interrupt Clear=0, Hold=1
bit 4: External sync error interrupt Clear=0, Hold=1
All other bits:1

Figure 4.1.8 Clear pattern format

Return value

None

Target GDC

MB86290A

Description

Clears the interrupt event indicated by bit0-4 in ISR (Interrupt Status Register) by the clear pattern specified as above. To clear an interrupt event, respective bit in the clear pattern for that event is set to 0 and all other bits are set to 1.

Notes

In MB86291 or later, use **GdcGeoClearInterruptStatus** command instead of **GdcClear-InterruptStatus**.

4.1.9 GdcGeoClearInterruptStatus [Clears interrupt status for MB86291 or later]

Interface

void **GdcGeoClearInterruptStatus** (GDC_ULONG *clear*)

Arguments

clear Clear pattern (shown below)

bit 31	...	4	3	2	1	0
1	...	x	x	x	x	x

bit 0: Command error interrupt Clear=0, Hold=1
bit 1: Command complete interrupt Clear=0, Hold=1
bit 2: VSYNC interrupt Clear=0, Hold=1
bit 3: Frame sync interrupt Clear=0, Hold=1
bit 4: External sync error interrupt Clear=0, Hold=1
All other bits:1

Figure 4.1.9 Clear pattern format

Return value

None

Target GDC

MB86291 or later

Description

Clears the interrupt event indicated by bit0-4 in ISR (Interrupt Status Register) by the clear pattern specified as above. To clear an interrupt event, respective bit in the clear pattern for that event is set to 0 and all other bits are set to 1.

Notes

In MB86290A, use **GdcClearInterruptStatus** command instead of **GdcGeoClearInterrupt-Status**.

4.1.10 GdcGetFIFOStatus [Gets status of Display List FIFO]

Interface

GDC_ULONG **GdcGetFIFOStatus** (void)

Arguments

None

Return value

Display List FIFO status in the following format:

bit 31	. . .	2	1	0
0	. . .	x	x	x

bit 0: Valid data exists in Display List FIFO =0
Valid data not exists in Display List FIFO =1
bit 1: Display List FIFO is full =0
Display List FIFO is not full =1
bit 2: More than half entries of Display List FIFO are empty =0
More than half entries of Display List FIFO are not empty =1
All other bits:0

Figure 4.1.10 Display List FIFO status

Target GDC

All

Description

Returns current Display List FIFO status.

4.1.11 GdcGetFIFORemain [Gets remains of Display List FIFO]

Interface

GDC_ULONG **GdcGetFIFORemain** (void)

Arguments

None

Return value

Number of open entries in the Display List FIFO

Target GDC

All

Description

Reads IFCNT (Input FIFO CouNTer) register and returns the number of open entries in the Display List FIFO.

4.1.12 GdcGetFIFOErrorStatus [Gets error status of Display List FIFO]

Interface

GDC_ULONG **GdcGetFIFOErrorStatus** (void)

Arguments

None

Return value

Display List FIFO error status in the following format:

bit 31	. . .	1	0
0	. . .	x	x

bit 0: Command error (type code is invalid) Yes=1, No=0

bit 1: Packet error (command code is invalid) Yes=1, No=0

All other bits:0

Figure 4.1.12 Display List FIFO error status

Target GDC

All

Description

Reads EST (Error Status Register) and returns Display List FIFO error status.

Command Error and Packet Error occurs if invalid Display List is sent to Display List FIFO. To clear error status, execute Software Reset or Hardware Reset.

4.1.13 GdcGeoGetFIFOStatus [Gets status of geometry Display List FIFO]

Interface

GDC_ULONG **GdcGeoGetFIFOStatus** (void)

Arguments

None

Return value

Geometry Display List FIFO status in the following format:

bit 31	. . .	2	1	0
0	. . .	x	x	x

bit 0:	Valid data exists in geometry Display List FIFO	=0
	Valid data not exists in geometry Display List FIFO	=1
bit 1:	Geometry Display List FIFO is full	=0
	Geometry Display List FIFO is not full	=1
bit 2:	More than half entries of geometry Display List FIFO are empty	=0
	More than half entries of geometry Display List FIFO are not empty	=1

All other bits:0

Figure 4.1.13 Display List FIFO status

Target GDC

MB86291 or later

Description

Returns current geometry Display List FIFO status.

4.1.14 GdcGeoGetFIFORemain [Gets remains of geometry Display List FIFO]

Interface

GDC_ULONG **GdcGeoGetFIFORemain** (void)

Arguments

None

Return value

Number of open entries in the geometry Display List FIFO

Target GDC

MB86291 or later

Description

Returns the number of open entries in the geometry Display List FIFO.

4.1.15 GdcGetPixelEngineStatus [Gets status of pixel engine]

Interface

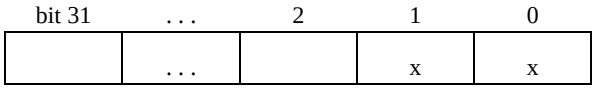
GDC_ULONG **GdcGetPixelEngineStatus** (void)

Arguments

None

Return value

Pixel engine status in the following format:



bit 1-0: Rendering is complete =00
Rendering is executing =01

All other bits: Don't care

Figure 4.1.15 Pixel engine status

Target GDC

MB86290A

Description

Returns Pixel Engine status.

Notes

In MB86291 or later, use **GdcGeoGetPixelEngineStatus** command instead of **GdcGetPixel-EngineStatus**.

4.1.16 GdcGeoGetPixelEngineStatus [Gets status of geometry pixel engine]

Interface

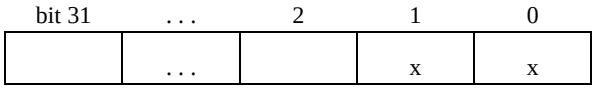
GDC_ULONG **GdcGeoGetPixelEngineStatus** (void)

Arguments

None

Return value

Pixel engine status in the following format:



bit 1-0: Rendering is complete =00
Rendering is executing =01

All other bits: Don't care

Figure 4.1.16 Geometry pixel engine status

Target GDC

MB86291 or later

Description

Returns Geometry Pixel Engine status.

Notes

In MB86290A, use **GdcGetPixelEngineStatus** command instead of **GdcGeoGetPixelEngine-Status**.

4.1.17 GdcGetLocalDisplayListTransferStatus [Gets status of Local Display List Transfer]

Interface

GDC_ULONG **GdcGetLocalDisplayListTransferStatus** (void)

Arguments

None

Return value

Transfer of Local Display List status in the following format:

bit 31	. . .	1	0
0	. . .	0	x

bit 0: Transfer is complete =0

Transfer is executing =1

All other bits:0

Figure 4.1.17 Local Display List transfer status

Target GDC

All

Description

Returns transfer of Local Display List status.

4.1.18 GdcQueryChipID [Queries chip ID]

Interface

void **GdcQueryChipID** (int **chip_no*, int **version*)

Arguments

chip_no	Pointer to the area which stores chip number
version	Pointer to the area which stores chip version number

Return value

None

Target GDC

MB86293 or later

Description

Returns chip number and chip version number.

Chip number and chip version number are numerical value.

For details about each number, refer to the hardware specification of the Graphics Controller of use.

4.1.19 GdcQueryGDCType [Queries Graphics Controller type]

Interface

GDC_ULONG **GdcQueryGDCType** (void)

Arguments

None

Return value

Type of Graphics Controller, specify any of the following

GDC_TYPE_MB86290A	MB86290A
GDC_TYPE_MB86291	MB86291 or MB86291S
GDC_TYPE_MB86291A	MB86291A
GDC_TYPE_MB86292	MB86292 or MB86292S
GDC_TYPE_MB86293	MB86293
GDC_TYPE_MB86294	MB86294 or MB86294S

Target GDC

All

Description

Returns type of Graphics Controller.

Notes

Graphics Controller type this command returns is type which is specified at the **GdcInitialize** command. Therefore, if the type different from type actually used is specified at the **GdcInitialize**, this command also returns type different from actual type.

4.1.20 GdcQueryVersion [Queries version number]

Interface

void **GdcQueryVersion** (int **version*, int **level*)

Arguments

version	Pointer to store version number
level	Pointer to store level number

Return value

None

Target GDC

All

Description

Returns current version and level number of the “Graphics Driver”.

Version number and level number are numerical value. For example, when the version number is 2, and the level number is 10, the following numbers are stored in each parameter.

*version = 2

*level = 10

4.2 Context Control Commands

4.2.1 XGdcCreateContext [Creates a context]

Interface

GDC_BOOL **XGdcCreateContext** (GDC_CTX **drvctx*,
GDC_ULONG *dlbufnum*,
GDC_DLBUF_STRUCT **dlbufstr*)

Arguments

drvctx	Pointer to Context
dlbufnum	Number of DL Buffer to be used, 1 or more
dlbufstr	Pointer to DL Buffer structure information table

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_ILLEGAL_NUMBER_OF_DL_BUF

Illegal number of DL Buffer has specified

GDC_ERR_ADDRESS_MISS_ALIGN

Address of Context, DL Buffer structure information or DL Buffer is not on 4 byte boundary

GDC_ERR_DL_BUF_TOO_SMALL

Size of DL Buffer is smaller than the minimum size

GDC_ERR_DL_SIZE

DL Buffer size is not a multiple of 32 bytes

Target GDC

All

Description

Creates a Context.

Allocates memory space for the Context, and specify that address for *drvctx*.

Specify number of DL Buffers to be used for *dlbufnum*.

Specify address of array of DL Buffer structure information for *dlbufstr*. The number of elements of that array is required for the number of DL Buffers to be used. It is necessary to allocate memory space for DL Buffer structure information and to set each parameter before the

calling this command. Refer to “3.2.10 GDC_DLBUF_STRUCT [DL Buffer structure information]” for specification of DL Buffer structure information.

Address of Context area, DL Buffer structure information and each DL Buffer must be on 4 bytes boundary. Also, size of each DL Buffer must be a multiple of 32 bytes.

Notes

* Since DL Buffer structure information is referred by each Driver Command after this function returns, please do not cancel it.
* Adjust the alignment of DL Buffer address according to DMA which you are using when you transfer Display List by DMA. Generally, DMA requires that start address is on boundary for transfer the unit.

4.3 Error Control Commands

4.3.1 XGdcGetErrCode [Gets an error code]

Interface

int **XGdcGetErrCode** (GDC_CTX **drvctx*)

Arguments

drvctx Pointer to Context

Return value

Error code

Target GDC

All

Description

Returns an error code of Driver Commands. After a Driver Command returns, the cause of abnormal termination can be taken by the calling of this function.

Specify NULL for *drvctx* when to get error code of Driver Commands which does not use Context such as Display Control Commands.

Notes

* Error code can be hold only one. Returns last error code when this command is called after the calling of two or more Driver Commands. However, holds separately the error code of commands use the Context and the error code of other commands.
* Error code is not cleared automatically. To clear the error code, call the **XGdcSetErrCode** command with **GDC_ERR_NOERROR**.

[Error code]	[Description]
GDC_ERR_NOERROR	No error has occurred
GDC_ERR_ADDRESS_MISS_ALIGN	Address of Context area, DL Buffer Info or DL Buffer is not 4byte boundary
GDC_ERR_DATA_TOO_BIG	Too large data
GDC_ERR_DL_BUF_TOO_SMALL	DL Buffer size is less than the minimum
GDC_ERR_DL_SIZE	DL Buffer size is not a multiple of 32 bytes
GDC_ERR_ILLEGAL_CAPTURE_SCALE	Illegal video capture scale has specified
GDC_ERR_ILLEGAL_CAPTURE_SIZE	Illegal video capture size has specified
GDC_ERR_ILLEGAL_DL_BUF_NO	Illegal DL Buffer number has specified
GDC_ERR_ILLEGAL_DIMENSION	Illegal vertical/horizontal size of pattern data

GDC_ERR_ILLEGAL_LINE_WIDTH	Illegal width of line
GDC_ERR_ILLEGAL_NUMBER_OF_DL_BUF	Illegal number of DL Buffer has specified
GDC_ERR_ILLEGAL_VERTEX_COUNT	Illegal number of vertex
GDC_ERR_INVALID_ATTRIBUTE	Invalid attribute has specified
GDC_ERR_INVALID_BANK	Invalid bank has specified
GDC_ERR_INVALID_COLOR_MODE	Invalid color mode has specified
GDC_ERR_INVALID_CPU_TYPE	Invalid CPU type has specified
GDC_ERR_INVALID_CURSOR_NUMBER	Invalid cursor number has specified
GDC_ERR_INVALID_FLAG	Invalid flag has specified
GDC_ERR_INVALID_GDC_TYPE	Invalid Graphics Controller type has specified
GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified
GDC_ERR_INVALID_PRIMITIVE	Invalid primitive has specified
GDC_ERR_NOT_READY	“Graphics Driver” is not initialized
GDC_ERR_NOT_START_GEO_DRAW_VERTEX	The **XGdcGeoPrimEnd** command has called before the call of the **XGdcGeoDrawVertex** command
GDC_ERR_NOT_START_GEO_PRIM_TYPE	The **XGdcGeoPrimEnd** command has called before the call of the **XGdcGeoPrimType** command and the **XGdcGeoDrawVertex** command
GDC_ERR_SWITCH_DL_BUF_FAILED	The **XGdcSwitchDLBuf** command has finished abnormally

4.3.2 XGdcSetErrCode [Sets an error code]

Interface

void **XGdcSetErrCode** (GDC_CTX **drvctx*, int *errcode*)

Arguments

drvctx	Pointer to Context
errcode	Error code

Return value

None

Target GDC

All

Description

Sets an error code. Use this command for the following purposes.

- To clear error code
- To set error code of the error occurred inside of System Dependent Command

Specify NULL for *drvctx* when to set error code of Driver Commands which does not use Context such as Display Control Commands.

Notes

Use more than 1000 if you define error codes newly.

4.4 Display Commands

4.4.1 GdcDispClock [Sets display clock mode]

Interface

void **GdcDispClock** (GDC_ULONG *mode*)

Arguments

mode	Sets display clock mode. This parameter is directly set to DCM(MB86290A/86291/86292) or DCEM(MB86293 or later) register of the Graphics Controller. For details of the DCM and DCEM register description, refer to the Graphics Controller hardware specification of use.

Return value

None

Target GDC

All

Description

Controls display clock and sync mode by setting parameters to Display Control Mode register.

- Sets display sync mode
- Sets external sync mode
- Sets signal type
- Sets dot clock frequency
- Sets dot clock source

4.4.2 GdcDispTiming [Sets display timing parameters]

Interface

void **GdcDispTiming** (GDC_USHORT *htp*, GDC_USHORT *hsp*,
GDC_USHORT *hsw*, GDC_USHORT *hdp*,
GDC_USHORT *vtr*, GDC_USHORT *vsp*,
GDC_USHORT *vsw*, GDC_USHORT *vdp*)

Arguments

htp	Total horizontal pixel count, range 1-4096
hsp	Hsync pulse timing, range 1-4096
hsw	Hsync pulse width, range 1-256
hdp	Horizontal display pixel count, range 1-4096
vtr	Total vertical raster count, range 1-4096
vsp	Vsync pulse timing, range 1-4096
vsw	Vsync pulse width, range 1-64
vdp	Vertical display raster count, range 1-4096

Return value

None

Target GDC

All

Description

Sets display window size and display timing parameters.

Each value of parameter are not checked of the range, specify a value within the decided range.

4.4.3 GdcDispTimingWindow [Sets display position of layer W]

Interface

void **GdcDispTimingWindow** (GDC_USHORT *x*, GDC_USHORT *y*,
GDC_USHORT *w*, GDC_USHORT *h*)

Arguments

x	x coordinate in the device coordinates (0 to 4095)
y	y coordinate in the device coordinates (0 to 4095)
w	Window width (pixel unit) (1 to 4095)
h	Window height (pixel unit) (1 to 4096)

Return value

None

Target GDC

All

Description

Sets display position of layer W(Window).

x and *y* should specify the display position at the upper left of a window frame.

Each value of parameter are not checked of the range, specify a value within the decided range.

4.4.4 GdcDispDividePos [Sets border position of screen partition]

Interface

void **GdcDispDividePos** (GDC_USHORT *hdb*)

Arguments

hdb Horizontal pixel count of left window, range 1-4096

Return value

None

Target GDC

All

Description

Sets position where screen is divided left and right.

Value 0 cannot be specified for *hdb*. The value of parameter is not checked of the range, specify a value within the decided range.

4.4.5 GdcDispDimension [Sets attributes of display frame]

Interface

GDC_BOOL **GdcDispDimension** (GDC_UCHAR *layer*, GDC_UCHAR *enable*,
GDC_UCHAR *cmode*, GDC_UCHAR *fmode*,
GDC_ULONG *loa0*, GDC_ULONG *loa1*,
GDC_USHORT *lw*, GDC_USHORT *lh*)

Arguments

layer Layer selection

GDC_DISP_LAYER_C	Layer C
GDC_DISP_LAYER_W	Layer W
GDC_DISP_LAYER_ML	Layer ML
GDC_DISP_LAYER_MR	Layer MR
GDC_DISP_LAYER_BL	Layer BL
GDC_DISP_LAYER_BR	Layer BR

[In MB86293 or later, following macros are available]

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

enable Layer display enable/disable

GDC_ENABLE	Layer display enable
GDC_DISABLE	Layer display disable

cmode Color mode selection

GDC_24BPP_FORMAT	24 bits color mode (*1)
GDC_16BPP_FORMAT	16 bits color mode
GDC_8BPP_FORMAT	8 bits color mode

fmode Flipping mode selection

GDC_FLIPMODE_0	Display Bank 0
GDC_FLIPMODE_1	Display Bank 1
GDC_FLIPMODE_AUTO	Display both banks alternately

loa0 Base address of logical frame of bank 0, specify offset from the top of the Graphics Memory

loa1	Base address of logical frame of bank 1, specify offset from the top of the Graphics Memory
lw	Logical frame width, in pixels, range 64-4096 in 24 bits color, range 32-4096 in 16 bits color, range 16-4096 in 8 bits color
lh	Logical frame height, in pixels, range 1-4096

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_PARAMETER	Invalid layer has specified

Target GDC

All

Description

Sets attributes of logical frame. Attributes are independent with each display layer(C, W, ML, MR, BL and BR).

If this function is called, the display origin address of the specified layer will be set as the same value as the specified display frame base address (Sets to upper left corner of the frame).

When layer W or layer L1 is specified, *cmode*, *fmode*, *loa1* and *lh* is not used.

When the following macros are specified for enabling/disabling to display layers, both ML and MR, both BL and BR layers are displayed simultaneously.

- **GDC_DISP_LAYER_ML**
- **GDC_DISP_LAYER_MR**
- **GDC_DISP_LAYER_BL**
- **GDC_DISP_LAYER_BR**

Also, following macros can enable/disable to display individually.

- **GDC_DISP_LAYER_L2**
- **GDC_DISP_LAYER_L3**
- **GDC_DISP_LAYER_L4**
- **GDC_DISP_LAYER_L5**

When layer L5 is used as a blend coefficient layer, this layer must be displayed in 8 bits color mode.

In case of extend display mode or extend overlay mode, layer L4 and L5 are not available in 8 bits color mode (except layer L5 is used as a blend coefficient layer).

(*1) The 24 bits color mode function is for MB86293 or later, and this mode can be used with the Graphics Controller supporting the 24 bits color mode option.

4.4.6 GdcDispOn [Asserts video signal output]

Interface

void **GdcDispOn** (void)

Arguments

None

Return value

None

Target GDC

All

Description

Outputs video signals.

Screen display is started at this command call, so this command must be called after all the rest display parameters are set. Nothing is displayed prior to this command call.

4.4.7 GdcDispOff [Negates video signal output]

Interface

void **GdcDispOff** (void)

Arguments

None

Return value

None

Target GDC

All

Description

Disables screen display of video signals.

This command does not affect drawing processing by the Graphics Controller.

4.4.8 GdcDispLayerOn [Asserts screen display]

Interface

GDC_BOOL **GdcDispLayerOn** (GDC_UCHAR *layer*)

Arguments

layer Layer selection

GDC_DISP_LAYER_C	Layer C
GDC_DISP_LAYER_W	Layer W
GDC_DISP_LAYER_M	Layer M
GDC_DISP_LAYER_B	Layer B

[In MB86293 or later, following macros are available]

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified

Target GDC

All

Description

Displays the layer specified by *layer*.

When the following macros are specified, both ML and MR, both BL and BR layers are displayed simultaneously.

- **GDC_DISP_LAYER_M**
- **GDC_DISP_LAYER_B**

When the layer L5 is used as a blend coefficient layer, this layer must be displayed.

4.4.9 GdcDispLayerOff [Negates screen display]

Interface

GDC_BOOL **GdcDispLayerOff** (GDC_UCHAR *layer*)

Arguments

layer Layer selection

GDC_DISP_LAYER_C	Layer C
GDC_DISP_LAYER_W	Layer W
GDC_DISP_LAYER_M	Layer M
GDC_DISP_LAYER_B	Layer B

[In MB86293 or later, following macros are available]

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified

Target GDC

All

Description

Stops the display of specified layer.

When the following macros are specified, both ML and MR, both BL and BR layers are not displayed simultaneously.

- **GDC_DISP_LAYER_M**
- **GDC_DISP_LAYER_B**

4.4.10 GdcDispPos [Sets display start position]

Interface

GDC_BOOL **GdcDispPos** (GDC_UCHAR *layer*, GDC_UCHAR *bank*,
GDC_USHORT *dx*, GDC_USHORT *dy*)

Arguments

layer Layer selection

GDC_DISP_LAYER_C	Layer C
GDC_DISP_LAYER_ML	Layer ML
GDC_DISP_LAYER_MR	Layer MR
GDC_DISP_LAYER_BL	Layer BL
GDC_DISP_LAYER_BR	Layer BR

[In MB86293 or later, following macros are available]

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

bank Logical frame bank selection

GDC_DISP_BANK_0	Bank 0
GDC_DISP_BANK_1	Bank 1

dx x coordinates of display start position, range 0-4095

dy y coordinates of display start position, range 0-4095

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_BANK	Invalid bank has specified

Target GDC

All

Description

Sets the display start position by the distance from the base position of logical frame.

After this command execution, it set as the target of display of the bank specified by *bank*.

4.4.11 GdcDispDoFlip [Flips display bank]

Interface

GDC_BOOL **GdcDispDoFlip** (GDC_UCHAR *layer*, GDC_UCHAR *bank*)

Arguments

layer Layer selection

GDC_DISP_LAYER_ML	Layer ML
GDC_DISP_LAYER_MR	Layer MR
GDC_DISP_LAYER_BL	Layer BL
GDC_DISP_LAYER_BR	Layer BR

[In MB86293 or later, following macros are available]

GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

bank Logical frame bank selection

GDC_DISP_BANK_0	Bank 0
GDC_DISP_BANK_1	Bank 1

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_BANK	Invalid bank has specified

Target GDC

All

Description

Switches bank to be displayed.

4.4.12 GdcOverlayPriorityMode [Sets overlay display mode]

Interface

GDC_BOOL **GdcOverlayPriorityMode** (GDC_UCHAR *mode*)

Arguments

mode C layer overlay mode

GDC_OVERLAY_C_PRIORITY
Simple priority mode (default)
GDC_OVERLAY_C_BLEND
Blend mode

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_INVALID_PARAMETER Invalid parameter has specified

Target GDC

All

Description

Sets overlay display mode. When simple priority mode is selected, layer C is displayed at the top of all the layers all the time. When blend mode is selected, after displaying all the rest layers according to the priority order, layer C color is transparently blended with the rest layers.

4.4.13 GdcOverlayBlend [Sets blend parameter for overlay blend]

Interface

GDC_BOOL **GdcOverlayBlend** (GDC_UCHAR *select*, GDC_UCHAR *blend*)

Arguments

select Overlay blend selection

GDC_BLEND_RATIO_C	Blend target is layer C color
GDC_BLEND_RATIO_WMB	Blend target is layer W/M/B color

blend Blending ratio, only upper 4 bits are valid: 0x00 to 0xf0

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

All

Description

Sets the blend coefficient to determine the layer C color when the overlay mode is blend mode.

The followings are the meanings of blend coefficient and formula to determine the layer C color.

[Blend coefficient]

blend	blend coefficient
0x00	0
0x10	1/16
0x20	2/16
0x30	3/16
:	:
0xf0	15/16

[Blend formula]

- For **GDC_BLEND_RATIO_C**

(layer_C_color * blend_coefficient)
+ (layer_ /M/B_compound_color * (1- blend_coefficient))

- For **GDC_BLEND_RATIO_WMB**

(layer_C_color * (1- blend_coefficient))
+ (layer_W/M/B_ compound_color * blend_coefficient)

4.4.14 GdcDispDisplayMode [Sets display mode]

Interface

GDC_BOOL **GdcDispDisplayMode** (GDC_UCHAR *mode*)

Arguments

mode Display mode

GDC_STANDARD_MODE Standard Display Mode (default)

GDC_OVERLAY_EXT_MODE Extend Overlay Mode

GDC_WINDOW_MODE Window Mode

GDC_EXTEND_MODE Extend Display Mode

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_INVALID_PARAMETER Invalid parameter has specified

Target GDC

MB86293 or later

Description

Sets all layers to same display mode.

There are four kinds of display modes. In each display mode, the layer order which can be set up is different. Difference among these modes are shown in Table 4.4.14a and Table 4.4.14b. And also displayed image of Standard Display Mode and Extend Display Mode are shown in Figure 4.4.14.

Table 4.4.14a Functions of each display mode

Function	Extend Display Mode	Window Mode	Extend Overlay Mode	Standard Display Mode
Overlay	6 layer	6 layer	4 layer + right and left division	4 layer + right and left division
Displaying window	6 layer	6 layer	1 layer	1 layer
Changing layer order	At pleasure	Layer L0 only	At pleasure	Layer L0 only
Number of color palette	4	2	4	2

Table 4.4.14b Layer order of each display mode

Layer	Standard Display Mode and Window Mode	Extend Display Mode and Extend Overlay Mode
L0	On highest or lowest (*1)	Changeable at pleasure
L1	On highest or under layer L0 (*1)	"
L2	Under layer L1	"
L3	"	"
L4	"	"
L5	"	"

(*1) The order of layer L0 and L1 is determined by the **GdcDispLayerOrder** command.

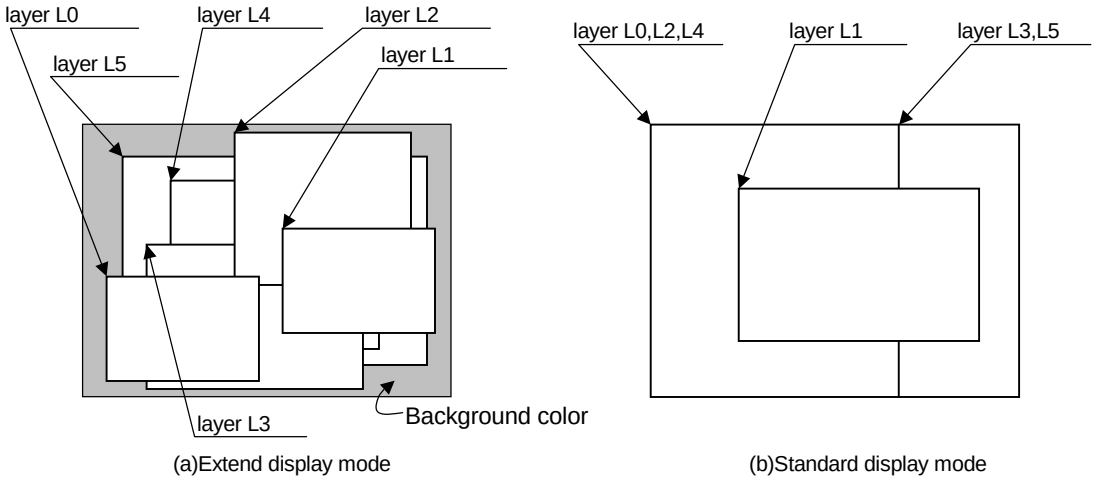


Figure 4.4.14 Displayed image of Standard Display Mode and Extend Display Mode

4.4.15 GdcDispDisplayLayerMode [Sets layer display mode]

Interface

GDC_BOOL **GdcDispDisplayLayerMode** (GDC_UCHAR *layer*, GDC_ULONG *mode*)

Arguments

layer Layer selection

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

mode Layer display mode

GDC_STANDARD_MODE	Standard Display Mode (default)
GDC_OVERLAY_EXT_MODE	Extend Overlay Mode
GDC_WINDOW_MODE	Window Mode
GDC_EXTEND_MODE	Extend Display Mode

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

MB86293 or later

Description

Changes display mode in each layer.

In extend display mode setting, window mode and an extend overlay mode can be set at the same time. **GDC_EXTEND_MODE** has the following meanings.

GDC_EXTEND_MODE = **GDC_WINDOW_MODE** |
GDC_OVERLAY_EXT_MODE

When layer L5 is used as a blend coefficient layer, this layer must be set extend display mode.

4.4.16 GdcDispSetBackColor [Sets background color]

Interface

GDC_BOOL **GdcDispSetBackColor** (GDC_COL24 *color*)

Arguments

color	24 bits background color

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

None

Target GDC

MB86293 or later

Description

Sets background color. This color is used to fill area where outside of layers.

This command is available in all display modes.

4.4.17 GdcDispSetLayerWindow [Sets position and size of the window mode layer]

Interface

GDC_BOOL **GdcDispSetLayerWindow** (GDC_UCHAR *layer*,
GDC_USHORT *x*, GDC_USHORT *y*,
GDC_USHORT *w*, GDC_USHORT *h*)

Arguments

layer Layer selection

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

x x coordinates in the device coordinates, range 0-4095

y y coordinates in the device coordinates, range 0-4095

w Window width, in pixels, range 1-4096

h Window height, in pixels, range 1-4096

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified

Target GDC

MB86293 or later

Description

Sets x, y position, width and height of the layer which has set up to Window Mode.

Setting to layer L1 is also available by the **GdcDispTimingWindow** command.

Each value of parameter are not checked of the range, specify a value within the decided range.

4.4.18 GdcLayerOverlayPriorityMode [Sets overlay display mode in every layer]

Interface

GDC_BOOL **GdcLayerOverlayPriorityMode** (GDC_UCHAR *layer*,
GDC_UCHAR *mode*)

Arguments

layer Layer selection

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

mode Overlay display mode

GDC_OVERLAY_PRIORITY	Overlay with transparent color (default)
GDC_OVERLAY_BLEND	Overlay with blend

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

MB86293 or later

Description

Sets overlay display mode of each layer.

To use overlay with transparent color (**GDC_OVERLAY_PRIORITY**), set to Extend Display Mode or Extend Overlay Mode by the **GdcDispDisplayLayerMode** command.

To use overlay with blend (**GDC_OVERLAY_BLEND**), set to Extend Display Mode or Extend Overlay Mode by the **GdcDispDisplayLayerMode** command, and set to Blend Mode by the **GdcLayerOverlayBlend** command.

If Standard Display Mode or Window Mode is selected, this command is not effective.

Setting of layer C by the **GdcOverlayPriorityMode** command reflects on layer L0.

On the contrary, setting to layer L0 by this command refracts on layer C.

When layer L5 is used as a blend coefficient layer, this layer is not target of overlaying by transparent color or blend.

4.4.19 GdcLayerOverlayBlend [Sets blend mode in every layer]

Interface

GDC_BOOL **GdcLayerOverlayBlend** (GDC_UCHAR *layer*, GDC_UCHAR *select*, GDC_UCHAR *correct*, GDC_UCHAR *source*, GDC_UCHAR *blend*)

Arguments

layer Layer selection

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

select Blend calculating method

GDC_BLEND_CURRENT_RATIO

layer color * blend ratio +
lower layer color * (1 - blend ratio)

GDC_BLEND_ONE_MINUS_CURRENT_RATIO

layer color * (1 - blend ratio) +
lower layer color * blend ratio

correct Correction by 1/256 value

GDC_BLEND_NO_CORRECT

Uses the blend ratio

GDC_BLEND_CORRECT

When the blend ratio is not 0, add 1/256
(When using the blend ratio of 100%)

source Selects source data of the blend ratio

GDC_BLEND_RATIO_CONSTANT

Uses the fixed value as blend ratio specified by *blend*

GDC_BLEND_RATIO_L5

Uses pixel value of layer L5 for the blend ratio

blend Blend ratio coefficient, effective values are 0-255
(Used when *source* is **GDC_BLEND_RATIO_CONSTANT**)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

MB86293 or later

Description

Sets the blend mode in the case of using overlay with blend.

Setting by the **GdcOverlayBlend** command reflect on layer L0. *correct* and *source* are set to **GDC_BLEND_NO_CORRECT** and **GDC_BLEND_RATIO_CONSTANT** respectively when the mode is set by the **GdcOverlayBlend** command.

Setting of *source* is not valid when layer L5 is set.

When the layer L5 is used as a blend coefficient layer, set layer L5 to Extend Display Mode by the **GdcDispDisplayLayerMode** command.

When the layer L1 is capture mode and layer L5 is used as blend coefficient layer, layer L1 is not blended correctly.

The meaning of blend coefficient is as follows.

[Blend coefficient]

blend	blend coefficient
0x00	0
0x01	1/256
0x02	2/256
0x03	3/256
:	:
0xff	255/256

4.4.20 GdcDispLayerOrder [Sets layer display order]

Interface

GDC_BOOL **GdcDispLayerOrder** (GDC_UCHAR *layer0*, GDC_UCHAR *layer1*,
GDC_UCHAR *layer2*, GDC_UCHAR *layer3*,
GDC_UCHAR *layer4*, GDC_UCHAR *layer5*)

Arguments

layer0 Top level layer selection

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5
GDC_NO_LAYER	Not select layer

layer1 Selects the second level layer (as for the value, the same in case of layer0)

layer2 Selects the third level layer (as for the value, the same in case of layer0)

layer3 Selects the fourth level layer (as for the value, the same in case of layer0)

layer4 Selects the fifth level layer (as for the value, the same in case of layer0)

layer5 Selects the sixth level layer (as for the value, the same in case of layer0)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified

Target GDC

MB86293 or later

Description

Sets display order of layers. Refer to the Graphics Controller hardware specifications for details

Not to display some of layers, specify **GDC_NO_LAYER**.

In order to set up the display order of layers at pleasure, it is necessary to set to Extend Display Mode or Extend Overlay Mode by the **GdcDispDisplayLayerMode** command. If display mode is set to Standard Display Mode or Window Mode, restrictions occur in order of layers. For

details of that restrictions, refer to Table 4.4.14b Layer order of each display mode in Table 4.4.14 **GdcDispDisplayMode** [Sets display mode].

The same layer cannot be specified for *layer0* - *layer5*. The result when same layer has specified is not guaranteed.

When the L5 layer is used as a blend coefficient layer, this layer should not be selected.

Figure 4.4.20a and 4.4.20b are shown displayed image of layer overlay. Figure 4.4.20a shows displayed image when all layers are set to Standard Display Mode (display order is Layer L1, L0, L2, L3, L4, L5). Figure 4.4.20b shows when Layer L0 and L5 are Standard Display Mode and other layers are Extend Display Mode (display order is L1, L0, L2, **GDC_NO_LAYER**, L4, L5).

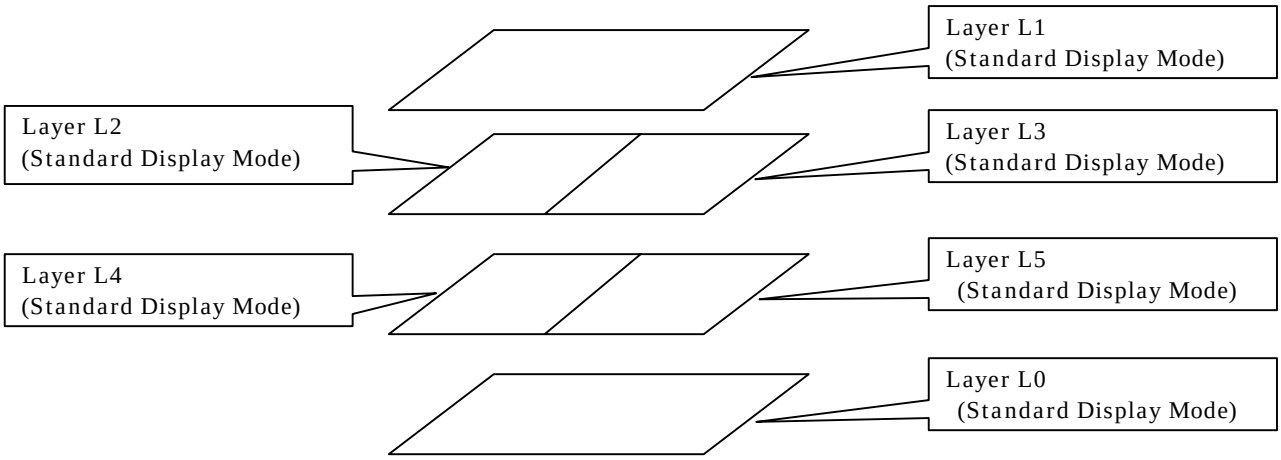


Figure 4.4.20a Displayed image of layer overlay (1)

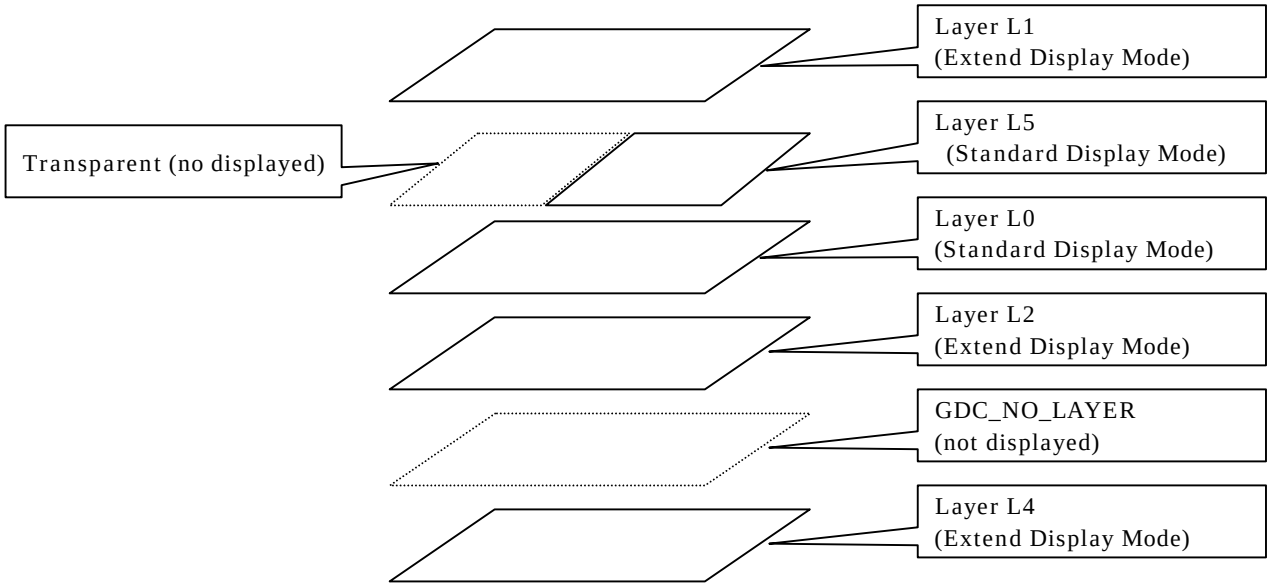


Figure 4.4.20b Displayed image of layer overlay (2)

4.5 Color Control Commands

4.5.1 GdcColorPalette [Sets palette colors]

Interface

GDC_BOOL **GdcColorPalette** (GDC_UCHAR *layer*, GDC_UCHAR *number*,
GDC_UCHAR *size*, const GDC_COL32 **lpColor*)

Arguments

layer Layer selection

GDC_C_LAYER_PALETTE	Layer C
GDC_MB_LAYER_PALETTE	Layer MB

[In MB86293 or later, following macros are available]

GDC_L0_LAYER_PALETTE	Layer L0 (same as Layer C)
GDC_L1_LAYER_PALETTE	Layer L1 (same as Layer MB)
GDC_L2_LAYER_PALETTE	Layer L2
GDC_L3_LAYER_PALETTE	Layer L3

number Top palette number, range 0-255

size Number of palette, range 0-255

lpColor Pointer to the color data

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified

Target GDC

All

Description

Sets color index code to palette table. If size is set to 0, all 256 entries of selected palette are set. Setting of palette is available without regard to display mode. However, layer L2 and L3 palettes are available only in Extend Display Mode (only in MB86293 or later).

4.5.2 GdcColorTransparent [Sets transparent color]

Interface

GDC_BOOL **GdcColorTransparent** (GDC_UCHAR *layer*, GDC_COLOR32 *color*)

Arguments

layer Layer selection

GDC_DISP_LAYER_C	Layer C
GDC_DISP_LAYER_ML	Layer ML
GDC_DISP_LAYER_MR	Layer MR

[In MB86293 or later, following macros are available]

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

color Transparent color code

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified

Target GDC

All

Description

Sets transparent color code.

L0, L2, L3 layers correspond to C, ML and MR layers.

The following values are used according to color mode of layer.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

Notes

In order to use color code 0 as transparent color, use the **GdcColorZeroMode** command.

4.5.3 GdcColorZeroMode [Sets color code 0 mode]

Interface

GDC_BOOL **GdcColorZeroMode** (GDC_UCHAR *layer*, GDC_UCHAR *mode*)

Arguments

layer Layer selection

GDC_DISP_LAYER_C	Layer C
GDC_DISP_LAYER_ML	Layer ML
GDC_DISP_LAYER_MR	Layer MR

[In MB86293 or later, following macros are available]

GDC_DISP_LAYER_L0	Layer L0
GDC_DISP_LAYER_L1	Layer L1
GDC_DISP_LAYER_L2	Layer L2
GDC_DISP_LAYER_L3	Layer L3
GDC_DISP_LAYER_L4	Layer L4
GDC_DISP_LAYER_L5	Layer L5

mode Color 0 mode

GDC_COLOR_NOTRANSPARENT
Not use as transparent color (default)

GDC_COLOR_TRANSPARENT
Use as transparent color

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

All

Description

Selects handling of color value 0 (palette number 0) from the following.

- Not use as transparent color, and use as normal color value (palette number)
- Use as transparent color

4.5.4 GdcChromaKeyMode [Sets Chroma-key mode]

Interface

GDC_BOOL **GdcChromaKeyMode** (GDC_UCHAR *mode*, GDC_UCHAR *source*)

Arguments

mode Chroma-key mode selection

GDC_ENABLE	Chroma-key operation enable
GDC_DISABLE	Chroma-key operation disable

source Source key color selection

GDC_CHROMAKEY_C	Layer C color
GDC_CHROMAKEY_DISP	Display color

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

All

Description

Sets whether Chroma-key is operated. When Chroma-key is operated, select target to be compared between layer C color and display color. When Chroma-key is not operated, setting of target of key color to be compared has no meaning.

4.5.5 GdcColorKey [Sets key color for Chroma-key]

Interface

GDC_BOOL **GdcColorKey** (GDC_COL16 *color*)

Arguments

color	Key color for Chroma-key operation

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

None

Target GDC

All

Description

Sets the key color for Chroma-key operation. The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*

4.5.6 GdcColorPaletteOffset [Sets of the color palette offset]

Interface

GDC_BOOL **GdcColorPaletteOffset** (GDC_UCHAR *layer*, GDC_ULONG *sub_no*)

Arguments

layer Layer selection

GDC_C_LAYER_PALETTE	Layer C
GDC_MB_LAYER_PALETTE	Layer MB
GDC_L0_LAYER_PALETTE	Layer L0, same as Layer C
GDC_L1_LAYER_PALETTE	Layer L1, same as Layer MB
GDC_L2_LAYER_PALETTE	Layer L2
GDC_L3_LAYER_PALETTE	Layer L3

sub_no Sub palette number, serial number of 16 divided parts of 256 palette, range 0-15, and more than 16 are not valid

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_LAYER	Invalid layer has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

MB86293 or later

Description

Sets the sub palette number of the color palette.

Sub-palette number is the serial number of 16 divided parts of 256 palettes.

By switching sub-palette, colors used for a display can be changed at once.

[Example of usage of the **GdcColorPaletteOffset** command]

Example for changing color set number to each sub palette numbers is shown in Figure 4.5.6.

In this example, 16 sub palettes have been already assigned to layer L0 palette.

GdcColorPaletteOffset(**GDC_L0_LAYER_PALETTE**, “sub palette number”)

	Layer L0 palette	
sub palette number 0	palette 0	Palette 0 of sub palette number 0
	:	:
	:	:
	:	:
	palette 15	Palette 15 of sub palette number 0
sub palette number 1	palette 16	Palette 0 of sub palette number 1
	:	:
	:	:
	:	:
	palette 31	Palette 15 of sub palette number 1
	:	:
	:	:
	:	:
sub palette number 15	palette 240	Palette 0 of sub palette number 15
	:	:
	:	:
	:	:
	palette 255	Palette 15 of sub palette number 15

Figure 4.5.6 Example of usage of the GdcColorPaletteOffset command

4.6 Cursor Control Commands

4.6.1 GdcCursorAddress [Sets cursor pattern memory address]

Interface

GDC_BOOL **GdcCursorAddress** (GDC_UCHAR *numCursor*, GDC_ULONG *ladrs*)

Arguments

numCursor	Cursor number (0 or 1)
ladrs	Cursor pattern address, Offset from top of the Graphics Memory

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_CURSOR_NUMBER	Invalid cursor number has specified

Target GDC

All

Description

Sets the start address of the Graphics Memory where the cursor pattern is stored.

The dimension of a cursor pattern is 64 * 64 pixels. The color is 8 bits color only. Two cursors are available. Memory size required two cursor patterns are stored is 8192(= 64*64*1*2) bytes.

4.6.2 GdcCursorPattern [Sets cursor pattern]

Interface

GDC_BOOL **GdcCursorPattern** (GDC_UCHAR *numCursor*,
const GDC_COL8 **lpCursor*)

Arguments

numCursor	Cursor number (0 or 1)
lpCursor	Pointer to cursor pattern

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_CURSOR_NUMBER	Invalid cursor number has specified

Target GDC

All

Description

Sets a cursor pattern. Transfer a cursor pattern data in main memory pointed via *lpCursor* to the Graphics Memory that start address is designated by the **GdcCursorAddress** command.
Size of cursor pattern is fixed to 64*64 pixels.

4.6.3 GdcCursorDisplay [Controls cursor display]

Interface

GDC_BOOL **GdcCursorDisplay** (GDC_UCHAR *numCursor*, GDC_UCHAR *enable*)

Arguments

numCursor	Cursor number (0 or 1)	
enable	Cursor display enable/disable	
	GDC_ENABLE	Cursor enable
	GDC_DISABLE	Cursor disable (default)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_CURSOR_NUMBER	Invalid cursor number has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

All

Description

Controls cursor display enable/disable.

4.6.4 GdcCursorPos [Sets cursor display position]

Interface

GDC_BOOL **GdcCursorPos** (GDC_UCHAR *numCursor*,
GDC_USHORT *x*, GDC_USHORT *y*)

Arguments

numCursor	Cursor number (0 or 1)
x	x coordinates of cursor display position (upper left position)
y	y coordinates of cursor display position (upper left position)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_CURSOR_NUMBER	Invalid cursor number has specified

Target GDC

All

Description

Sets display position of cursor.

4.6.5 GdcCursorPriority [Sets cursor display priority mode]

Interface

GDC_BOOL **GdcCursorPriority** (GDC_UCHAR *numCursor*, GDC_UCHAR *mode*)

Arguments

numCursor	Cursor number (0 or 1)
mode	Cursor display priority mode

GDC_PRIORITY_C_LAYER	Prioritizes layer C
GDC_PRIORITY_CURSOR	Prioritizes cursor (default)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_CURSOR_NUMBER	Invalid cursor number has specified
GDC_ERR_INVALID_PARAMETER	Invalid parameter has specified

Target GDC

All

Description

Selects which is prioritized in display, layer C or cursor.

4.6.6 GdcCursorColorTransparent [Sets cursor transparent color]

Interface

GDC_BOOL **GdcCursorColorTransparent** (GDC_COL8 *color*)

Arguments

color	Color code which is used as transparent color

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

None

Target GDC

All

Description

Sets a transparent color code for cursor.

4.6.7 GdcCursorColorZeroMode [Sets cursor color code 0 mode]

Interface

GDC_BOOL **GdcCursorColorZeroMode** (GDC_UCHAR *mode*)

Arguments

mode Color code 0 mode

GDC_COLOR_NOTRANSPARENT
Not use as transparent color

GDC_COLOR_TRANSPARENT
Use as transparent color (default)

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_INVALID_PARAMETER Invalid parameter has specified

Target GDC

All

Description

Selects the color option applied for color code 0 in cursor pattern. Color code 0 is use as either transparent color or ordinary color code.

4.7 Display List Commands

4.7.1 XGdcSetDLBuf [Sets current DL Buffer]

Interface

GDC_ULONG **XGdcSetDLBuf** (GDC_CTX **drvctx*, GDC_ULONG *bufNo*)

Arguments

drvctx	Pointer to Context
bufNo	DL Buffer number

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_ILLEGAL_DL_BUF_NO	Illegal DL Buffer number has specified

Target GDC

All

Description

Sets current DL Buffer to the DL Buffer specified by *bufNo*.

Display List in DL Buffer specified by *bufNo* is canceled.

4.7.2 XGdcQueryCurrentDLBuf [Queries current DL Buffer]

Interface

GDC_ULONG **XGdcQueryCurrentDLBuf** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

The number of current DL Buffer

Target GDC

All

Description

Returns the number of current DL Buffer.

4.7.3 XGdcGetDLBufNum [Gets the number of DL Buffers]

Interface

GDC_ULONG **XGdcGetDLBufNum** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

The number of DL Buffers

Target GDC

All

Description

Returns the number of DL Buffers.

4.7.4 XGdcGetDLBufInfo [Gets DL Buffer structure information]

Interface

GDC_DLBUF_STRUCT *__XGdcGetDLBufInfo__ (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

Pointer to DL Buffer structure information

Target GDC

All

Description

Returns the pointer to DL Buffer structure information.

4.7.5 XGdcFlush [Transfers Display List in current DL Buffer]

Interface

GDC_ULONG **XGdcFlush** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

Bytes of transferred Display List

Target GDC

All

Description

Transfers Display List in current DL Buffer to the Graphics Controller.

This command returns immediately without waiting for the transmission when DMA or Local Display List Transfer is used. When transmission by CPU is used, this command returns after the transmission.

This command returns bytes of transferred Display List.

4.7.6 XGdcFlushEx [Transfers Display List in specified DL Buffer]

Interface

GDC_ULONG **XGdcFlushEx** (GDC_CTX **drvctx*, GDC_ULONG *bufNo*)

Arguments

drvctx	Pointer to Context
bufNo	The number of a DL Buffer

Return value

Bytes of transferred Display List

Target GDC

All

Description

Transfers Display List in the DL Buffer specified by *bufNo* to the Graphics Controller.

This command returns immediately without waiting for the transmission when DMA or Local Display List Transfer is used. When transmission by CPU is used, this command returns after the transmission.

This command returns bytes of transferred Display List.

4.7.7 XGdcCancelDisplayList [Cancels Display List]

Interface

void **XGdcCancelDisplayList** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

None

Target GDC

All

Description

Cancels Display List in current DL Buffer.

4.8 Drawing Frame Control Commands

4.8.1 XGdcDrawDimension [Sets drawing frame]

Interface

GDC_BOOL **XGdcDrawDimension** (GDC_CTX **drvctx*,
GDC_UCHAR *cmode*, GDC_ULONG *dadrs*,
GDC_USHORT *dw*, GDC_USHORT *dh*)

Arguments

drvctx Pointer to Context

cmode Color mode

GDC_24BPP_FORMAT	24 bits color mode
GDC_16BPP_FORMAT	16 bits color mode
GDC_8BPP_FORMAT	8 bits color mode

dadrs Drawing frame base address, specify offset from top of the Graphics Memory)

dw Drawing frame width, in pixels, range 64-4096 in 24bits color mode, range 32-4096 in 16 bits color mode, range 16-4096 in 8 bits color mode

dh Drawing frame height, pixel count, range 1-4096.

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_COLOR_MODE

Invalid color mode has specified

Target GDC

All

Description

Sets color mode and size of drawing frame.

4.8.2 XGdcSetZPrecision [Sets precision of Z value]

Interface

GDC_BOOL **XGdcSetZPrecision** (GDC_CTX **drvctx*, GDC_ULONG *mode*)

Arguments

drvctx	Pointer to Context	
mode	Precision of Z value	
	GDC_Z_16BIT	Precision of Z value is 16 bits (default)
	GDC_Z_8BIT	Precision of Z value is 8 bits

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets precision of Z value.

4.8.3 XGdcBufferCreateZ [Sets Z buffer base address]

Interface

GDC_BOOL **XGdcBufferCreateZ** (GDC_CTX **drvctx*, GDC_ULONG *zadrs*)

Arguments

drvctx	Pointer to Context
zadrs	Z buffer base address, specify offset from top of the Graphics Memory

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets the base address of Z buffer. The vertical/horizontal size of Z buffer is assumed to be equal to that of drawing frame.

When precision of Z value is 16 bits, memory size of 16 bits per 1 pixel is needed.

When precision of Z value is 8 bits, memory size of 8 bits per 1 pixel is needed.

4.8.4 XGdcBufferCreateC [Sets base address of polygon drawing control buffer]

Interface

GDC_BOOL **XGdcBufferCreateC** (GDC_CTX **drvctx*, GDC_ULONG *cadrs*)

Arguments

drvctx	Pointer to Context
cadrs	Polygon drawing control buffer base address, specify offset from top of the Graphics Memory

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets the base address of polygon drawing control buffer. The vertical/horizontal size of this control buffer is assumed to be equal to that of drawing frame. For each pixel, 1bit of data is required for this buffer.

4.8.5 XGdcBufferClearZ [Clears Z buffer]

Interface

GDC_BOOL **XGdcBufferClearZ** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Clears Z buffer. Prior to hidden surface elimination, Z buffer should be cleared.

Notes

Execute the **XGdcDrawDimension** command before the calling this command because size of Z buffer is determined by the **XGdcDrawDimension** command.

4.8.6 XGdcBufferClearC [Clears polygon drawing control buffer]

Interface

GDC_BOOL **XGdcBufferClearC** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Clears polygon drawing control buffer.

4.8.7 XGdcDrawClipFrame [Sets drawing clip border]

Interface

GDC_BOOL **XGdcDrawClipFrame** (GDC_CTX **drvctx*,
GDC_USHORT *x0*, GDC_USHORT *y0*,
GDC_USHORT *x1*, GDC_USHORT *y1*)

Arguments

drvctx	Pointer to Context
x0	x coordinates of left top edge of clip border, range 0-4095
y0	y coordinates of left top edge of clip border, range 0-4095
x1	x coordinates of right bottom edge of clip border, range 0-4095
y1	y coordinates of right bottom edge of clip border, range 0-4095

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets clip border of drawing. Clip border is set as a Blt located relatively from the base point of drawing frame. Drawing to the area outside of this clip border is not performed.

4.8.8 XGdcSetAlphaMapBase [Sets base address of alpha map area]

Interface

GDC_BOOL **XGdcSetAlphaMapBase** (GDC_CTX **drvctx*, GDC_ULONG *adrs*)

Arguments

drvctx	Pointer to Context
adrs	Alpha map area base address, specify offset from top of the Graphics Memory

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets the base address of alpha map area.

Alpha map area is an alpha coefficient area to be used by the **XGdcBltCopyAltAlpha** command. An offset from the Graphics Memory top must be set to *adrs*.

For details about alpha map, refer to hardware specifications of the Graphics Controller.

4.9 Primitive Drawing Commands for Device Coordinate System

4.9.1 XGdcPrimType [Starts drawing procedure]

Interface

GDC_BOOL **XGdcPrimType** (GDC_CTX **drvctx*, GDC_UCHAR *type*)

Arguments

drvctx	Pointer to Context	
type	Primitive type	
	GDC_POINTS	Points
	GDC_LINES	Lines
	GDC_POLYLINE	Poly-line
	GDC_LINES_FAST	Fast 2D lines
	GDC_POLYLINE_FAST	Fast 2D poly-line
	GDC_TRIANGLES	Triangles
	GDC_TRIANGLE_STRIP	Triangle strip
	GDC_TRIANGLE_FAN	Triangle fan
	GDC_POLYGON	Polygon
	GDC_TRIANGLES_FAST	Fast 2D triangles
	GDC_TRIANGLE_STRIP_FAST	Fast 2D triangle strip
	GDC_TRIANGLE_FAN_FAST	Fast 2D triangle fan

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_NOT_READY	“Graphics Driver” is not initialized
GDC_ERR_INVALID_PRIMITIVE	Invalid primitive has specified

Target GDC

All

Description

Sets the primitive type to be drawn by the **XGdcDrawVertex2D[i]** or the **XGdcDrawVertex-3D[f]** command. Once either of these commands is executed, same type of primitive will keep being drawn till the **XGdcPrimEnd** command will be executed.

4.9.2 XGdcPrimEnd [Completes drawing procedure]

Interface

GDC_BOOL **XGdcPrimEnd** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Stops drawing the primitive applied by the **XGdcPrimType** command.

4.9.3 XGdcTexCoord2D[f/Nf] [Sets coordinates of 2D texture]

Interface

void **XGdcTexCoord2D** (GDC_CTX **drvctx*, GDC_FIXED32 *u*, GDC_FIXED32 *v*)

void **XGdcTexCoord2Df** (GDC_CTX **drvctx*, GDC_SFLOAT *u*, GDC_SFLOAT *v*)

void **XGdcTexCoord2DNf** (GDC_CTX * *drvctx*, GDC_SFLOAT *u*, GDC_SFLOAT *v*)

Arguments

drvctx	Pointer to Context
u	u coordinates of texture mapped on the vertex
v	v coordinates of texture mapped on the vertex

Return value

None

Target GDC

All

Description

Sets the texture coordinates for the vertex to be drawn by the vertex coordinate setting command. Once this command is executed, the same texture coordinates is continuously applied till this command will be executed.

The **XGdcTexCoord2D** command must be used when the type of texture coordinate is GDC_FIXED32.

The **XGdcTexCoord2Df** command must be used when the type of texture coordinate is GDC_SFLOAT.

The **XGdcTexCoord2DNf** command must be used when the type of texture coordinate is GDC_SFLOAT and normalized. In this case, the range of texture coordinate must be within 0.0 to 1.0. The minimum size of texture coordinate is 0.0 and the maximum size is 1.0.

These commands are applicable to the following primitives. If these commands are used except the following primitives, the result is not guaranteed.

GDC_TRIANGLES

GDC_TRIANGLE_STRIP

GDC_TRIANGLE_FAN

4.9.4 XGdcTexCoord3D[f/Nf] [Sets coordinates of 3D texture]

Interface

void **XGdcTexCoord3D** (GDC_CTX **drvctx*,
GDC_FIXED32 *u*, GDC_FIXED32 *v*, GDC_FIXED32 *rw*)

void **XGdcTexCoord3Df** (GDC_CTX * *drvctx*,
GDC_SFLOAT *u*, GDC_SFLOAT *v*, GDC_SFLOAT *rw*)

void **XGdcTexCoord3DNf** (GDC_CTX * *drvctx*,
GDC_SFLOAT *u*, GDC_SFLOAT *v*, GDC_SFLOAT *rw*)

Arguments

drvctx	Pointer to Context
u	u coordinates of texture mapped on the vertex
v	v coordinates of texture mapped on the vertex
rw	Reciprocal of w coordinates of texture mapped on the vertex

Return value

None

Target GDC

All

Description

Sets the texture coordinates for the vertex to be drawn by the vertex coordinate setting command. Once this command is executed, the same texture coordinates is continuously applied till this command will be executed.

The **XGdcTexCoord3D** command must be used when the type of texture coordinate is GDC_FIXED32.

The **XGdcTexCoord3Df** command must be used when the type of texture coordinate is GDC_SFLOAT.

The **XGdcTexCoord3DNf** command must be used when the type of texture coordinate is GDC_SFLOAT and normalized. In this case, the range of texture coordinate must be within 0.0 to 1.0. The minimum size of texture coordinate is 0.0 and the maximum size is 1.0.

These commands are applicable to the following primitives. If these commands are used except the following primitives, the result is not guaranteed.

GDC_TRIANGLES
GDC_TRIANGLE_STRIP
GDC_TRIANGLE_FAN

4.9.5 XGdcDrawVertex2D[i] [Sets coordinates of 2D vertex]

Interface

GDC_BOOL **XGdcDrawVertex2D** (GDC_CTX **drvctx*,
GDC_FIXED32 *x*, GDC_FIXED32 *y*)

GDC_BOOL **XGdcDrawVertex2Di** (GDC_CTX * *drvctx*,
GDC_LONG *x*, GDC_LONG *y*)

Arguments

drvctx	Pointer to Context
x	x coordinates of 2D vertex
y	y coordinates of 2D vertex

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets 2D vertex coordinates and drawing a designated primitive.

Current values of color and texture coordinates are used in drawing, which has been set by the drawing color setting command and texture coordinates setting command respectively.

The **XGdcDrawVertex2D** command must be used when the type of vertex coordinate is GDC_FIXED32.

The **XGdcDrawVertex2Di** command must be used when the type of vertex coordinate is GDC_LONG.

The **XGdcDrawVertex2Di** command is applicable to the following primitives. If this command is used except the following primitives, the result is not guaranteed.

GDC_LINES_FAST
GDC_POLYLINE_FAST
GDC_POLYGON
GDC_TRIANGLES_FAST
GDC_TRIANGLE_STRIP_FAST

GDC_TRIANGLE_FAN_FAST

4.9.6 XGdcDrawVertex3D[f] [Sets coordinates of 3D vertex]

Interface

GDC_BOOL **XGdcDrawVertex3D** (GDC_CTX **drvctx*,
GDC_FIXED32 *x*, GDC_FIXED32 *y*, GDC_USHORT *z*)

GDC_BOOL **XGdcDrawVertex3Df** (GDC_CTX * *drvctx*,
GDC_SFLOAT *x*, GDC_SFLOAT *y*, GDC_SFLOAT *z*)

Arguments

drvctx	Pointer to Context
x	x coordinates of 3D vertex
y	y coordinates of 3D vertex
z	z coordinates of 3D vertex

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets 3D vertex coordinates and drawing a designated primitive.

Current values of color and texture coordinates are used in drawing, which has been set by drawing color setting command and texture coordinates setting command respectively.

The **XGdcDrawVertex3D** command must be used when the type of vertex coordinate is GDC_FIXED32.

The **XGdcDrawVertex3Df** command must be used when the type of vertex coordinate is GDC_SFLOAT.

The **XGdcDrawVertex3Df** command is applicable to the following primitives. If this command is used except the following primitives, the result is not guaranteed.

GDC_TRIANGLES

GDC_TRIANGLE_STRIP

GDC_TRIANGLE_FAN

When drawing a polygon primitive (**GDC_POLYGON**), z coordinates of a parameter is ignored.

4.9.7 XGdcDrawPrimitive [Draws multiple 3D triangles]

Interface

GDC_BOOL **XGdcDrawPrimitive** (GDC_CTX **drvctx*, GDC_ULONG *type*,
const GDC_VERTEX **lpVertices*,
int *count*)

Arguments

drvctx	Pointer to Context
type	Primitive type
	GDC_TRIANGLES Triangles
	GDC_TRIANGLE_STRIP Triangle strip
	GDC_TRIANGLE_FAN Triangle fan
lpVertices	Pointer to vertex parameter list (coordinates, color texture coordinates)
count	Number of vertices, 3 or more

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_ILLEGAL_VERTEX_COUNT

Illegal number of vertex

Target GDC

All

Description

Draws a primitive specified in the type formed with multiple vertices designated by *lpVertices*.

4.10 Primitive Drawing Control Commands for Object Coordinate System

4.10.1 XGdcGeoPrimType [Starts drawing procedure]

Interface

GDC_BOOL **XGdcGeoPrimType** (GDC_CTX **drvctx*, GDC_UCHAR *type*)

Arguments

drvctx	Pointer to Context	
type	Primitive type	
	GDC_POINTS	Points
	GDC_LINES	Lines
	GDC_POLYLINE	Poly-line
	GDC_TRIANGLES	Triangles
	GDC_TRIANGLE_STRIP	Triangle strip
	GDC_TRIANGLE_FAN	Triangle fan
	GDC_POLYGON	Polygon

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_INVALID_PRIMITIVE	Invalid primitive has specified

Target GDC

MB86291 or later

Description

Sets primitive drawn with the **XGdcGeoDrawVertex2D[f/i]** or the **XGdcGeoDrawVertex3D[f/i]**.

Once this command is executed, the same primitive is drawn until the **XGdcGeoPrimEnd** command is executed.

In MB86293 or later, to draw triangles with 8 bits Gouraud-shading, proceed in the following procedures. (The drawing of except the following procedures is not guaranteed.)

(1) Set the following attributes before the calling of this command

- Set 4x4 matrix to convert from object coordinates to clip coordinates by the **XGdcGeoLoadMatrix[f]** command
- Set color mode to 8bpp by the **XGdcDrawDimension** command

- Set shading mode to Gouraud-shading by the **XGdcSetAttrSurf** command
- Disable texture mapping/tiling by the **XGdcSetAttrSurf** command

(2) Call this command, specify any of the following for *type*

- **GDC_TRIANGLES**
- **GDC_TRIANGLE_STRIP**
- **GDC_TRIANGLE_FAN**

(3) Set 8bits color of each vertex by the **XGdcVertexColor32** command

(4) Set coordinates of each vertex by the **XGdcGeoDrawVertex2D** or the **XGdcGeoDrawVertex2Di** command

(5) Call the **XGdcGeoPrimEnd** command

4.10.2 XGdcGeoPrimEnd [Completes drawing procedure]

Interface

GDC_BOOL **XGdcGeoPrimEnd** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_NOT_START_GEO_PRIM_TYPE

The **XGdcGeoPrimEnd** command has called before the call of the **XGdcGeoPrimType** command and the **XGdcGeoDrawVertex** command

GDC_ERR_NOT_START_GEO_DRAW_VERTEX

The **XGdcGeoPrimEnd** command has called before the call of the **XGdcGeoDrawVertex** command

Target GDC

MB86291 or later

Description

Terminates a series of processes to draw primitives following the **XGdcGeoPrimType** command.

4.10.3 XGdcGeoDrawVertex2D[f/i] [Sets XY coordinates of vertex]

Interface

GDC_BOOL **XGdcGeoDrawVertex2D** (GDC_CTX **drvctx*,
GDC_FIXED32 *x*, GDC_FIXED32 *y*)

GDC_BOOL **XGdcGeoDrawVertex2Df** (GDC_CTX * *drvctx*,
GDC_SFLOAT *x*, GDC_SFLOAT *y*)

GDC_BOOL **XGdcGeoDrawVertex2Di** (GDC_CTX * *drvctx*,
GDC_LONG *x*, GDC_LONG *y*)

Arguments

drvctx	Pointer to Context
x	x coordinates of the vertex
y	y coordinates of the vertex

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_NOT_START_GEO_PRIM_TYPE

The **XGdcGeoPrimEnd** command has called before the call of the **XGdcGeoPrimType** command and the **XGdcGeoDrawVertex** command

Target GDC

MB86291 or later

Description

Specifies a vertex coordinates in object coordinates and drawing a primitive currently set. In this case, z is treated as zero.

Current values of color and texture coordinates are used in drawing, which has been set by drawing color setting command and texture coordinates setting command respectively.

The **XGdcGeoDrawVertex2D** command must be used when the type of vertex coordinate is GDC_FIXED32.

The **XGdcGeoDrawVertex2Df** command must be used when the type of vertex coordinate is GDC_SFLOAT.

The **XGdcGeoDrawVertex2Di** command must be used when the type of vertex coordinate is GDC_LONG.

4.10.4 XGdcGeoDrawVertex3D[f/i] [Sets XYZ coordinates of vertex]

Interface

GDC_BOOL **XGdcGeoDrawVertex3D** (GDC_CTX **drvctx*,
GDC_FIXED32 *x*,GDC_FIXED32 *y*, GDC_FIXED32 *z*)

GDC_BOOL **XGdcGeoDrawVertex3Df** (GDC_CTX * *drvctx*,
GDC_SFLOAT *x*, GDC_SFLOAT *y*, GDC_SFLOAT *z*)

GDC_BOOL **XGdcGeoDrawVertex3Di** (GDC_CTX * *drvctx*,
GDC_LONG *x*, GDC_LONG *y*, GDC_FIXED32 *z*)

Arguments

drvctx	Pointer to Context
x	x coordinates of the vertex
y	y coordinates of the vertex
z	z coordinates of the vertex

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_NOT_START_GEO_PRIM_TYPE

The **XGdcGeoPrimEnd** command has called before the call of the **XGdcGeoPrimType** command and the **XGdcGeoDrawVertex** command

Target GDC

MB86291 or later

Description

Sets vertex coordinates in object coordinates and drawing a primitive currently set. In this case, z is treated as zero.

Current values of color and texture coordinates are used in drawing, which has been set by drawing color setting command and texture coordinates setting command respectively.

The **XGdcGeoDrawVertex3D** command must be used when the type of vertex coordinate is GDC_FIXED32.

The **XGdcGeoDrawVertex3Df** command must be used when the type of vertex coordinate is GDC_SFLOAT.

The **XGdcGeoDrawVertex3Di** command must be used when the type of vertex coordinate is GDC_LONG.

Notes

When drawing a polygon primitive (**GDC_POLYGON**) by MB86291/86292, z coordinates of a parameter is ignored.

4.10.5 XGdcGeoTexCoord2D[N/Nf] [Sets texture coordinates]

Interface

void **XGdcGeoTexCoord2DN** (GDC_CTX **drvctx*,
GDC_FIXED32 *u*, GDC_FIXED32 *v*)

void **XGdcGeoTexCoord2DNf** (GDC_CTX * *drvctx*,
GDC_SFLOAT *u*, GDC_SFLOAT *v*)

Arguments

drvctx	Pointer to Context
u	Texture u coordinates of the vertex
v	Texture v coordinates of the vertex

Return value

None

Target GDC

MB86291 or later

Description

Sets a texture coordinates (2 dimensions) of vertex in drawing with the vertex coordinates setting command. Once this command is executed, the same texture coordinates is used in drawing unless texture coordinates is changed by this command. This command treat texture coordinates as normalized (1.0 is maximum size of current texture).

The **XGdcGeoTexCoord2DN** command must be used when the type of texture coordinate is GDC_FIXED32.

The **XGdcGeoTexCoord2DNf** command must be used when the type of texture coordinate is GDC_SFLOAT.

These commands are applicable to the following primitives. If these commands are used except the following primitives, the result is not guaranteed.

GDC_TRIANGLES

GDC_TRIANGLE_STRIP

GDC_TRIANGLE_FAN

However, MB86293 or later can also be used the following primitives:

GDC_POLYGON

4.10.6 XGdcVertexColor[32/3f] [Sets color of vertex]

Interface

void **XGdcVertexColor32** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

void **XGdcVertexColor3f** (GDC_CTX * *drvctx*,
GDC_SFLOAT *r*, GDC_SFLOAT *g*, GDC_SFLOAT *b*)

Arguments

drvctx Pointer to Context

color [In 16/24 bits color mode]
Packed format in which each color elements (r,g,b) is normalized to [0,255]. In this case, r,g,b are 8 bits respectively. (Figure 4.10.6a)

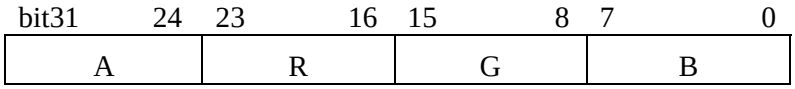


Figure 4.10.6a 16/24 bits vertex color format

[In 8bits color mode]
Specify 8bits color code in the following format.

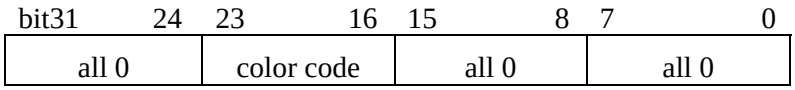


Figure 4.10.6b 8bits vertex color format

r, g, b Normalized values in which each color elements (r,g,b) are normalized to [0,1].

Return value

None

Target GDC

MB86291 or later, 8 bits Gouraud shading is only MB86293 or later

Description

Sets a color of vertex. Once this command is executed, the same color is used in drawing for object coordinates unless the color is changed by this command.

When you work 8 bits Gouraud shading, use the **XGdcVertexColor32** command. (The **XGdcVertexColor3f** command is not valid)

This command is used when shading mode is smooth shading. If the shading mode is flat shading, use the **XGdcColor** command.

The **XGdcVertexColor32** command must be used when the type of vertex color is GDC_COLOR32.

The **XGdcVertexColor3f** command must be used when the type of vertex color is GDC_SFLOAT.

When drawing a polygon (**GDC_POLYGON**), setup of this command is ignored.

4.11 Drawing Attribute Control Commands

4.11.1 XGdcColor [Sets vertex color/foreground color]

Interface

GDC_BOOL **XGdcColor** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Vertex and foreground color

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets vertex color and foreground color applied for bitmap drawing and broken line drawing to be executed by set coordinates of vertex command. Once this command is executed, the same color is continuously applied till this command will be executed.

The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

4.11.2 XGdcBackColor [Sets background color]

Interface

GDC_BOOL **XGdcBackColor** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Background color

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets background color applied for binary pattern drawing and broken line drawing. Once this command is executed, the same color is continuously applied till this command will be executed.

The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

In order to make background color transparent, sets the following bit to 1 according to color mode.

- 8 bits color mode : bit 15 of *color*
- 16 bits color mode : bit 15 of *color*
- 24 bits color mode : bit 31 of *color*

4.11.3 XGdcClipMode [Sets clipping mode]

Interface

GDC_BOOL **XGdcClipMode** (GDC_CTX **drvctx*, GDC_ULONG *mode*)

Arguments

drvctx	Pointer to Context
mode	Clipping mode

GDC_CLIP_X_ON	Validates clipping toward x axis
GDC_CLIP_Y_ON	Validates clipping toward y axis
GDC_CLIP_DISABLE	Invalidates clipping

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets clipping mode.

GDC_CLIP_X_ON and **GDC_CLIP_Y_ON** can be set at the same time with OR operator.

4.11.4 XGdcSetAttrLine [Sets line drawing attribute]

Interface

GDC_BOOL **XGdcSetAttrLine** (GDC_CTX **drvctx*,
GDC_ULONG *target*, GDC_ULONG *param*)

Arguments

drvctx Pointer to Context

target Line drawing attribute

GDC_DEPTH_TEST	Z value compare mode
GDC_DEPTH_FUNC	Z value compare function
GDC_DEPTH_WRITE_MASK	Z value write permission mask
GDC_BLEND_MODE	Blending mode
GDC_BROKEN_LINE	Broken line mode
GDC_LINE_WIDTH	Line width
GDC_ANTI_ALIAS	Antialias option
GDC_LINE_ENDPOINT	End of the line control
GDC_ROP_MODE	Logical operation

[In MB86291 or later, following macros are available]

GDC_BROKEN_LINE_OFFSET
Offset control of broken line pattern

GDC_BROKEN_LINE_PERIOD
Period set of broken line pattern

[In MB86293 or later, following macros are available]

GDC_SHADOW_DEPTH_TEST
Z value compare mode of shadow

GDC_SHADOW_DEPTH_FUNC
Z value compare function of shadow

GDC_SHADOW_DEPTH_WRITE_MASK
Z value write permission mask of shadow

GDC_SHADOW_BLEND_MODE
Blending mode of shadow

GDC_SHADOW_BROKEN_LINE
Broken line mode of shadow

GDC_SHADOW_LINE_WIDTH
Line width of shadow

GDC_SHADOW_BROKEN_LINE_PERIOD
Period set of broken line pattern of shadow

GDC_SHADOW_ROP_MODE
Logical operation of shadow

GDC_BORDER_DEPTH_TEST
Z value compare mode of border

GDC_BORDER_DEPTH_FUNC
Z value compare function of border

GDC_BORDER_DEPTH_WRITE_MASK
Z value write permission mask of border

GDC_BORDER_BLEND_MODE
Blending mode of border

GDC_BORDER_BROKEN_LINE
Broken line mode of border

GDC_BORDER_LINE_WIDTH
Line width of border

GDC_BORDER_BROKEN_LINE_PERIOD
Period set of broken line pattern of border

GDC_BORDER_ROP_MODE
Logical operation of border

param Parameter corresponding to *target* (*1)

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED
The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_ILLEGAL_LINE_WIDTH
Illegal width of line

GDC_ERR_INVALID_ATTRIBUTE
Invalid attribute has specified

Target GDC

All

Description

Sets attribute for drawing lines.

Notes

When 8 bits color mode is used in drawing frame, logical operations do not use actual color

(GDC_COL32 format color), but use color code (GDC_COL8 format). For example, when the pixel color (palette color code) of drawing frame is value 1 and drawing color (palette color code) is value 2, **GDC_ROP_OR** operation is stored value 3 in drawing frame.

*1 Line drawing attribute (*target*) and parameter (*param*) corresponding to each line drawing attribute is shown below.

[Explanatory notes]

Line drawing attribute	Description of line drawing attribute
Parameter 1 that can set	Description of parameter 1
Parameter 2 that can set	Description of parameter 2
:	:
GDC_DEPTH_TEST	Sets Z value compare mode.
GDC_ENABLE	Validates Z value comparison.
GDC_DISABLE	Invalidates Z value comparison.
GDC_DEPTH_FUNC	Selects Z value comparison type.
GDC_DEPTH_NEVER	Always NOT drawn.
GDC_DEPTH_ALWAYS	Always drawn.
GDC_DEPTH_LESS	Drawn if current Z value is less than Z buffer value.
GDC_DEPTH_LEQUAL	Drawn if current Z value equal to or less than Z buffer value.
GDC_DEPTH_EQUAL	Drawn if current Z value equal to Z buffer value.
GDC_DEPTH_GEQUAL	Drawn if current Z value equal to or more than Z buffer value.
GDC_DEPTH_GREATER	Drawn if current Z value more than Z buffer value.
GDC_DEPTH_NOTEQUAL	Drawn if current Z value is not equal to Z buffer value.
GDC_DEPTH_WRITE_MASK	Enables write access to Z buffer. If **GDC_ENABLE**, according to the result of Z value comparison, Z value is written to Z buffer.
GDC_ENABLE	Disable Z buffer write.
GDC_DISABLE	Enable Z buffer write.

GDC_BLEND_MODE	Sets blending mode of pixel write.
GDC_BLEND_COPY	Regular drawing operation (writes pixel color to drawing frame).
GDC_BLEND_ALPHA	Enables alpha blending.
GDC_BLEND_ROP	Draws with logical arithmetic.
GDC_BROKEN_LINE	Selects broken line mode.
GDC_ENABLE	Draws a broken line utilizing applied line pattern.
GDC_DISABLE	Draws a solid line.
GDC_LINE_WIDTH	Sets line width.
GDC_LINE_WIDTH_1	Draws a line of 1 pixel width.
GDC_LINE_WIDTH_2	Draws a line of 2 pixels width.
:	:
GDC_LINE_WIDTH_32	Draws a line of 32 pixels width.
GDC_ANTI_ALIAS	Sets antialias mode.
GDC_ENABLE	Enables antialias operation.
GDC_DISABLE	Disables antialias operation.
GDC_LINE_ENDPOINT	Controls the end point of line in **GDC_LINES** and **GDC_LINES_FAST** commands. End point is not drawn in **GDC_POLYLINE** and **GDC_POLYLINE_FAST** of the **XGdcPrimType** command regardless of this setting. In **GDC_POLYLINE** of the **XGdcGeoPrimType** command, this setting is available.
GDC_ENABLE	Draws the end point.
GDC_DISABLE	NOT draws the end point.
GDC_BROKEN_LINE_OFFSET	Specifies the way of drawing broken line (only for MB86291 or later).
GDC_ENABLE	Starts new drawing broken line pattern.
GDC_DISABLE	Continues from the last drawing broken line pattern.

GDC_BROKEN_LINE_PERIOD	Sets broken line pattern period (only for MB86291 or later).
GDC_BROKEN_LINE_32	32 bits period.
GDC_BROKEN_LINE_24	24 bits period.

GDC_ROP_MODE	Sets logical operation mode of the body
GDC_ROP_CLEAR	Sets all bits to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	Sets all bits to 1

s: drawing color
d: destination color

GDC_SHADOW_DEPTH_TEST	Sets Z value compare mode of shadow (only for MB86293 or later).
GDC_ENABLE	Validates Z value comparison.
GDC_DISABLE	Invalidates Z value comparison.

GDC_SHADOW_DEPTH_FUNC	Selects Z value comparison type of shadow (only for MB86293 or later).
GDC_DEPTH_NEVER	Always NOT drawn.
GDC_DEPTH_ALWAYS	Always drawn.
GDC_DEPTH_LESS	Drawn if current Z value is less than Z buffer value.
GDC_DEPTH_LEQUAL	Drawn if current Z value equal to or less than Z buffer value.
GDC_DEPTH_EQUAL	Drawn if current Z value equal to Z buffer value.
GDC_DEPTH_GEQUAL	Drawn if current Z value equal to or more than Z buffer value.
GDC_DEPTH_GREATER	Drawn if current Z value more than Z buffer value.
GDC_DEPTH_NOTEQUAL	Drawn if current Z value is not equal to Z buffer value.

GDC_SHADOW_DEPTH_WRITE_MASK	Enables write access to Z buffer of shadow (only for MB86293 or later).
GDC_ENABLE	Disable Z buffer write.
GDC_DISABLE	Enable Z buffer write.

GDC_SHADOW_BLEND_MODE	Sets blending mode of pixel write of shadow (only for MB86293 or later).
GDC_BLEND_COPY	Regular drawing operation (writes pixel color to drawing frame).
GDC_BLEND_ALPHA	Enables alpha blending.
GDC_BLEND_ROP	Draws with logical arithmetic.

GDC_SHADOW_BROKEN_LINE	Selects broken line mode of shadow (only for MB86293 or later).
GDC_ENABLE	Draws a broken line utilizing applied line pattern.
GDC_DISABLE	Draws a solid line.

GDC_SHADOW_LINE_WIDTH	Sets line width of shadow (only for MB86293 or later).
GDC_LINE_WIDTH_1	Draws a line of 1 pixel width.
GDC_LINE_WIDTH_2	Draws a line of 2 pixel width.
:	:
GDC_LINE_WIDTH_32	Draws a line of 32 pixel width.

GDC_SHADOW_BROKEN_LINE_PERIOD

Sets broken line pattern period of shadow (only for MB86293 or later).

GDC_BROKEN_LINE_32	32 bits period.
GDC_BROKEN_LINE_24	24 bits period.

GDC_SHADOW_ROP_MODE	Sets logical operation mode of the shadow. Same logical operation mode is applied to Shadow primitives and shadow composition primitives. In the portion with which shadow primitive and shadow composition primitive overlap, logical operation works between shadow and shadow composition.
GDC_ROP_CLEAR	Sets all bits to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	Sets all bits to 1

s: drawing color
d: destination color

GDC_BORDER_DEPTH_TEST	Sets Z value compare mode of border (only for MB86293 or later).
GDC_ENABLE	Validates Z value comparison.
GDC_DISABLE	Invalidates Z value comparison.

GDC_BORDER_DEPTH_FUNC	Selects Z value comparison type of border (only for MB86293 or later).
GDC_DEPTH_NEVER	Always NOT drawn.
GDC_DEPTH_ALWAYS	Always drawn.
GDC_DEPTH_LESS	Drawn if current Z value is less than Z buffer value.
GDC_DEPTH_LEQUAL	Drawn if current Z value equal to or less than Z buffer value.
GDC_DEPTH_EQUAL	Drawn if current Z value equal to Z buffer value.
GDC_DEPTH_GEQUAL	Drawn if current Z value equal to or more than Z buffer value.
GDC_DEPTH_GREATER	Drawn if current Z value more than Z buffer value.
GDC_DEPTH_NOTEQUAL	Drawn if current Z value is not equal to Z buffer value.

GDC_BORDER_DEPTH_WRITE_MASK	
	Enables write access to Z buffer of border (only for MB86293 or later).
GDC_ENABLE	Disable Z buffer write.
GDC_DISABLE	Enable Z buffer write.

GDC_BORDER_BLEND_MODE	Sets blending mode of pixel write of border (only for MB86293 or later).
GDC_BLEND_COPY	Regular drawing operation (writes pixel color to drawing frame).
GDC_BLEND_ALPHA	Enables alpha blending.
GDC_BLEND_ROP	Draws with logical arithmetic.

GDC_BORDER_BROKEN_LINE	Selects broken line mode of border (only for MB86293 or later).
GDC_ENABLE	Draws a broken line utilizing applied line pattern.
GDC_DISABLE	Draws a solid line.

GDC_BORDER_LINE_WIDTH	Sets line width of border (only for MB86293 or later).
GDC_LINE_WIDTH_1	Draws a line of 1 pixel width.
GDC_LINE_WIDTH_2	Draws a line of 2 pixel width.
:	:
GDC_LINE_WIDTH_32	Draws a line of 32 pixel width.

GDC_BORDER_BROKEN_LINE_PERIOD

Sets broken line pattern period of border (only for MB86293 or later).

GDC_BROKEN_LINE_32	32 bits period.
GDC_BROKEN_LINE_24	24 bits period.

GDC_BORDER_ROP_MODE Sets logical operation mode of the border

GDC_ROP_CLEAR	Sets all bits to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	Sets all bits to 1

s: drawing color
d: destination color

4.11.5 XGdcSetAttrSurf [Sets surface drawing attribute]

Interface

GDC_BOOL **XGdcSetAttrSurf** (GDC_CTX **drvctx*,
GDC_ULONG *target*, GDC_ULONG *param*)

Arguments

drvctx Pointer to Context

target Surface drawing attribute

GDC_SHADE_MODE	Shading mode
GDC_DEPTH_TEST	Z value compare mode
GDC_DEPTH_NEVER	Z value compare type
GDC_DEPTH_WRITE_MASK	Z value write mask
GDC_BLEND_MODE	Blending mode
GDC_TEXTURE_SELECT	Texture mode
GDC_ROP_MODE	Logical operation mode

[In MB86293 or later, following macros are available]

GDC_ALPHA_SHADE_MODE
Alpha shading mode

GDC_SHADOW_DEPTH_TEST
Z value compare mode of shadow

GDC_SHADOW_DEPTH_FUNC
Z value compare type of shadow

GDC_SHADOW_DEPTH_WRITE_MASK
Z value write mask of shadow

GDC_SHADOW_BLEND_MODE
Blending mode of shadow

GDC_SHADOW_ROP_MODE
Logical operation mode of shadow primitives

GDC_NON_TOPLEFT_SHADE_MODE
Shading mode of non top-left primitives

GDC_NON_TOPLEFT_DEPTH_TEST
Z value compare mode of non top-left primitives

GDC_NON_TOPLEFT_DEPTH_FUNC
Z value compare type of non top-left primitives

GDC_NON_TOPLEFT_DEPTH_WRITE_MASK
Z value write mask of non top-left primitives

GDC_NON_TOPLEFT_BLEND_MODE
Blending mode of non top-left primitives

GDC_NON_TOPLEFT_TEXTURE_SELECT
Texture mode of non top-left primitives

GDC_NON_TOPLEFT_ALPHA_SHADE_MODE
Alpha shading mode of non top-left primitives

GDC_NON_TOPLEFT_ROP_MODE
Logical operation mode of non top-left primitives

param Parameter corresponding to *target* (*1)

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED
The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_ATTRIBUTE
Invalid attribute has specified

Target GDC

All

Description

Sets attribute for surface drawing (except texture mapping attribute).

Notes

When drawing frame is 8 bits color mode, logical operations do not use actual color (GDC_COL32 format color), but use color code (GDC_COL8 format). For example, when the pixel color (palette color code) of drawing frame is value 1 and drawing color (palette color code) is value 2, **GDC_ROP_OR** operation is stored value 3 in drawing frame.

*1 Surface drawing attribute (*target*) and parameter (*param*) corresponding to each surface drawing attribute is shown below.

[Explanatory notes]

Surface drawing attribute	Description of surface drawing attribute
Parameter 1 that can set	Description of parameter 1
Parameter 2 that can set	Description of parameter 2
:	:

GDC_SHADE_MODE	Sets shading mode.
GDC_SHADE_FLAT	Flat shading.
GDC_SHADE_SMOOTH	[In 16/24 bits color mode] Works smooth shading [In 8 bits color mode] Works smooth shading which uses color palette code. The surface is drawn in color code which interpolated between each color code of vertex. This mode is available in MB86293 or later.
GDC_DEPTH_TEST	Sets Z value compare mode.
GDC_ENABLE	Validate Z value comparison.
GDC_DISABLE	Invalidate Z value comparison.
GDC_DEPTH_NEVER	Selects Z value comparison type.
GDC_DEPTH_NEVER	Always NOT drawn.
GDC_DEPTH_ALWAYS	Always drawn.
GDC_DEPTH_LESS	Drawn if current Z value is less than Z buffer value.
GDC_DEPTH_LEQUAL	Drawn if current Z value is equal to or less than Z buffer value.
GDC_DEPTH_EQUAL	Drawn if current Z value is equal to Z buffer value.
GDC_DEPTH_GEQUAL	Drawn if current Z value is equal to or more than Z buffer value.
GDC_DEPTH_GREATER	Drawn if current Z value is more than Z buffer value.
GDC_DEPTH_NOTEQUAL	Drawn if current Z value is not equal to Z buffer value.
GDC_DEPTH_WRITE_MASK	Enables write access to Z buffer. If **GDC_ENABLE**, according to the result of Z value comparison, Z value is written to Z buffer.
GDC_ENABLE	Disables Z buffer write.
GDC_DISABLE	Enables Z buffer write.
GDC_BLEND_MODE	Sets blending mode of pixel write.
GDC_BLEND_COPY	Regular drawing operation (writes pixel color to drawing frame).
GDC_BLEND_ALPHA	Enables alpha blending.
GDC_BLEND_ROP	Draws with logical arithmetic.
GDC_TEXTURE_SELECT	Sets texture mapping mode.
GDC_SELECT_TEXTURE	Draws with texture mapping.
GDC_SELECT_TILE	Draws with tiling.
GDC_SELECT_PLAIN	Invalidates texture mapping.

GDC_ALPHA_SHADE_MODE	Sets alpha shading mode. To works alpha shading, alpha-blending mode must be enabled (specify **GDC_BLEND_ALPHA** for **GDC_BLEND_MODE**). Alpha value is specified for each vertex by the **XGdcVertexColor32** command. This mode is available in 16/24 bits color mode, only in MB86293 or later.
GDC_SHADE_FLAT	Works flat shading of alpha value. All pixels of the surface are blending by the same alpha value.
GDC_SHADE_SMOOTH	Works smooth shading of alpha value. Alpha values of each pixel are generated by interpolation between alpha values of each vertex. So, each pixel of the surface is blended individually. In addition, to works this function, also specify **GDC_SHADE_SMOOTH** for **GDC_SHADE_MODE**.

GDC_ROP_MODE	Sets logical operation mode of the body.
GDC_ROP_CLEAR	Sets all bits to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	Sets all bits to 1

s: drawing color
d: destination color

GDC_SHADOW_DEPTH_TEST	Sets Z value compare mode of shadow (only for MB86293 or later).
GDC_ENABLE	Validate Z value comparison.
GDC_DISABLE	Invalidate Z value comparison.
GDC_SHADOW_DEPTH_FUNC	Selects Z value comparison type of shadow (only for MB86293 or later).
GDC_DEPTH_NEVER	Always NOT drawn.
GDC_DEPTH_ALWAYS	Always drawn.
GDC_DEPTH_LESS	Drawn if current Z value is less than Z buffer value.
GDC_DEPTH_LEQUAL	Drawn if current Z value is equal to or less than Z buffer value.
GDC_DEPTH_EQUAL	Drawn if current Z value is equal to Z buffer value.
GDC_DEPTH_GEQUAL	Drawn if current Z value is equal to or more than Z buffer value.
GDC_DEPTH_GREATER	Drawn if current Z value is more than Z buffer value.
GDC_DEPTH_NOTEQUAL	Drawn if current Z value is not equal to Z buffer value.
GDC_SHADOW_DEPTH_WRITE_MASK	Enables write access to Z buffer of shadow (only for MB86293 or later). If **GDC_ENABLE**, according to the result of Z value comparison, Z value is written to Z buffer.
GDC_ENABLE	Disables Z buffer write.
GDC_DISABLE	Enables Z buffer write.
GDC_SHADOW_BLEND_MODE	Sets blending mode of pixel write of shadow (only for MB86293 or later).
GDC_BLEND_COPY	Regular drawing operation (writes pixel color to drawing frame).
GDC_BLEND_ALPHA	Enables alpha blending.
GDC_BLEND_ROP	Draws with logical arithmetic.

GDC_SHADOW_ROP_MODE	Sets logical operation mode of the shadow.
GDC_ROP_CLEAR	Sets all bits to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	Sets all bits to 1
	s: drawing color d: destination color
GDC_NON_TOPLEFT_SHADE_MODE	Sets shading mode of non top-left primitive (only for MB86293 or later).
GDC_SHADE_FLAT	Flat shading.
GDC_SHADE_SMOOTH	Gouraud shading.
GDC_NON_TOPLEFT_DEPTH_TEST	Sets Z value compare mode of non top-left primitive (only for MB86293 or later).
GDC_ENABLE	Validate Z value comparison.
GDC_DISABLE	Invalidate Z value comparison.
GDC_NON_TOPLEFT_DEPTH_FUNC	Selects Z value comparison type of non top-left primitive (only for MB86293 or later).
GDC_DEPTH_NEVER	Always NOT drawn.
GDC_DEPTH_ALWAYS	Always drawn.
GDC_DEPTH_LESS	Drawn if current Z value is less than Z buffer value.
GDC_DEPTH_LEQUAL	Drawn if current Z value is equal to or less than Z buffer value.
GDC_DEPTH_EQUAL	Drawn if current Z value is equal to Z buffer value.
GDC_DEPTH_GEQUAL	Drawn if current Z value is equal to or more than Z buffer value.
GDC_DEPTH_GREATER	Drawn if current Z value is more than Z buffer value.
GDC_DEPTH_NOTEQUAL	Drawn if current Z value is not equal to Z buffer value.

GDC_NON_TOPLEFT_DEPTH_WRITE_MASK

Enables write access to Z buffer of non top-left primitive (only for MB86293 or later).

If **GDC_ENABLE**, according to the result of Z value comparison, Z value is written to Z buffer.

GDC_ENABLE	Disables Z buffer write.
GDC_DISABLE	Enables Z buffer write.

GDC_NON_TOPLEFT_BLEND_MODE

Sets blending mode of pixel write of non top-left primitive (only for MB86293 or later).

GDC_BLEND_COPY	Regular drawing operation (writes pixel color to drawing frame).
GDC_BLEND_ALPHA	Enables alpha blending.
GDC_BLEND_ROP	Draws with logical arithmetic.

GDC_NON_TOPLEFT_TEXTURE_SELECT

Sets texture mapping mode of non top-left primitive (only for MB86293 or later).

GDC_SELECT_TEXTURE	Draws with texture mapping.
GDC_SELECT_TILE	Draws with tiling.
GDC_SELECT_PLAIN	Invalidates texture mapping.

GDC_NON_TOPLEFT_ALPHA_SHADE_MODE

Sets alpha shading mode of non top-left primitives.
To works this function, alpha-blending mode must be enabled (specify **GDC_BLEND_ALPHA** for **GDC_NON_TOPLEFT_BLEND_MODE**). Alpha value is specified for each vertex by the **XGdcVertexColor32** command.
This mode is available in 16/24 bits color mode, only in MB86293 or later.

GDC_SHADE_FLAT	Works flat shading of alpha value. All pixels of the surface are blending by the same alpha value.
GDC_SHADE_SMOOTH	Works smooth shading of alpha value. Alpha values of each pixel are generated by interpolation between alpha values of each vertex. So, each pixel of the surface is blended individually. In addition, to works this function, also specify **GDC_SHADE_SMOOTH** for **GDC_NON_TOPLEFT_SHADE_MODE** for.

GDC_NON_TOPLEFT_ROP_MODE	Sets logical operation mode of non top-left primitives.

GDC_ROP_CLEAR	Sets all bits to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	Sets all bits to 1

s: drawing color
d: destination color

4.11.6 XGdcSetAttrTexture [Sets texture mapping attribute]

Interface

GDC_BOOL **XGdcSetAttrTexture** (GDC_CTX **drvctx*,
GDC_ULONG *target*, GDC_ULONG *param*)

Arguments

drvctx Pointer to Context

target Texture mapping attribute

GDC_TEXTURE_PERSPECTIVE	Perspective correction
GDC_TEXTURE_FILTER	Texture filter
GDC_TEXTURE_WRAP_S	S coordinates wrap
GDC_TEXTURE_WRAP_T	T coordinates wrap
GDC_TEXTURE_BLEND	Texture blend mode
GDC_TEXTURE_ALPHA	Texture alpha mode

[In MB86293 or later, following macros are available]
GDC_TEXTURE_FAST_MODE
Bi-linear filtering fast mode

param Parameter corresponding to *target* (*1)

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_ATTRIBUTE

Invalid attribute has specified

Target GDC

All

Description

Sets attribute for texture mapping.

(*1) Texture mapping attribute (*target*) and parameter (*param*) corresponding to each texture mapping attribute is shown below.

[Explanatory notes]

Texture mapping attribute	Description of texture mapping attribute
Parameter 1 that can set	Description of parameter 1
Parameter 2 that can set	Description of parameter 2
:	:
GDC_TEXTURE_PERSPECTIVE	Selects perspective correction mode.
GDC_ENABLE	Validates perspective correction.
GDC_DISABLE	Invalidates perspective correction.
GDC_TEXTURE_FILTER	Selects texture filter mode.
GDC_TEXTURE_POINT	Point sampling mode.
GDC_TEXTURE_BILINEAR	Bi-linear filtering mode.
GDC_TEXTURE_WRAP_S	Defines S coordinates wrapping option when S coordinates value exceed the texture size.
GDC_TEXTURE_REPEAT	Repeats the texture pattern.
GDC_TEXTURE_CLAMP	Sets out-most texture color.
GDC_TEXTURE_BORDER	Sets defined border color.
GDC_TEXTURE_WRAP_T	Sets T coordinates wrapping option when T coordinates value exceeds the texture size.
GDC_TEXTURE_REPEAT	Repeats the texture pattern.
GDC_TEXTURE_CLAMP	Sets out-most texture color.
GDC_TEXTURE_BORDER	Sets defined border color.
GDC_TEXTURE_BLEND	Sets blending mode of texture color and polygon color. This is applicable only when texture mapping mode is selected.
GDC_TEXTURE_DECAL	Texture color is drawn.
GDC_TEXTURE_MODULATE	Blended color is drawn.
GDC_TEXTURE_STENCIL	If MSB of texture color is 1, texture color is drawn, otherwise polygon color is drawn.

GDC_TEXTURE_ALPHA	Sets alpha blending mode between drawn color and current pixel color of the drawing frame. This is applicable only when texture mapping and alpha blending are selected.
GDC_TEXTURE_ALPHA_ALL	Alpha blend between post texture mapping color and current pixel color of the drawing frame.
GDC_TEXTURE_ALPHA_STENCIL	If MSB of texture color is 1, texture color is drawn, otherwise not drawn.
GDC_TEXALPHA_ALPHA_STENCILALPHA	If MSB of texture color is 1, alpha blend between texture color and current pixel color in the drawing frame is performed, otherwise not drawn.
GDC_TEXTURE_FAST_MODE	Sets bi-linear fast mode (only for MB86293 or later).
GDC_ENABLE	Texture mapping is executed at fast speed using a default * 4 as texture area.
GDC_DISABLE	A default texture area is used.

4.11.7 XGdcSetAttrBlt [Sets BitBlt attribute]

Interface

GDC_BOOL **XGdcSetAttrBlt** (GDC_CTX **drvctx*,
GDC_ULONG *target*, GDC_ULONG *param*)

Arguments

drvctx Pointer to Context

target Bitmap drawing attribute

GDC_BLEND_MODE	Blend mode
GDC_ROP_MODE	Logical operation mode

[In MB86291 or later, following macros are available]

GDC_TRANSPARENT_MODE	Transparent mode

param Parameter corresponding to *target* (*1)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_ATTRIBUTE

Invalid attribute has specified

Target GDC

All (See Notes)

Description

Sets attribute when copying and drawing BitBlt.

Notes

- Transparent mode is available in MB86291 or later.
- Specify the transparent color by the **XGdcBltColorTransparent** command.
- When drawing frame is 8 bits color mode, logical operations do not use actual color (GDC_COL32 format color), but use color code (GDC_COL8 format). For example, when the pixel color (palette color code) of drawing frame is value 1 and drawing color (palette color code) is value 2, **GDC_ROP_OR** operation is stored value 3 in drawing frame.

*1 Bitmap drawing attribute (*target*) and parameter (*param*) corresponding to each bitmap drawing attributes are shown below.

[Explanatory notes]

Bitmap drawing attribute	Description of bitmap drawing attributes
Parameter 1 that can set	Description of parameter 1
Parameter 2 that can set	Description of parameter 2
:	:

GDC_BLEND_MODE	Sets blend mode.
GDC_BLEND_COPY	Regular drawing operation (writes pixel color to drawing frame).
GDC_BLEND_ROP	Draws with logical arithmetic.
GDC_TRANSPARENT_MODE	Sets transparent mode (only for MB86291 or later).
GDC_ENABLE	The color which was set by the **XGdcBltColorTransparent** command regards as transparent color.
GDC_DISABLE	The color which was set by the **XGdcBltColorTransparent** command is not treated as transparent color.
GDC_ROP_MODE	Sets logical operation mode.

GDC_ROP_CLEAR	Sets all bits to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	Sets all bits to 1

s: drawing color
d: destination color

4.11.8 XGdcSetAlpha [Sets alpha blend ratio]

Interface

GDC_BOOL **XGdcSetAlpha** (GDC_CTX **drvctx*, GDC_UCHAR *alpha*)

Arguments

drvctx	Pointer to Context
alpha	Alpha blending ratio, range 0-255

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets the ratio of alpha blending.

Transmissivity is 100%, when *alpha* is 0. And transmissivity is 0% when *alpha* is 255.

4.11.9 XGdcSetLinePattern [Sets broken line pattern]

Interface

GDC_BOOL **XGdcSetLinePattern** (GDC_CTX **drvctx*, GDC_ULONG *pattern*)

Arguments

drvctx	Pointer to Context
pattern	Broken line pattern

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets broken line pattern with 32 bits pattern when drawing broken line.

Upper 24 bits pattern values are set as a broken line pattern, when the period of broken line pattern is 24 bits.

Broken line is drawn with foreground color when the bit of pattern is 1, and drawn with background color when the bit is 0.

Example of broken line with broken line pattern “0xaaaaaaaa” is shown in Figure 4.11.9.

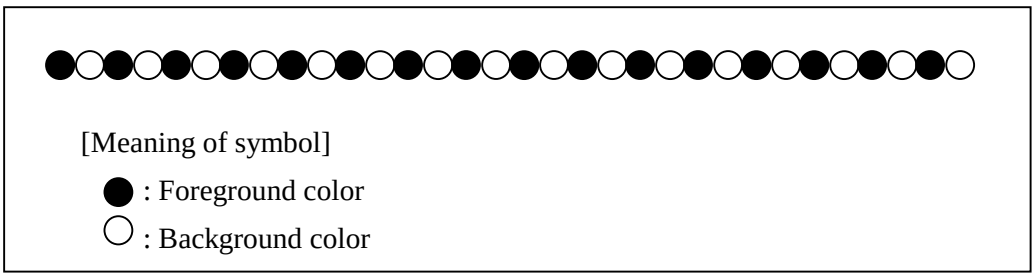


Figure 4.11.9 Example of broken line

4.11.10 XGdcSetTextureBorder [Sets texture border color]

Interface

GDC_BOOL **XGdcSetTextureBorder**(GDC_CTX ***drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Texture border color

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets the border color of the texture applied in border mode of texture wrap.

The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

4.11.11XGdcSetRop [Sets logical operation mode]

Interface

GDC_BOOL **XGdcSetRop**(GDC_CTX **drvctx*, GDC_UCHAR *mode*)

Arguments

drvctx Pointer to Context

mode Logical arithmetic mode

GDC_ROP_CLEAR	All bits are set to 0
GDC_ROP_AND	s & d
GDC_ROP_AND_REVERSE	s & !d
GDC_ROP_COPY	s
GDC_ROP_AND_INVERTED	!s & d
GDC_ROP_NOP	d
GDC_ROP_XOR	s ^ d
GDC_ROP_OR	s | d
GDC_ROP_NOR	!(s | d)
GDC_ROP_EQUIV	!(s ^ d)
GDC_ROP_INVERT	!d
GDC_ROP_OR_REVERSE	s | !d
GDC_ROP_COPY_INVERTED	!s
GDC_ROP_OR_INVERTED	!s | d
GDC_ROP_NAND	!(s & d)
GDC_ROP_SET	All bits are set to 1

s:drawing value
d:destination value

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets logical operation for all drawing process.

This operation is performed between the pixel color to be drawn and current pixel color in the drawing frame. Result of this operation is to be drawn to the drawing frame.

This operation is applicable only when drawing attribute of line or surface or BitBlt's **GDC_BLEND_ROP** option of **GDC_BLEND_MODE** is selected.

To set logical operation mode individually for lines, surfaces and rectangles, use the

XGdcSetAttrLine command, the **XGdcSetAttrSurf** command, and the **XGdcSetAttrBlt** command, respectively.

Notes

* In MB86293 or later, Keep in mind that it will perform logical arithmetic in the shadows when the primitive between shadow composition overlaps with the shadow, since the same logical arithmetic mode as the shadow of a line and the primitive between shadow composition is applied if the logical arithmetic mode of the shadow is set up.

* In MB86293 or later, same logical operation is applied to shadow primitives and shadow composition primitives. Therefore, in the portion with which shadow primitive and shadow composition primitive overlap, logical operation works between shadow and shadow composition.

When 8 bits color mode is used in drawing frame, logical operations do not use actual color (GDC_COL32 format color), but use color code (GDC_COL8 format). For example, when the pixel color (palette color code) of drawing frame is value 1 and drawing color (palette color code) is value 2, **GDC_ROP_OR** operation is stored value 3 in drawing frame.

4.12 Attribute Control Commands for Object Coordinates

4.12.1 XGdcGeoSetAttrLine [Sets line drawing attribute for object coordinates]

Interface

GDC_BOOL **XGdcGeoSetAttrLine**(GDC_CTX **drvctx*,
GDC_ULONG *target*, GDC_ULONG *param*)

Arguments

drvctx Pointer to Context

target Line drawing attribute

GDC_GEO_THICK_LINE_CORRECT
Sets correction mode of thick line connection

GDC_GEO_BROKEN_LINE_CORRECT
Sets correction mode of broken line pattern

GDC_GEO_BROKEN_LINE_CORRECT_LENGTH
Sets the number of broken line pattern address fixation pixel

GDC_GEO_UNIFORM_LINE_WIDTH
Sets uniform mode of line width

GDC_GEO_THICK_LINE_VERTICAL
Sets thick/broken line vertical mode

GDC_GEO_BORDER_LINE
Sets drawing mode of border primitive

GDC_GEO_SHADOW_MODE
Sets drawing mode of shadow primitive

param Parameter corresponding to *target* (*1)

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED
The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_ATTRIBUTE
Invalid attribute has specified

Target GDC

MB86293 or later

Description

Sets drawing attributes for object coordinate line primitives.

Table 4.12.1 shows available combinations of drawing attributes.

Notes

Drawing attributes which have been set by this command effect on lines which no z coordinates are specified. Therefore, this command effect on only lines which vertices are specified the **XGdcGeoDrawVertex2D[f/i]** command.

*1 Line drawing attribute (*target*) and parameter (*param*) corresponding to each line drawing attribute is shown below.

[Explanatory notes]

Line drawing attribute	Description of line drawing attribute
Parameter 1 that can set	Description of parameter 1
Parameter 2 that can set	Description of parameter 2
:	:

GDC_GEO_THICK_LINE_CORRECT	Sets thick line connection correct mode.
GDC_ENABLE	Enables thick line connection correct.
GDC_DISABLE	Disables thick line connection correct.
GDC_GEO_BROKEN_LINE_CORRECT	Sets broken line connection correct mode.
GDC_ENABLE	Refer to the same broken line pattern with front and back number pixel of broken line connection part (“broken line pattern address fixation mode”). Number of pixel is set by **GDC_GEO_BROKEN_LINE_CORRECT_LENGTH**.
GDC_DISABLE	Not correct broken line pattern.
GDC_GEO_BROKEN_LINE_CORRECT_LENGTH	Sets the number of broken line pattern address fixation pixel. A recommended value is the same as line width. This parameter is available when the correction mode of broken line pattern is “broken line pattern address fixation mode”.
0-32	Number of pixels.

GDC_GEO_UNIFORM_LINE_WIDTH	Sets uniform mode of line width.
GDC_ENABLE	Enables uniform of line width. When thick/broken line perpendicular to ideal line is drawn, this mode must be specified.
GDC_DISABLE	Disables uniform of line width.
GDC_GEO_THICK_LINE_VERTICAL	Sets thick/broken line vertical mode.
GDC_ENABLE	Draws perpendicular section of thick/broken line to an ideal line.
GDC_DISABLE	Draws perpendicular section of thick/broken line to a base axis.
GDC_GEO_BORDER_LINE	Sets drawing mode of border primitive.
GDC_ENABLE	Draws border primitive. For the border color, the color specified by the **XGdcGeoBorderColor** command is used.
GDC_DISABLE	Not draw border primitive.
GDC_GEO_SHADOW_MODE	Sets drawing mode of shadow primitive.
GDC_ENABLE	Draws shadow primitive. For the shadow color, the color specified by the **XGdcGeoShadowColor** command is used.
GDC_DISABLE	Not draw shadow primitive.

Table 4.12.1 Combinations of drawing attributes for lines

		GDC_GEO_THICK_LINE_VERTICAL	
		GDC_ENABLE	**GDC_DISABLE**
GDC_GEO_THICK_LINE_CORRECT	**GDC_ENABLE**	Y	N
	GDC_DISABLE	Y	Y
GDC_GEO_UNIFORM_LINE_WIDTH	**GDC_ENABLE**	Y	N
	GDC_DISABLE	N	Y
GDC_GEO_BORDER_LINE	**GDC_ENABLE**	Y	N
	GDC_DISABLE	Y	Y
GDC_GEO_SHADOW_MODE	**GDC_ENABLE**	Y	N
	GDC_DISABLE	Y	Y

Y: available N: not available (no guarantee the result)

4.12.2 XGdcGeoSetAttrSurf [Sets surface drawing attribute for object coordinates]

Interface

GDC_BOOL **XGdcGeoSetAttrSurf** (GDC_CTX **drvctx*,
GDC_ULONG *target*, GDC_ULONG *param*)

Arguments

drvctx Pointer to Context

target Surface drawing attribute

GDC_GEO_FACE_CULL

Enable/disable culling back face of triangle

GDC_GEO_FACE_INVERT

Specify direction of surface of triangle

[In MB86293 or later, following macros are available]

GDC_GEO_NON_TOPLEFT

Sets drawing algorithm

GDC_GEO_SHADOW_MODE

Sets drawing mode of shadow primitive

param Parameter corresponding to *target* (*1)

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_ATTRIBUTE

Invalid attribute has specified

Target GDC

MB86291 or later

Description

Sets surface drawing attribute in object coordinates.

Culling back face of triangle and specify direction of surface of triangle doesn't affect polygons (**GDC_POLYGON**).

*1 Surface drawing attribute (*target*) and parameter (*param*) corresponding to each surface drawing attribute is shown below.

[Explanatory notes]

Surface drawing attribute	Description of surface drawing attribute
Parameter 1 that can set	Description of parameter 1
Parameter 2 that can set	Description of parameter 2
:	:

GDC_GEO_FACE_CULL	Specifies culling back face of triangle.
GDC_ENABLE	Enables culling back face of triangle.
GDC_DISABLE	Disables culling back face of triangle.
GDC_GEO_FACE_INVERT	Specifies direction of surface of triangle. Counterclockwise surface is front facing by default.
GDC_ENABLE	Invert direction of surface from default.
GDC_DISABLE	Direction of surface is default.
GDC_GEO_NON_TOPLEFT	Sets drawing algorithm (for MB86293 or later).
GDC_ENABLE	Non top-left applying rule is used.
GDC_DISABLE	Non top-left applying rule is not used.
GDC_GEO_SHADOW_MODE	Sets drawing mode of shadow primitive (for MB86293 or later).
GDC_ENABLE	Draws shadow primitive. For the shadow color, the color specified by the **XGdcGeoShadowColor** command is used.
GDC_DISABLE	Not draw shadow primitive.

4.12.3 XGdcGeoLoadMatrix[f] [Sets matrix]

Interface

GDC_BOOL **XGdcGeoLoadMatrix** (GDC_CTX **drvctx*,
const GDC_FIXED32 **ptMatrix*)

GDC_BOOL **XGdcGeoLoadMatrixf** (GDC_CTX **drvctx*,
const GDC_SFLOAT **ptMatrix*)

Arguments

drvctx Pointer to Context

ptMatrix A pointer to an array {m1, m2, m3, …, m16} which corresponds to the 4 x 4 matrix M such as,

$$M=\begin{pmatrix} m1 & m5 & m9 & m13 \\ m2 & m6 & m10 & m14 \\ m3 & m7 & m11 & m15 \\ m4 & m8 & m12 & m16 \end{pmatrix}$$

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86291 or later

Description

Sets a 4 x 4 matrix that transforms an object coordinates to a clip coordinate.

Each element in the matrix is put in the following order.

$$M=\begin{pmatrix} m1 & m5 & m9 & m13 \\ m2 & m6 & m10 & m14 \\ m3 & m7 & m11 & m15 \\ m4 & m8 & m12 & m16 \end{pmatrix}$$

Elements (m4 ,m8, m12, m16) in the matrix specify whether the projection type is orthographic or perspective. Therefore the projection type is set automatically by the result of their values.

If (m4 ,m8, m12, m16) == (0,0,0,1), then orthographic projection.
Else if (m4 ,m8, m12, m16) != (0,0,0,1) then perspective projection.

4.12.4 XGdcGeoNdcDcViewportCoef[f] [Sets coefficients of NdcDc transformation for XY]

Interface

GDC_BOOL **XGdcGeoNdcDcViewportCoef** (GDC_CTX **drvctx*,
GDC_FIXED32 *scalex*, GDC_FIXED32 *offsetx*,
GDC_FIXED32 *scaley*, GDC_FIXED32 *offsety*)

GDC_BOOL **XGdcGeoNdcDcViewportCoeff** (GDC_CTX **drvctx*,
GDC_SFLOAT *scalex*, GDC_SFLOAT *offsetx*,
GDC_SFLOAT *scaley*, GDC_SFLOAT *offsety*)

Arguments

drvctx	Pointer to Context
scalex	Magnification of x
offsetx	Offset of x
scaley	Magnification of y
offsety	Offset of y

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86291 or later

Description

Sets the magnifications and offsets of x, y that is used for transforming Normalized Device Coordinates (NDC) to Device Coordinates (DC).

4.12.5 XGdcGeoNdcDcDepthCoef[f] [Sets coefficients of NdcDc transformation for Z]

Interface

GDC_BOOL **XGdcGeoNdcDcDepthCoef** (GDC_CTX **drvctx*,
GDC_FIXED32 *scalez*, GDC_FIXED32 *offsetz*)

GDC_BOOL **XGdcGeoNdcDcDepthCoeff** (GDC_CTX **drvctx*,
GDC_SFLOAT *scalez*, GDC_SFLOAT *offsetz*)

Arguments

drvctx	Pointer to Context
scalez	Magnification of z
offsetz	Offset of z

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86291 or later

Description

Sets the magnification and offset of z that is used for transforming Normalized Device Coordinates (NDC) to Device Coordinates (DC).

4.12.6 XGdcGeoViewVolumeXYClip[f] [Sets view volume boundary for XY]

Interface

GDC_BOOL **XGdcGeoViewVolumeXYClip** (GDC_CTX **drvctx*,
GDC_FIXED32 *xmin*, GDC_FIXED32 *xmax*,
GDC_FIXED32 *ymin*, GDC_FIXED32 *ymax*)

GDC_BOOL **XGdcGeoViewVolumeXYClipf** (GDC_CTX **drvctx*,
GDC_SFLOAT *xmin*, GDC_SFLOAT *xmax*,
GDC_SFLOAT *ymin*, GDC_SFLOAT *ymax*)

Arguments

drvctx	Pointer to Context
xmin	Minimum clip value of x
xmax	Maximum clip value of x
ymin	Minimum clip value of y
ymax	Maximum clip value of y

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86291 or later

Description

Sets the view volume boundary in the clip coordinates for x, y.

4.12.7 XGdcGeoViewVolumeZClip[f] [Sets view volume boundary for Z]

Interface

GDC_BOOL **XGdcGeoViewVolumeZClip** (GDC_CTX **drvctx*,
GDC_FIXED32 *zmin*, GDC_FIXED32 *zmax*)

GDC_BOOL **XGdcGeoViewVolumeZClipf** (GDC_CTX **drvctx*,
GDC_SFLOAT *zmin*, GDC_SFLOAT *zmax*)

Arguments

drvctx	Pointer to Context
zmin	Minimum clip value of z
zmax	Maximum clip value of z

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86291 or later

Description

Sets the view volume boundary in the clip coordinates for z.

4.12.8 XGdcGeoViewVolumeWminClip[f] [Sets view volume boundary for w]

Interface

GDC_BOOL **XGdcGeoViewVolumeWminClip** (GDC_CTX **drvctx*,
GDC_FIXED32 *wmin*)

GDC_BOOL **XGdcGeoViewVolumeWminClipf** (GDC_CTX **drvctx*,
GDC_SFLOAT *wmin*)

Arguments

drvctx	Pointer to Context
wmin	Minimum clip value of w

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86291 or later

Description

Sets the view volume boundary in the clip coordinates for w.

As the front clip face (zmin) closes with the viewpoint limitlessly, w also approximates to zero limitlessly.

Since w is used to calculate 1/w internally, *wmin* must be the one that does not occur overflow in division.

w has only minimum value. *wmin* is not minus value.

4.12.9 XGdcGeoSetLogOutBase [Sets base address for log output of device coordinates]

Interface

GDC_BOOL **XGdcGeoSetLogOutBase** (GDC_CTX **drvctx*, GDC_ULONG *adrs*)

Arguments

drvctx	Pointer to Context
adrs	Base address for log output of device coordinates (Offset from top of the Graphics Memory)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

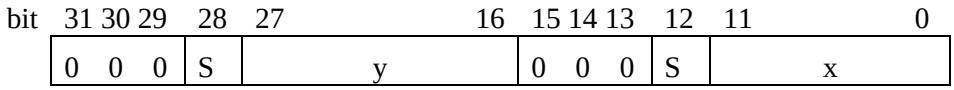
MB86293 or later

Description

Sets base address for log output of device coordinates.

Specify *adrs* for offset address from top of the Graphics Memory.

Log format consists of packed number of x and y coordinates of vertex (refer to Figure 4.12.9).



S: signed bit

y: y coordinates values (integer)

x: x coordinates values (integer)

All other bits: 0

Figure 4.12.9 Log format of device coordinates

Log needs 4 bytes for every vertex.

Therefore, memory area must be allocated with size of number of vertex * 4 byte.

These vertices are specified between the **XGdcGeoPrimType** command and the **XGdcGeoPrimEnd** command.

Each time a log is outputted, the address for log output is added 4 bytes.

If this memory area is used repeatedly, sets the base address for log output again.
The address for log output must be set after drawing is finished by “drawing command end interruption” or the **GdcGetPixelEngineStatus** command or the **GdcGeoGetPixelEngineStatus** command.

4.12.10 XGdcGeoSetLogOutMode [Sets log output mode of the device coordinates]

Interface

void **XGdcGeoSetLogOutMode** (GDC_CTX **drvctx*, GDC_ULONG *mode*)

Arguments

drvctx Pointer to Context

mode Log output mode of device coordinate

GDC_GEO_LOGOUT_ENABLE

Outputs logs

GDC_GEO_LOGOUT_DISABLE

Not output logs (default)

GDC_GEO_LOGOUT_ONLY

Output logs without drawing

Return value

None

Target GDC

MB86293 or later

Description

Sets log output mode of device coordinates. Each mode is explained below.

- GDC_GEO_LOGOUT_ENABLE

Log is outputted and drawing is executed.

- GDC_GEO_LOGOUT_DISABLE

Log is not outputted and drawing is executed.

- GDC_GEO_LOGOUT_ONLY

Log is outputted and drawing is not executed.

This mode is available only when drawing point primitive. When **GDC_GEO_LOGOUT_ONLY** is specified for drawing other primitives, log is not outputted.

4.12.11 XGdcGeoShadowXY [Sets xy offset of shadow]

Interface

GDC_BOOL **XGdcGeoShadowXY** (GDC_CTX **drvctx*,
GDC_ULONG *type*, GDC_LONG *offsetx*, GDC_LONG *offsety*)

Arguments

drvctx	Pointer to Context
type	A kind of primitive **GDC_GEO_SHADOW** Shadow primitive **GDC_GEO_SHADOW_COMPOSITION** Shadow composition primitive
offsetx	x offset of shadow (or shadow composition) primitive for body primitive (pixel unit)
offsety	y offset of shadow (or shadow composition) primitive for body primitive (pixel unit)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets offset of shadow from body. Offset must be specified with pixel unit.

Shadow composition primitive is the 2nd shadow of lines. When you wish shadow composition primitive is not drawn, specify same offsets for shadow and shadow composition. Shadow composition primitive is not drawn in triangles. Figure 4.12.11 shows relation between shadow primitive and shadow composition primitive.

When offset is positive number, position of x is right side of body, y is lower side of body.

When offset is negative number, position of x is left side of body, y is upper side of body.

Offset position of shadow form body must be set before drawing shadow primitive.

Shadow primitive drawing function is available in object coordinates drawing.

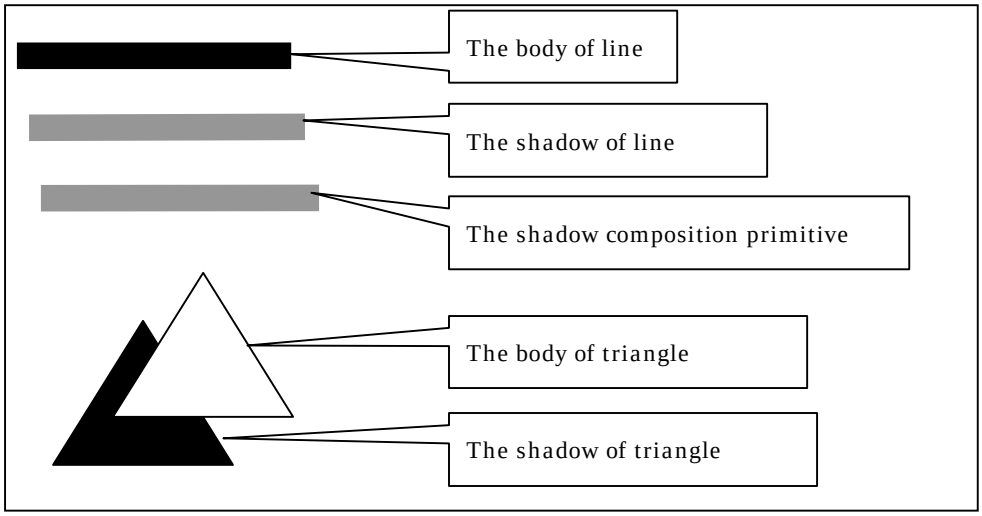


Figure 4.12.11 Relation between shadow primitive and shadow composition primitive

4.12.12 XGdcGeoOverlapZ [Sets Z value of primitives (body / shadow / border / correction in top-left rule non-applicable mode)]

Interface

GDC_BOOL **XGdcGeoOverlapZ** (GDC_CTX **drvctx*, GDC_ULONG *origin_offset*, GDC_ULONG *non_topleft_offset*, GDC_ULONG *border_offset*, GDC_ULONG *shadow_offset*)

Arguments

drvctx	Pointer to Context
origin_offset	Z value of body primitive
non_topleft_offset	Z value of correction primitive in top-left rule non-applicable mode
border_offset	Z value of border primitive
shadow_offset	Z value of shadow primitive

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets Z value of body, shadow, border and correction primitive in top-left rule non-applicable mode.

Z value must be set before drawing these primitives.

When precision of Z value is 8 bits, lower 8 bits of each parameter is effective.

When precision of Z value is 16 bits, lower 16 bits of each parameter is effective.

4.12.13 XGdcGeoShadowColor [Sets color or shadow]

Interface

GDC_BOOL **XGdcGeoShadowColor** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Color of shadow

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets color of shadow. This setting is effective in drawing lines with shadow, triangles with shadow, and polygons with shadow.

Once this command is executed, the same color is continuously applied till this command will be executed.

The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

4.12.14 XGdcGeoShadowBackColor [Sets background color of shadow]

Interface

GDC_BOOL **XGdcGeoShadowBackColor** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Background color of shadow

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets background color of shadow. Background color of shadow is effective when drawing lines with shadow and its shadow is broken line.

Background color is corresponding to 0 in bits of broken line pattern when drawing shadow as broken line (refer to Figure 4.12.14).

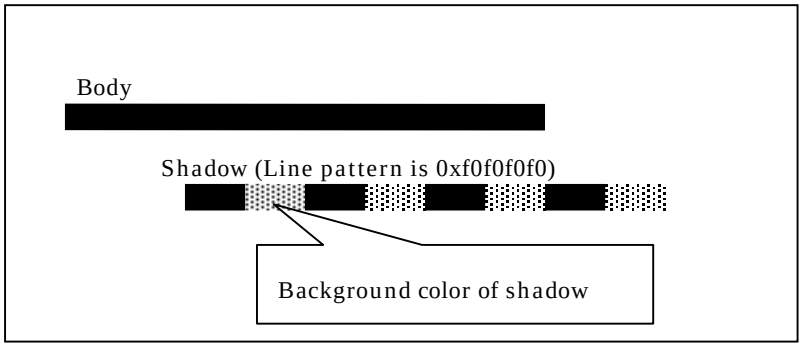


Figure 4.12.14 Background color of broken shadow line

Once this command is executed, the same color is continuously applied till this command will be executed.

The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

4.12.15 XGdcGeoBorderColor [Sets color or border]

Interface

GDC_BOOL **XGdcGeoBorderColor** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Color of border

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets color of border. This setting is effective in drawing lines with border.

Once this command is executed, the same color is continuously applied till this command will be executed.

The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

4.12.16 XGdcGeoBorderBackColor [Sets background color of border]

Interface

GDC_BOOL **XGdcGeoBorderBackColor** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Background color of border

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Sets background color of border. Background color of border is effective when drawing lines with border and its border is broken line.

Background color is corresponding to 0 in bits of broken line pattern when drawing border as broken line (refer to Figure 4.12.16).

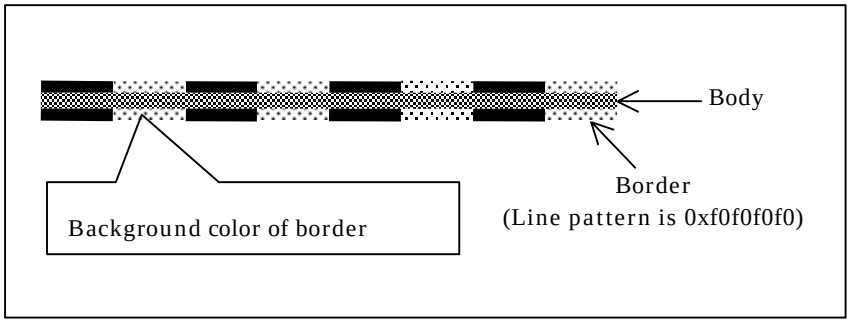


Figure 4.12.16 Background color of broken border line

Once this command is executed, the same color is continuously applied till this command will be executed.

The following values are used according to color mode.

- 8 bits color mode : lower 8 bits of *color*
- 16 bits color mode : lower 16 bits of *color*
- 24 bits color mode : lower 24 bits of *color*

4.13 Texture Pattern Control Commands

4.13.1 XGdcTextureMemoryMode [Sets Texture Memory mode]

Interface

GDC_BOOL **XGdcTextureMemoryMode** (GDC_CTX **drvctx*, GDC_UCHAR *mode*)

Arguments

drvctx Pointer to Context

mode Texture memory mode

GDC_TEX_MEM_MODE_EXT

Read from the Graphics Memory

GDC_TEX_MEM_MODE_INT

Read from Internal Texture Memory

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_PARAMETER

Invalid parameter has specified

Target GDC

All

Description

Sets the source memory to refer texture pattern from, either Internal Texture Memory or the Graphics Memory.

4.13.2 XGdcTextureLoadInt[8/16/24] [Loads texture/tile pattern to internal Texture Memory]

Interface

GDC_BOOL **XGdcTextureLoadInt8** (GDC_CTX **drvctx*,
const GDC_COL8 **lpTexture*,
GDC_ULONG *oadrs*)

GDC_BOOL **XGdcTextureLoadInt16** (GDC_CTX **drvctx*,
const GDC_COL16 * *lpTexture*,
GDC_ULONG *oadrs*)

GDC_BOOL **XGdcTextureLoadInt24** (GDC_CTX **drvctx*,
const GDC_COL24 * *lpTexture*,
GDC_ULONG *oadrs*)

Arguments

drvctx	Pointer to Context
lpTexture	Pointer to refer texture pattern
oadrs	Offset address of the memory where texture pattern is stored, specify offset from the top of internal Texture Memory

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_DATA_TOO_BIG

Data is too big

Target GDC

XGdcTextureLoadInt8 : All

XGdcTextureLoadInt16 : All

XGdcTextureLoadInt24 : MB86293 or later (only in 24bpp color supported Graphics Controller)

Description

Loads a texture/tile pattern to internal Texture Memory.

The **XGdcTextureLoadInt8**, the **XGdcTextureLoadInt16**, the **XGdcTextureLoadInt24** command handle 8bpp, 16bpp and 24bpp images respectively.

Specify *oadrs* for offset from the top of internal Texture Memory. The size of internal Texture Memory is 8KB(address range: 0x0000-0x1fff). When the texture/tile pattern is loaded to out of the range, error occurs.

Notes

Prior to this command execution, sets the dimension of an image to be loaded by the **XGdcTextureDimension** command.

4.13.3 XGdcTextureLoadExt[8/16/24] [Loads texture pattern to the Graphics Memory]

Interface

GDC_BOOL **XGdcTextureLoadExt8** (GDC_CTX **drvctx*,
const GDC_COL8 **lpTexture*, GDC_ULONG *adrs*)

GDC_BOOL **XGdcTextureLoadExt16** (GDC_CTX **drvctx*,
const GDC_COL16 **lpTexture*, GDC_ULONG *adrs*)

GDC_BOOL **XGdcTextureLoadExt24** (GDC_CTX **drvctx*,
const GDC_COL24 **lpTexture*, GDC_ULONG *adrs*)

Arguments

drvctx	Pointer to Context
lpTexture	Pointer to refer texture pattern
adrs	Offset address of the memory where texture pattern is stored (Offset from top of Graphics Memory)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

XGdcTextureLoadExt8 : MB86293 or later

XGdcTextureLoadExt16 : All

XGdcTextureLoadExt24 : MB86293 or later (only in 24bpp color supported Graphics Controller)

Description

Copies texture pattern to the Graphics Memory.

The **XGdcTextureLoadExt8**, the **XGdcTextureLoadExt16**, the **XGdcTextureLoadExt24** command handle 8bpp, 16bpp and 24bpp images respectively.

Specify *adrs* for offset from the top of the Graphics Memory.

Notes

Prior to this command execution, sets the dimension of an image to be loaded by the **XGdcTextureDimension** command.

4.13.4 XGdcTextureLoadInt16Fast [Loads texture pattern to internal Texture Memory]

Interface

GDC_BOOL **XGdcTextureLoadInt16Fast** (GDC_CTX **drvctx*,
const GDC_COL16 **lpTexture*, GDC_ULONG *adrs*)

Arguments

drvctx	Pointer to Context
lpTexture	Pointer to refer texture pattern
adrs	Address of the memory where texture pattern is loaded, specify offset from the top of internal Texture Memory

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_DATA_TOO_BIG

Too large data

Target GDC

MB86293 or later

Description

Converts a 16bpp texture pattern to the format for fast Bi-Linear mode, and loaded it to internal Texture Memory. Specify *adrs* for offset from the top of internal Texture Memory. The size of internal Texture Memory is 8KB(address range: 0x0000-0x1fff). When the converted image is loaded to out of the range, error occurs.

Notes

* Prior to this command execution, sets the dimension of an image to be loaded by the **XGdcTextureDimension** command. Specify the dimension as it is (not 4 fold).
* The image converted for fast Bi-Linear mode gets 4 fold the size of original.

4.13.5 XGdcTextureLoadExt16Fast [Loads texture pattern to the Graphics Memory]

Interface

GDC_BOOL **XGdcTextureLoadExt16Fast** (GDC_CTX **drvctx*,
const GDC_COL16 **lpTexture*, GDC_ULONG *adrs*)

Arguments

drvctx	Pointer to Context
lpTexture	Pointer to a texture pattern
adrs	Address of the memory where texture pattern is loaded, specify offset from the top of the Graphics Memory

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86293 or later

Description

Converts a 16bpp texture pattern to the format for fast Bi-Linear mode, and loaded it to the Graphics Memory. Specify *adrs* for offset from the top of the Graphics Memory.

Notes

* Prior to this command execution, sets the dimension of an image to be loaded by the **XGdcTextureDimension** command. Specify the dimension as it is (not 4 fold).
* The image converted for fast Bi-Linear mode gets 4 fold the size of original.

4.13.6 XGdcTextureDimension [Sets texture / tile information]

Interface

GDC_BOOL **XGdcTextureDimension** (GDC_CTX **drvctx*,
GDC_ULONG *adrs*, GDC_ULONG *oadrs*,
GDC_ULONG *w*, GDC_ULONG *h*)

Arguments

drvctx	Pointer to Context
adrs	Address of texture/tile pattern, specify the offset from top of the Graphics Memory
oadrs	Offset
w	Pattern data width (pixel unit, power of 2 only)
h	Pattern data height (pixel unit, power of 2 only)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_ILLEGAL_DIMENSION

Illegal vertical/horizontal size of pattern data

Target GDC

All

Description

Sets following texture information .

- Address of texture/tile pattern
- Offset
- Pattern data width
- Pattern data height

adrs and *oadrs* must be specified with suitable value for the loaded position of the referred pattern according to Table 4.13.6a.

Table 4.13.6a Values to be specified to *adrs* and *oadrs*

Stored position of referred pattern (*1)	adrs	oadrs
Internal texture	0	Offset from top address of internal Texture Memory
External texture	Address of texture pattern which is offset from top of the Graphics Memory	0
Internal tile	0	Offset from top address of internal Texture Memory
External tile (*2)	Base address of loaded area of tile pattern, specify the offset from the top of the Graphics Memory	Offset from base address of loaded area of tile pattern

[Note]

*1 Available destination address of texture/tile pattern is changed according to the Graphics Controller.

*2 External tile function is only for MB86293 or later with which internal Texture Memory is not equipped.

Range of pattern data width and pattern data height according to the Graphics Controller are shown in Table 4.13.6b.

Table 4.13.6b Range of pattern data width and height

	MB86290A/291/292	MB86293 or later
Internal Texture Memory	16,32,64	16,32,64
The Graphics Memory	16,32,64,128,256	16-4096 (power of 2 only)

4.13.7 XGdcBltTexture [Loads Blt texture to internal Texture Memory for MB86290A]

Interface

GDC_BOOL **XGdcBltTexture** (GDC_CTX **drvctx*,
GDC_ULONG *sadrs*, GDC_ULONG *sstride*,
GDC_USHORT *x*, GDC_USHORT *y*,
GDC_USHORT *w*, GDC_USHORT *h*, GDC_ULONG *oadrs*)

Arguments

drvctx	Pointer to Context
sadrs	Base address of a drawing frame where the texture pattern is installed (Offset from top of the Graphics Memory)
sstride	Stride (memory size of horizontal span) of the drawing frame [Range of the value] In 24bits color mode: 32 to 4096 in pixel unit In 16bits color mode: 16 to 4096 in pixel unit In 8bits color mode: 8 to 4096 in pixel unit
x	x coordinates of the top left corner of the texture pattern (0 to 4095)
y	y coordinates of the top left corner of the texture pattern (0 to 4095)
w	Width of the texture pattern (1 to 4095 in pixel unit)
h	Height of the texture pattern (1 to 4095 in pixel unit)
oadrs	Offset address from the top of Internal Texture Memory where texture pattern to be stored

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_ILLEGAL_DIMENSION

Illegal vertical/horizontal size of pattern data

Target GDC

All

Description

Loads a texture pattern from the Graphics Memory to Internal Texture Memory.

4.14 Binary Pattern Drawing Commands

4.14.1 XGdcBitPatternDraw [Draws binary pattern]

Interface

GDC_BOOL **XGdcBitPatternDraw** (GDC_CTX **drvctx*,
GDC_USHORT *x*, GDC_USHORT *y*,
GDC_USHORT *w*, GDC_USHORT *h*,
const GDC_BINIMAGE **lpPattern*)

Arguments

drvctx	Pointer to Context
x	x coordinate on the drawing frame where the top left point of the binary pattern is drawn, range 0 to 4095
y	y coordinate on the drawing frame where the top left point of the binary pattern is drawn, range 0 to 4095.
w	Width of the binary pattern, range 1 to 2016, in pixel unit
h	Height of the binary pattern, in pixels, 1 or more
lpPattern	Pointer to the binary pattern

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_DATA_TOO_BIG

Too large data

Target GDC

All

Description

Draws a binary pattern.

The binary pattern specified by *lpPattern* must be GDC_BINIMAGE format which packed in 32

pixels. Pixel with a value 1 is drawn in foreground color (specified by the **XGdcColor** command), and with a value 0 is drawn in background color (specified by the **XGdcBackColor** command).

When *w* is over 2016, sets error code **GDC_ERR_DATA_TOO_BIG** and returns **GDC_FALSE**. When *w* or *h* is 0, returns **GDC_TRUE** immediately**.**

Notes

DO NOT specify parameters (for *x*, *y*, *w* and *h*) which makes to draw to outside of drawing frame since clipping by hardware is not performed.

4.14.2 XGdcBitPatternDrawByte [Draws binary pattern]

Interface

GDC_BOOL **XGdcBitPatternDrawByte** (GDC_CTX **drvctx*,
GDC_ULONG *x0*, GDC_ULONG *y0*, int *x1*, int *y1*,
GDC_ULONG *w*, GDC_ULONG *h*,
const GDC_UCHAR **sadr*, GDC_ULONG *mwidth*)

Arguments

drvctx	Pointer to Context
x0	x coordinate in the binary pattern, range 0-2016
y0	y coordinate in the binary pattern, range 0-4095
x1	x coordinate on the drawing frame where the top left point of the binary pattern is drawn , range 0-4095
y1	y coordinate on the drawing frame where the top left point of the binary pattern is drawn , range 0-4095
w	Width of the binary pattern, in pixels, range 1-2016
h	Height of the binary pattern, in pixels, 1 or more
sadr	Pointer to the binary pattern
mwidth	Width of area where the binary pattern is installed, in bytes, 1 or more

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_DATA_TOO_BIG

Too large data

GDC_ERR_INVALID_PARAMETER

Invalid parameter has specified

Target GDC

All

Description

Draws binary pattern specified by rectangle (*x0*, *y0*) - (*x0*+*w*, *y0*+*h*) to drawing frame. This rectangle is a part of the binary pattern specified by *sadr*. Starting point (0,0) of the binary pattern is the top left corner of it. The pixel format of the binary pattern must be pack of 8 pixels a byte. Pixel with a value 1 is drawn in foreground color (specified by the **XGdcColor** command), and with a value 0 is drawn in background color (specified by the **XGdcBackColor** command).

When *w* is over 2016, sets error code **GDC_ERR_DATA_TOO_BIG** and returns **GDC_FALSE**. And also, when *x0*+*w* is over *mwidth**8, sets error code **GDC_ERR_INVALID_PARAMETER** and returns **GDC_FALSE.**

When *w* or *h* is 0, returns **GDC_TRUE** immediately**.**

x1 and *y1* can be specified in a signed 32bits value, but only the pixels within the range 0 - 4095 are drawn.

4.14.3 XGdcBitPatternMode [Sets enlarge/shrink mode]

Interface

GDC_BOOL **XGdcBitPatternMode** (GDC_CTX **drvctx*, GDC_UCHAR *mode*)

Arguments

drvctx Pointer to Context

mode Enlarge/shrink mode(GDC_BPSCALE_H and GDC_BPSCALE_V are applicable at the same time)

GDC_BPSCALE_H_EQUIV	Horizontal enlarge x1
GDC_BPSCALE_H_TWICE	Horizontal enlarge x2
GDC_BPSCALE_H_HALF	Horizontal enlarge x1/2
GDC_BPSCALE_V_EQUIV	Vertical enlarge x1
GDC_BPSCALE_V_TWICE	Vertical enlarge x2
GDC_BPSCALE_V_HALF	Vertical enlarge x1/2

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Sets enlarge/shrink mode for binary pattern drawing.

4.15 BLT Commands

4.15.1 XGdcBltCopy [Copies BitBlt pattern in current drawing frame]

Interface

GDC_BOOL **XGdcBltCopy** (GDC_CTX **drvctx*, GDC_ULONG *x0*, GDC_ULONG *y0*, int *x1*, int *y1*, GDC_ULONG *w*, GDC_ULONG *h*)

Arguments

drvctx	Pointer to Context
x0	x coordinate of the top left corner of source rectangle, range 0-4095
y0	y coordinate of the top left corner of source rectangle, range 0 to 4095
x1	x coordinate of the top left corner of destination, range 0 to 4095
y1	y coordinate of the top left corner of destination, range 0 to 4095
w	Width of rectangle, in pixels, range 1-4096
h	Height of rectangle, in pixels, range 1-4096

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_ILLEGAL_DIMENSION

Illegal vertical/horizontal size of pattern data

Target GDC

All

Description

Copies rectangle inside current drawing frame. Both the source and the destination are rectangle in current drawing frame.

When *w* or *h* is 4096 or more, sets error code **GDC_ERR_ILLEGAL_DIMENSION** and returns **GDC_FALSE**. And when *w* or *h* is 0, returns **GDC_TRUE** immediately**.**

x1 and *y1* can be specified in a signed 32bits value, but only the pixels within the range 0 - 4095 are drawn.

In MB86290A, the following values cannot be specified because of specification limitation.

- 2 <= *w* <= 4, in 16 bits color mode
- 2 <= *w* <= 8, in 8 bits color mode

4.15.2 XGdcBltCopyAlt[Sync] [Copies BitBlt pattern between any drawing frame]

Interface

GDC_BOOL **XGdcBltCopyAlt** (GDC_CTX **drvctx*,
GDC_ULONG *x0*, GDC_ULONG *y0*,
int *x1*, int *y1*,
GDC_ULONG *w*, GDC_ULONG *h*,
GDC_ULONG *sadr*, GDC_ULONG *sstride*,
GDC_ULONG *dadr*, GDC_ULONG *dstride*)

GDC_BOOL **XGdcBltCopyAltSync** (GDC_CTX **drvctx*,
GDC_ULONG *x0*, GDC_ULONG *y0*,
int *x1*, int *y1*,
GDC_ULONG *w*, GDC_ULONG *h*,
GDC_ULONG *sadr*, GDC_ULONG *sstride*,
GDC_ULONG *dadr*, GDC_ULONG *dstride*)

Arguments

drvctx	Pointer to Context
x0	The x coordinates of the top left corner of source rectangle, range 0 to 4095
y0	The y coordinates of the top left corner of source rectangle, range 0 to 4095
x1	The x coordinates of the top left corner of destination, range 0 to 4095
y1	The y coordinates of the top left corner of destination, range 0 to 4095
w	Width of rectangle, in pixels, range 1-4096
h	Height of rectangle, in pixels, range 1-4096
sadr	Base address of the source frame, specify the offset from top of the Graphics Memory
sstride	Stride (memory size of horizontal span) of source frame, in pixels, range 1-4096

dadr	Base address of the destination frame (Offset from top of the Graphics Memory)
dstride	Stride (memory size of horizontal span) of destination frame, in pixels

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_ILLEGAL_DIMENSION

Illegal vertical/horizontal size of pattern data

Target GDC

All

Description

Copies rectangle between different frames. Each frame must be on the Graphics Memory.

Color mode of source and destination must be the same.

The **XGdcBltCopyAltSync** command is synchronously executed to the vertical blanking interval. Source and destination field must not be overlapped to each other.

When *w* is 4096 or more, sets error code **GDC_ERR_ILLEGAL_DIMENSION** and returns **GDC_FALSE**. When *w* or *h* is 0, returns **GDC_TRUE** immediately**.**

x1 and *y1* can be specified in a signed 32bits value, but only the pixels within the range 0 - 4095 are drawn. Clipping operation by the **XGdcDrawClipFrame** command is not applicable.

In MB86290A, the following values cannot be specified because of specification limitation.

- 2 <= *w* <= 4, in 16 bits color mode
- 2 <= *w* <= 8, in 8 bits color mode

4.15.3 XGdcBltDraw[8/16/24] [Draws BitBlt pattern]

Interface

GDC_BOOL **XGdcBltDraw8** (GDC_CTX **drvctx*,
int *x*, int *y*,
GDC_ULONG *w*, GDC_ULONG *h*,
const GDC_COL8 *lpRect*)

GDC_BOOL **XGdcBltDraw16** (GDC_CTX **drvctx*,
int *x*, int *y*,
GDC_ULONG *w*, GDC_ULONG *h*,
const GDC_COL16 *lpRect*)

GDC_BOOL **XGdcBltDraw24** (GDC_CTX **drvctx*,
int *x*, int *y*,
GDC_ULONG *w*, GDC_ULONG *h*,
const GDC_COL24 *lpRect*)

Arguments

drvctx	Pointer to Context
x	x coordinate of the top left corner of destination, range 0 to 4095
y	y coordinate of the top left corner of destination, range 0 to 4095
w	Width of rectangle, in pixels, range 1 to 4096
h	Height of rectangle, in pixels, range 1 to 4096
lpRect	Pointer to the bitmap

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_DATA_TOO_BIG

Too large data

Target GDC

XGdcBltDraw8 : All

XGdcBltDraw16 : All

XGdcBltDraw24 : MB86293 or later (only in 24bpp color supported Graphics Controller)

Description

Copies bitmap from main memory to the current drawing frame.

Color mode of the bitmap is assumed to be the same as that of current drawing frame.

w or *h* is 0, returns **GDC_TRUE** immediately**.**

When *w* is over 4096, sets error code **GDC_ERR_DATA_TOO_BIG** and returns **GDC_FALSE**. *x* and *y* can be specified in a signed 32bits value, but only the pixels within the range 0 - 4095 are drawn.

4.15.4 XGdcBltFill [Fills BitBlt field]

Interface

GDC_BOOL **XGdcBltFill** (GDC_CTX **drvctx*,
int *x*, int *y*,
GDC_ULONG *w*, GDC_ULONG *h*)

Arguments

drvctx	Pointer to Context
x	The x coordinates of the top left corner of rectangle to be filled, range 0-4095
y	The y coordinates of the top left corner of rectangle to be filled, range 0-4095
w	Width of the rectangle to be filled, in pixels, range 1-4096
h	Height of the rectangle to be filled, in pixels, range 1-4096

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Fills a rectangle with the foreground color specified by the **XGdcColor** command.

When *w* or *h* is 0, returns **GDC_TRUE** immediately**.** x and y can be specified in a signed 32bits value, but only the pixels within the range 0 - 4095 are drawn.

4.15.5 XGdcBltColorTransparent [Sets transparent color of transparent BitBlt]

Interface

GDC_BOOL **XGdcBltColorTransparent** (GDC_CTX **drvctx*, GDC_COLOR32 *color*)

Arguments

drvctx	Pointer to Context
color	Color code to be treated as transparent

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

MB86291 or later

Description

Sets the transparent color for Blt function command.

The value of *color* is used according to color mode as follows.

- 8 bits color mode : lower 8 bits is used
- 16 bits color mode : lower 16 bits is used
- 24 bits color mode : lower 24 bits is used

4.15.6 XGdcBltCopyAltAlpha [Copies BitBlt pattern with alpha blending]

Interface

GDC_BOOL **XGdcBltCopyAlt**Alpha (GDC_CTX **drvctx*,
GDC_ULONG *x0*, GDC_ULONG *y0*,
GDC_ULONG *x1*, GDC_ULONG *y1*,
GDC_ULONG *bx*, GDC_ULONG *by*,
GDC_ULONG *w*, GDC_ULONG *h*,
GDC_ULONG *sadr*, GDC_ULONG *sstride*,
GDC_ULONG *bstride*)

Arguments

drvctx	Pointer to Context
x0	x coordinate of the top left corner of source rectangle, range 0-4095
y0	y coordinate of the top left corner of source rectangle, range 0-4095
x1	x coordinate of the top left corner of destination, range 0-4095
y1	y coordinate of the top left corner of destination, range 0-4095
bx	x coordinate of the top left corner of the alpha map, range 0-4095
by	y coordinate of the top left corner of the alpha map, range 0-4095
w	Width of rectangle, in pixels, range 1-4096
h	Height of rectangle, in pixels, range 1-4096
sadr	Base address of the source frame, specify the offset from top of the Graphics Memory
sstride	Stride (memory size of horizontal span) of source frame, in pixels, range 1-4096
bstride	Stride of the alpha map area, in pixels, range 1-4096

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_ILLEGAL_DIMENSION

Illegal vertical/horizontal size of pattern data

Target GDC

MB86293 or later

Description

Execute alpha blending with source Blt area and destination area.

Copies alpha blended area to the *x1*, *y1* coordinates of current drawing frame.

Source Blt area is specified with *sadrs*, *sstride*, *x0*, *y0*, *w* and *h*. And destination Blt area is specified with *x1* and *y1*.

Alpha blend coefficient area is specified with *bstride*, *bx*, *by*, *w* and *h*. And its top address is specified by the **XGdcSetAlphaMapBase** command.

When *w* or *h* is 4096 or more, sets error code **GDC_ERR_ILLEGAL_DIMENSION** and returns **GDC_FALSE**. When *w* or *h* is 0, returns **GDC_TRUE** immediately**.**

Notes

Do not specify a value outside the range 0-4095 for *x0*, *y0*, *x1*, *y1*, *bx* and *by*. If this works, the result is not guaranteed.

4.16 Execution Control Commands

4.16.1 XGdcVerticalSync [Generates Sync command]

Interface

GDC_BOOL **XGdcVerticalSync** (GDC_CTX *_drvctx_)

Arguments

drvctx	Pointer to Context

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Generates a Display List Command for waiting VSYNC (Sync command), and adds it to the end of current Display List.

Display Lists after Sync command are executed synchronously with VSYNC.

4.16.2 XGdcInterrupt [Generates Interrupt command]

Interface

GDC_BOOL **XGdcInterrupt** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Description

Generates a Display List Command for requesting interrupt (Interrupt command), and adds it to the end of current Display List.

4.17 Drawing Attributes Restore Command

4.17.1 XGdcRestoreAttr [Restores drawing attributes]

Interface

GDC_BOOL **XGdcRestoreAttr** (GDC_CTX **drvctx*, GDC_ULONG *attr*)

Arguments

drvctx Pointer to Context

attr Specify attributes to be restored in the following, use OR operation to specify two or more

GDC_RESTORE_COMMON

Common drawing attributes

GDC_RESTORE_GEOMETRY

Geometry drawing attributes

GDC_RESTORE_EXTEND

Extended drawing attributes (for MB86293 or later)

GDC_RESTORE_ALL

All of them

Return value

GDC_TRUE Complete

GDC_FALSE Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

GDC_ERR_INVALID_ATTRIBUTE

Invalid attribute has specified

Target GDC

All

Description

Generates Display List for setting Graphics Controller to drawing attributes in the Context.

In the system which Driver Commands are used in two or more tasks, drawing attributes of Graphics Controller set by a task may be changed by other tasks. Therefore, after a executing of the **XGdcFlush**/**XGdcFlushEx** command, it is necessary to re-set up each drawing attribute. By using this command, multiple attributes can be restored at once.

Macros to be specified for *attr* are as follows. And also, attributes to be restored by each macro

are shown in Table 4.17.1.

- GDC_RESTORE_COMMON

Means common attribute in all Graphics Controllers, concerns drawing frame, clipping, device coordinates, BLT, etc.

- GDC_RESTORE_GEOMETRY

Means object coordinates drawing, available in MB86291 or later.

- GDC_RESTORE_EXTEND

Means attributes for MB86293 or later, shadow, border, device coordinate log, etc.

- GDC_RESTORE_ALL

Means all of drawing attribute. However, this command cannot restore the following attributes. Use suitable command for these attributes if you need restore them.

+ Vertex color for Gouraud shading (use the **XGdcColor** or the **XGdcVertexColor[32/3f]** command)
+ Texture/tile pattern loaded in the Graphics Memory or internal Texture Memory (use the **XGdcTextureLoadInt[8/16/24]**, the **XGdcTextureLoadExt[8/16/24]**, the **XGdcTextureLoadInt16Fast**, the **XGdcTextureLoadExt16Fast** or the **XGdcBltTexture** command)
+ 4x4 matrix (use the **XGdcGeoLoadMatrix[f]** command)

Table 4.17.1 Drawing attributes to be restored

No.	Macros	Drawing attributes	Corresponding commands
1	GDC_RESTORE_COMMON	Color mode	XGdcDrawDimension
2		Clipping mode	XGdcClipMode
3		Scaling mode of binary pattern	XGdcBitPatternMode
4		Z precision	XGdcSetZPrecision
5		For lines on device coordinate system	XGdcSetAttrLine
6		For surfaces on device coordinate system	XGdcSetAttrSurf
7		For texture mapping	XGdcSetAttrTexture
8		For BitBLT	XGdcSetAttrBlt
9		Base address of drawing frame	XGdcDrawDimension
10		Width of drawing frame	XGdcDrawDimension
11		Base address of Z Buffer	XGdcBufferCreateZ
12		Base address of texture memory	XGdcTextureDimension
13		Base address of polygon flag buffer	XGdcBufferCreateC
14		Minimum x value of clipping frame	XGdcDrawClipFrame
15		Maximum x value of clipping frame	XGdcDrawClipFrame
16		Minimum y value of clipping frame	XGdcDrawClipFrame
17		Maximum y value of clipping frame	XGdcDrawClipFrame
18		Dimension of texture pattern	XGdcTextureDimension
19		Dimension of tile pattern	XGdcTextureDimension
20		Address texture/tile pattern	XGdcTextureDimension
21		Drawing color	XGdcColor
22		Background color	XGdcBackColor
23		Alpha blending coefficient	XGdcSetAlpha
24		Broken line pattern	XGdcSetLinePattern
25		Texture border color	XGdcSetTextureBorder
26	GDC_RESTORE_GEOMETRY	Transparent color of BitBLT	XGdcBltColorTransparent
27		NdcDc conversion coefficient for x and y, type GDC_FIXED32	XGdcGeoNdcDcViewportCoef
28		NdcDc conversion coefficient for x and y, type GDC_SFLOAT	XGdcGeoNdcDcViewportCoeff
29		NdcDc conversion coefficient for z, type GDC_FIXED32	XGdcGeoNdcDcDepthCoef
30		NdcDc conversion coefficient for z, type GDC_SFLOAT	XGdcGeoNdcDcDepthCoeff
31		x,y coordinate for view volume clipping, type GDC_FIXED32	XGdcGeoViewVolumeXYClip
32		x,y coordinate for view volume clipping, type GDC_SFLOAT	XGdcGeoViewVolumeXYClipf
33		z coordinate for view volume clipping, type GDC_FIXED32	XGdcGeoViewVolumeZClip
34		z coordinate for view volume clipping, type GDC_SFLOAT	XGdcGeoViewVolumeZClipf
35		w coordinate for view volume clipping, type GDC_FIXED32	XGdcGeoViewVolumeWminClip
36		w coordinate for view volume clipping, type GDC_SFLOAT	XGdcGeoViewVolumeWminClipf
37		For lines on object coordinate system	XGdcSetAttrLine
38		For surfaces on object coordinate system	XGdcGeoSetAttrSurf
39	GDC_RESTORE_EXTEND	Base address of alpha map	XGdcSetAlphaMapBase
40		For extended lines	XGdcGeoSetAttrLine
41		Number of pixels for broken lines fixation	XGdcGeoSetAttrLine
42		For extended surfaces	XGdcGeoSetAttrSurf
43		For shadow of lines	XGdcSetAttrLine
44		For shadow of surfaces	XGdcSetAttrSurf
45		Shadow color	XGdcGeoShadowColor
46		Background color of shadow	XGdcGeoShadowBackColor
47		For border of lines	XGdcSetAttrLine
48		Border color of lines	XGdcGeoBordeColor
49		Background border color of lines	XGdcGeoBordeBackColor
50		For surfaces of non top-left primitives	XGdcSetAttrSurf
51		For lines	XGdcSetAttrLine
52		For surfaces	XGdcSetAttrSurf
53		x,y offset of shadow	XGdcGeoShadowXY
54		x,y offset of shadow composition	XGdcGeoShadowXY
55		Z value of body, shadow, border and correction primitive in non top-left mode	XGdcGeoOverlapZ
56		Base address of area to be outputted device coordinate logs	XGdcGeoSetLogOutBase

4.18 Video Capture Commands

Please use video capture command (**GdcCap***) in this section after initializing a digital video decoder with an I²C control command (**GdcI2C***), when using the digital video decoder connected with an I²C interface. For details information, refer to the "Application Note".

4.18.1 GdcCapSetVideoCaptureMode [Sets mode of video capture]

Interface

void **GdcCapSetVideoCaptureMode** (GDC_ULONG *mode*)

Arguments

mode Sets modes of video capture. The value is set to VCM (Video Capture Mode) register as it is. For details information about VCM register, refer to the Graphics Controller hardware specifications of using.
Macros representing each mode are prepared. These can be used as the need arises. Sets each mode by combining (use OR operation) macros in Table 4.18.1.

Table 4.18.1 Video capture mode control macro list

Macros	Meaning
GDC_CAP_START	Starts capturing
GDC_CAP_STOP	Stops capturing
GDC_CAP_ENABLE_V_INTERPOLATION	Performs the interpolation of perpendicular direction
GDC_CAP_DISABLE_V_INTERPOLATION	NOT perform the interpolation of perpendicular direction
GDC_CAP_NTSC	Video= NTSC
GDC_CAP_PAL	Video=PAL

Return value

None

Target GDC

MB86291 or later

Description

Sets video capture mode by setting VCM(Video Capture Mode) register to *mode*.

4.18.2 GdcCapGetErrorStatus [Gets error status of video capture]

Interface

GDC_ULONG **GdcCapGetErrorStatus** (void)

Arguments

None

Return value

Video Capture Status (VCS) register in the following format:

31	- - -	4	3	2	1	0
	- - -	x	x	x	x	x

bit 0-4: Error status Yes = except 00000, No = all 0

All other bits: Don't care

Figure 4.18.2 Error status of video capture

Target GDC

MB86291 or later

Description

Reads VCS (Video Capture Status) register and returns error status.

4.18.3 GdcCapClearErrorStatus [Clears error status of video capture]

Interface

void **GdcCapClearErrorStatus** (void)

Arguments

None

Return value

None

Target GDC

MB86291 or later

Description

Clears error status by writing VCS (Video Capture Status) register to 0.

4.18.4 GdcCapSetVideoCaptureBuffer [Sets video capture buffer]

Interface

void **GdcCapSetVideoCaptureBuffer** (GDC_ULONG *saddr*, GDC_ULONG *eaddr*, GDC_ULONG *stride*)

Arguments

saddr	Top address of the video capture buffer (Offset from top of the Graphics Memory)
eaddr	Bottom address +1 of the video capture buffer (Offset from top of the Graphics Memory)
stride	Stride of video capture buffer (in blocks of 64 bytes)

Return value

None

Target GDC

MB86291 or later

Description

Sets video capture buffer.

The start address needs to be in a 16 bytes boundary.

Please specify the end address +1 of the video capture buffer as the end address.

The video capture buffer size need a size which is a part for the picture to take at least.

4.18.5 GdcCapSetImageArea [Sets range of image]

Interface

void **GdcCapSetImageArea** (GDC_USHORT *x0*, GDC_USHORT *y0*,
GDC_USHORT *x1*, GDC_USHORT *y1*)

Arguments

x0	The upper left x coordinate of the picture (range 0-4095)
y0	The upper left y coordinate of the picture (range 0-4095)
x1	The lower right x coordinate of the picture (range 0-4095)
y1	The lower right y coordinate of the picture (range 0-4095)

Return value

None

Target GDC

MB86291 or later

Description

Sets the dimension of the image to be stored in the video capture buffer.

The part of inputted picture which from (*x0*, *y0*) and (*x1*, *y1*) is stored in the video capture buffer. The starting point (0,0) is upper left corner of the inputted picture.

Please set coordinates *x0*<*x1* and *y0*<*y1* to specify the range of the picture.

The number of pixels of the image width from *x0* and *x1* must be even.

4.18.6 GdcCapSetDisplaySize [Sets dimension of captured image for scaling]

Interface

GDC_BOOL **GdcCapSetDisplaySize** (GDC_ULONG *w*, GDC_ULONG *h*)

Arguments

w	Width of scaled image (in pixels, range 1-4096)
h	Height of scaled image (in pixels, range 1-4096)

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_ILLEGAL_CAPTURE_SIZE

Illegal video capture size has specified

GDC_ERR_ILLEGAL_CAPTURE_SCALE

Illegal video capture scale has specified

Target GDC

MB86294

Description

Sets dimension of the captured image which is scaled.

Specify dimension of original captured image by the **GdcCapSetImageArea** command before the calling of this command.

If 0 is specified for *w* or *h*, sets error code **GDC_ERR_ILLEGAL_CAPTURE_SIZE**, and returns **GDC_FALSE**.

4.18.7 GdcCapGetImageAddress [Gets address of captured image]

Interface

GDC_ULONG *__GdcCapGetImageAddress__ (void)

Arguments

None

Return value

Top address of captured image (offset address from the top of the Graphics Memory)

Target GDC

MB86294

Description

Returns the top address of the captured image, which is the value of register L1DA.

Before the calling of this command, Stop the video capturing by the **GdcCapSetVideoCaptureMode** command as shown below, because the top address of captured image is progressed while the video capture is running.

```
GDC_ULONG *adrs;
GdcCapSetVideoCaptureMode(GDC_CAP_STOP); /* Stops the video capturing */
adrs = GdcCapGetImageAddress(); /* Gets address of captured image */
```

4.18.8 GdcCapSetWindowMode [Sets w-layer mode]

Interface

void **GdcCapSetWindowMode** (GDC_ULONG *format*, GDC_ULONG *mode*)

Arguments

format Color format of layer W, Sets to YC modes when using video capture

GDC_CAP_RGB_MODE	RGB mode (default)
GDC_CAP_YC_MODE	YC mode

mode Sets whether layer W is used as a normal display layer or a video capture

GDC_CAP_NORMAL_MODE	Normal mode (default)
GDC_CAP_CAPTURE_MODE	Capture mode

Return value

None

Target GDC

MB86291 or later

Description

Sets mode of layer W. When using video capture, be sure to set mode.

Before execute this command, sets attribute of layer W by the **GdcDispDimension** command beforehand. Color mode supports only 16 bits mode.

4.18.9 GdcCapSetVideoCaptureScale [Sets scale of video capture]

Interface

void **GdcCapSetVideoCaptureScale** (GDC_FIXED_SCALE *hscale*,
GDC_FIXED_SCALE *vscale*)

Arguments

hscale	Horizontal scaling rate (default = 0x0800)
vscale	Vertical scaling rate (default = 0x0800)

Return value

None

Target GDC

MB86291/86292/86293

Description

Sets scales for reducing video capture.

Sets horizontal scaling rate (*hscale*) which is cast to type GDC_FIXED_SCALE.

Sets vertical scaling rate(*vscale*) which is cast to type GDC_FIXED_SCALE.

Specify *hscale* for horizontal scaling rate which the value divided number of horizontal pixels of captured image by scaled image's one. *vscale* is vertical scaling rate which the value divided number of vertical pixels of captured image by scaled image's one.

The type GDC_FIXED_SCALE is fixed point format which consists of 5 bits integer and 11 bits fraction. The range of the value is as follows.

0xffff (1/31.99951171875) - 0x0800 (1.0)

Initial value of *hscale* and *vscale* is 0x0800(no scaling) respectively.

Examples of calculation of scaling rate are shown below.

[Example of calculation of scaling rate]

(1)When the image of size 720*576 is reduced to the size 648*490, scaling rate is calculated as below.

--Reduction of horizontal direction

720pixel to 648pixel

720/648=1.111

1.111*2048=2275 (expressed in hexadecimal is 0x08e3)

--Reduction of vertical direction

576line to 490line

576/490=1.176

1.176*2048=2275 (expressed in hexadecimal is 0x0968)

--A value to be set to *hscale* is 0x08e3.

--A value to be set to *vscale* is 0x0968.

(2)When 1/n times, scaling rate is calculated as below.

--*hscale* = n * 2048

--*vscale* = n * 2048

4.18.10 GdcCapSetAttrMisc [Sets attribute of video capture]

Interface

void **GdcCapSetAttrMisc** (GDC_ULONG *target*, GDC_ULONG *param*)

Arguments

target Video capture attribute

GDC_CAP_ODD_MODE	Capturing method
GDC_CAP_CNV_MODE	Non-interlace conversion mode

param Parameter corresponding to *target* (*1)

Return value

None

Target GDC

MB86291 or later

Description

Sets attribute of video capture.

*1 Video capture attribute (*target*) and parameter (*param*) corresponding to each video capture attributes are shown below.

[Explanatory notes]

Video capture attribute	Description of video capture attributes
Parameter 1 that can set	Description of parameter 1
Parameter 2 that can set	Description of parameter 2
:	:

GDC_CAP_ODD_MODE	Specifies the capture method.
GDC_CAP_EVEN_AND_ODD_MODE	Captures both the odd number and the even number fields (default).
GDC_CAP_ODD_ONLY_MODE	Captures only the odd number field.

GDC_CAP_CNV_MODE	Specifies the non-interlace conversion mode of the picture which is captured.
GDC_CAP_CNV_BOB_MODE	BOB mode (*2).
GDC_CAP_CNV_WEAVE_MODE	WEAVE mode (default) (*3).

(*2) BOB mode :The mode is a frame which is the even field of the raster is averaged interpolation then it is added to the odd field.

(*3) WEAVE mode :The mode is a frame which is the odd field and the even field merge on the video capture buffer.

4.18.11 GdcCapSetInputDataCountNTSC [Sets the video capture buffer for NTSC]

Interface

void **GdcCapSetInputDataCountNTSC** (GDC_ULONG *blank_data*,
GDC_ULONG *valid_data*)

Arguments

blank_data	The horizontal blanking interval, specified in number of dot clock cycles
valid_data	Number of data in the term of validity, specified in number of dot clock cycles

Return value

None

Target GDC

MB86291 or later

Description

Sets the input video stream number at the time of NTSC format.

This command is used to detect an error occurred. When the input data is not same as the value set up by this command, an error occurs. The video capture status becomes the value other than zero at this time.

Also, capturing is continued when the error occurred.

4.18.12 GdcCapSetInputDataCountPAL [Sets the video capture buffer for PAL]

Interface

void **GdcCapSetInputDataCountPAL** (GDC_ULONG *blank_data*,
GDC_ULONG *valid_data*)

Arguments

blank_data	The horizontal blanking interval, specified in number of dot clock cycles
valid_data	Number of data in the term of validity, specified in number of dot clock cycles

Return value

None

Target GDC

MB86291 or later

Description

Sets the input video stream number at the time of PAL format.

This command is used to detect an error occurred. When the input data is not same as the value set up by this command, an error occurs. The video capture status becomes the value other than zero at this time.

Also, capturing is continued when the error occurred.

4.18.13 GdcCapSetLPFMode [Sets low pass filter mode]

Interface

void **GdcCapSetLPFMode** (GDC_ULONG *vlpf_y*, GDC_ULONG *vlpf_c*,
GDC_ULONG *hlpf_y*, GDC_ULONG *hlpf_c*)

Arguments

vlpf_y	The vertical LPF coefficient code for luminance signals (range 0-2)
vlpf_c	The vertical LPF coefficient code for chrominance signals (range 0-2)
hlpf_y	The horizontal LPF coefficient code for luminance signals (range 0-3)
hlpf_c	The horizontal LPF coefficient code for chrominance signals (range 0-3)

Return value

None

Target GDC

MB86291 or later

Description

The mode (formula) of the low pass filter at the time of a video capture is set up.

Coefficients in Table 4.18.13a are available for *vlpf_y* and *vlpf*_c. When a coefficient is 0, it outputs as it is, without covering a low pass filter. The initial value of *vlpf_y* and *vlpf_c* are 0.

Table 4.18.13a Low pass filter formula (for *vlpf_y* and *vlpf_c*)

Coefficient	Low pass filter formula
0	y[i,j] = x[i,j]
1	y[i,j] = x[i,j-1]/4 + x[i,j]/2 + x[i,j+1]/4
2	y[i,j] = x[i,j-1]*3/16 + x[i,j]*5/8 + x[I,j+1]*3/16

Note:

x[i,j] : input pixel value at i-th column and j-th raster

y[i,j] : output pixel value at i-th column and j-th raster

Coefficients in Table 4.18.13b are available for *hlpf_y* and *hlpf*_c. When a coefficient is 0, it outputs as it is, without covering a low pass filter. The initial value of *hlpf_y* and *hlpf_c* are 0.

Table 4.18.13b Low pass filter formula (for hlpf_y and hlpf_c)

Coefficient	Low pass filter formula
0	y[i,j] = x[i,j]
1	y[i,j] = x[i-1,j]/4 + x[i,j]/2 + x[i+1,j]/4
2	y[i,j] = x[i-1,j]*3/16 + x[i,j]*5/8 + x[i+1,j]*3/16
3	y[i,j] = x[i-2,j]*3/32 + x[i-1,j]/4 + x[i,j]*5/16+ x[I+1,j]/4 + x[i+2,j]*3/32

Note:

x[i,j] : input pixel value at i-th column and j-th raster

y[i,j] : output pixel value at i-th column and j-th raster

4.19 I²C Control Commands

4.19.1 GdcI2CGetBusStatus [Gets I²C bus status]

Interface

GDC_ULONG **GdcI2CGetBusStatus** (void)

Arguments

None

Return value

I²C bus status in the following format (value of BSR register):

bit 31	---	7	6	5	4	3	2	1	0
		x	x	x	x	x	x	x	x

bit7:Detects START/STOP condition
0:STOP condition
1:START condition (The bus is in use)
bit6:Detects repeated START condition
0:Repeated START condition was not detected
1:START condition was detected again while bus is in use
bit5:Indicates Arbitration lost
0:Arbitration lost was not detected
1:Arbitration lost occurred during master transmission
bit4:Status of Acknowledge
0:No acknowledge
1:Acknowledge
bit3:Status of data transfer
0:Receiving status
1:Transmitting status
bit2:Detects addressing
0:Addressing was not performed in a slave mode
1:Addressing was performed in a slave mode
bit1:Detects “General call address (00h)”
0:”General call address(00h)” was not received in a slave mode
1:”General call address(00h)” was received in a slave mode
bit0:Detects the first byte
0:Received data is not the 1st byte
1:Receiverd data is the 1st byte (address data)
All other bits: Don't care

Figure 4.19.1 I²C bus status

Target GDC

MB86291S, MB86292S and MB86293 or later with I²C

Description

Reads BSR (Bus Status Register) and returns I^2C bus status.

4.19.2 GdcI2CSetBusControl [Controls I²C bus]

Interface

void **GdcI2CSetBusControl** (GDC_UCHAR *param*)

Arguments

param The parameter for I²C bus control (the following format)

bit 7	6	5	4	3	2	1	0
x	x	x	x	x	x	x	x

bit7:Flag bit for request of bus error interruption
0:Clears a request of bus error interruption
1:Don't care
bit6:Permits bus error interruption
0:Prohibition of bus error interruption
1:Permission of bus error interruption
bit5:Generates START condition
0:Don't care
1:START condition is generated again at the time of master transmission
bit4:Selects master/slave mode
0:Becomes a slave mode after generation of STOP condition and completing transfer
1:Becomes a master mode, generates START condition and starts transfer
bit3:Permits generation of acknowledge at the time of data reception
0:Acknowledge is not generated
1:Acknowledge is generated
bit2: Permits generation of acknowledge at the time of "General call address(00h)" reception
0: Acknowledge is not generated
1: Acknowledge is generated
bit1:Permits interruption
0:Prohibition of interrupt
1:Permission of interrupt
bit0:Flag bit for request of interruption for transfer end
0:Clears the flag
1:Don't care

Figure 4.19.2 Parameter for I²C bus control

Return value

None

Target GDC

MB86291S, MB86292S and MB86293 or later with I²C

Description

Controls I^2C bus by writing BCR (Bus Control Register) to *param*.

4.19.3 GdcI2CGetBusControlStatus [Gets I²C bus control status]

Interface

GDC_ULONG **GdcI2CGetBusControlStatus** (void)

Arguments

None

Return value

I²C bus control status in the following format (value of BCR register):

bit 7	6	5	4	3	2	1	0
x	x		x	x	x	x	x

bit7: Flag bit for request of bus error interruption
0:A bus error was not detected
1:Invalid START condition or STOP condition was detected while data transfer
bit6: Permission of bus error interruption
0:Prohibition of bus error interruption
1:Permission of bus error interruption
bit5: Don't care
bit4: Master / slave mode
0:Stops data transfer with STOP condition, and shift to slave mode
1:Shift to master mode, and starts data transfer with START condition.
bit3: Permission of generating acknowledge at the time of data reception
0:Acknowledge is not generated
1:Acknowledge is generated
bit2: Permission of generating acknowledge at the time of “General call address (00h)” reception
0:Acknowledge is not generated
1:Acknowledge is generated
bit1: Permission of interruption
0:Prohibition of interrupt
1:Permission of interrupt
bit0: Flag bit for request of interruption for transfer end
0:The transfer is not ended
1:It is set when 1 byte transfer including the acknowledge bit is completed and it corresponds to the following conditions
- It is a bus master
- It is an addressed slave
- It received “General call address (00h)”
- It was going to generate START condition while other systems by which arbitration lost happened used the bus

Figure 4.19.3 I²C bus control status

Target GDC

MB86291S, MB86292S and MB86293 or later with I^2C

Description

Reads BCR (Bus Control Register) and returns I^2C bus control status.

4.19.4 GdcI2CSetClock [Sets I²C clock]

Interface

void **GdcI2CSetClock** (GDC_UCHAR *param*)

Arguments

param Parameter for setup of I²C clock (the following format)

bit 7	6	5	4	3	2	1	0
0	x	x	x	x	x	x	x

bit6:Selects standard-mode / high-speed-mode
0:Standard-mode
1:High-speed-mode
bit5:Permits I²C operation
0:Prohibition of operation
1:Permission of operation
bit4-0:Frequency of a transfer clock
00000-11111
All other bits: 0

Figure 0 I²C clock parameter

For details information about I²C clock, refer to the chapter of the "clock control register" of "I²C Interface specification".

Return value

None

Target GDC

MB86291S, MB86292S and MB86293 or later with I²C

Description

Sets I²C clock by writing CCR (Clock Control Register) to *param*.

4.19.5 GdcI2CSetData [Sets transfer data]

Interface

void **GdcI2CSetData** (GDC_UCHAR *param*)

Arguments

param	A data to be transferred

Return value

None

Target GDC

MB86291S, MB86292S and MB86293 or later with I^2C

Description

Sets a data to be transferred by writing DAR(DAta Register) to *param*.

5 System Dependent Commands

The list of System Dependent Command is shown. An application developer needs to create these commands.

5.1 System Dependent Commands

System Dependent Command list is shown in Table 5.1.

Table 5.1 System Dependent Command list

No.	Command name	Function
1	**GdcFlushDisplayList**	Transfer Display List
2	**XGdcSwitchDLBuf**	Changes DL Buffer
3	**GdcWait**	Waiting routine

6 System Dependent Command interface

This chapter explains System Dependent Command API in the form of the following.

Interface

Function Interface.

Arguments

Description of arguments.

Return value

The return values and the description of them.

Error code

Error code at when the command returns abnormally.

This item is omitted if the command has no return value.

Target GDC

Target Graphics Controllers of the command.

Called by

The command name of the calling.

Description

Description of the command.

Notes

Notes.

6.1 Command Interface

6.1.1 GdcFlushDisplayList [Transfers a Display List]

Interface

void **GdcFlushDisplayList** (GDC_ULONG **src*, GDC_ULONG *count*)

Arguments

src	Source address (Display List buffer)
count	Transfer count (Range is from 1 to 4294967295)

Return value

None

Target GDC

All

Called by

The **XGdcFlush** command

The **XGdcFlushEx** command

Description

This command is to transfer a Display List of the size specified by “count” started from the source address specified by “*src*”. The “*src*” specifies the Display List buffer address mapped to the host CPU address field. The unit of “*count*” is what specified by the **GdcSetDMAMode** command (32 bytes or 4 bytes). If the **GdcSetDMAMode** command is not applied since DMA is not used, this unit is set to 4 bytes. For the Display List transfer, the following three methods are applicable. For each procedure, refer to the description [Display List transfer procedure] as follows:

- DMA transfer
- Transfer of Local Display List
- CPU transfer

This command can be used by all Graphics Controller.

[Display List transfer procedure]

*** DMA transfer**

This is a method of Display List transfers utilizing the DMA controller of the host CPU (the Graphics Controller does not contain a DMA controller). The operation procedure of this case is shown as follows. Prior to call this command, DMA transfer mode must be appropriately set on both DMAC (the host CPU) and the Graphics Controller.

(1) Checks DMA transfer enable/disable
- Checks the appropriate operation mode check of the DMAC and wait till it will be ready to accept a new DMA transaction request.

(2) Sets DMA (According to the applied procedure for the DMAC, set the following parameter)
- Source address (the address specified in “src”).
- Destination address (Display List FIFO of the Graphics Controller).
- Transfer count (the value specified in “*count*”).

(3) Sets transfer count (the MB86290 Series side)
- Sets DMA transfer count (the value specified in “*count*”) to DTC (DMA Transfer Count) register.

(4) Starts DMA transaction
- Appropriate start up operation for the applied DMA controller.

(5) Issues the DMA request
- Sets 1 to DRQ (DMA ReQuest) register.

(6) Waits for the completion of the DMA transfer
- 1 block configuration Display List buffer is applied, wait till the end of DMA transaction.

[Remark]

When the unit of transfer count is 32byte, if the total byte size of the Display List is not a multiple of 32byte, the Driver Command fills appropriate number of NOP and makes the size to be a multiple of 32byte.

[Example]

```
/* Start address of the host interface register field */
/* Please set a suitable value to the following ########
   according to the environment of use*/
#define HOSTBASE 0x########

/* Start address of drawing control register field */
/* Please set a suitable value to the following ########
   according to the environment of use*/
#define DRAWBASE          0x########

#define WRITE_DTC(i)      ( *(GDC_ULONG*)(HOSTBASE+0x00) = (i) )
#define WRITE_DRQ(i)      ( *(GDC_ULONG*)(HOSTBASE+0x18) = (i) )

#ifdef GDC_MB86290A
#define FIFO_ADDRESS      (DRAWBASE+0x4a0) /* for MB86290A */
#else
#define FIFO_ADDRESS      (DRAWBASE+0x8400) /* for MB86291 or later */
#endif

void GdcFlushDisplayList(GDC_ULONG *src, GDC_ULONG count){
    /* Polling for DMA ready DMA */
    /* Please create DMA_BUSY function according to the environment of use */
    while( DMA_BUSY() );

    /* Sets transfer count */
    /* Please create SET_DMA_COUNT function according to the environment of use */
    SET_DMA_COUNT(CHANNEL0, count);

    /* Sets source address */
    /* Please create SET_DMA_SRC function according to the environment of use */
    SET_DMA_SRC(CHANNEL0, src);

    /* Sets destination address */
    /* Please create SET_DMA_DEST function according to the environment of use */
    SET_DMA_DEST(CHANNEL0, FIFO_ADDRESS);

    /* Sets transfer count (Graphics Controller) */
    WRITE_DTC(count);

    /* Trigger of DMA transaction */
    /* Please create DMA_START function according to the environment of use */
    DMA_START();

    /* Issue of external DMA request */
    WRITE_DRQ(1);

#ifdef SINGLE_DL_BUFFER
    /* Wait for the next Display List buffer write to be ready */
    while( DMA_BUSY() );
#endif
}
```

***Transfer of Local Display List**

This is a method of the Display List transfers utilizing the bus master function of the Graphics Controller. Transfer count is 4byte unit. In this case, the Display List buffer must be located to the Graphics Memory of the Graphics Controller. And the source address "src" must be converted to the local address of the Graphics Controller. The operation procedure of this case is shown as follows:

(1) Checks transfer enable/disable
- Checks the status of LSTA (display List transfer STAtus) register and wait until it will be 0.

(2) Sets source address
- Sets the source address to LSA (display List Source Address) register. The address to be set to this register is
("src" value) – (start address of host interface register field)

(3) Sets transfer count
- Sets the transfer count ("count" value) to LCO (display List COunt) register.

(4) Starts the transaction
- Sets 1 to LREQ (display List transfer REQuest) register.

(5) Waits for the completion of the transfer (in case of single DL Buffer mode)
- Same as (1).

[Example]

```
/* Start address of host interface register field */
/* Please set a suitable value to the following ########
   according to the environment of use*/
#define HOSTBASE            0x########

/* Start address of the Graphics Memory field */
/* Please set a suitable value to the following ########
   according to the environment of use*/
#define MB86290_BASE        0x########

#define READ_LSTA()         *((volatile GDC_ULONG*)(HOSTBASE+0x10))
#define WRITE_LSA(i)        ( *((GDC_ULONG*)(HOSTBASE+0x40)) = (i) )
#define WRITE_LCO(i)        ( *((GDC_ULONG*)(HOSTBASE+0x44)) = (i) )
#define WRITE_LREQ(i)       ( *((GDC_ULONG*)(HOSTBASE+0x48)) = (i) )

void GdcFlushDisplayList(GDC_ULONG *src, GDC_ULONG count){
    GDC_ULONG       src_local;

    /* Polling of transfer ready */
    while( READ_LSTA() );

    /* Source address set */
    src_local = (GDC_ULONG)src – MB86290_BASE;
    WRITE_LSA(src_local);

    /* Transfer count set */
    WRITE_LCO(count);

    /* Trigger */
    WRITE_LREQ(1);

#ifdef SINGLE_DL_BUFFER
    /* Wait for next the Display List buffer write to be ready */
    while( READ_LSTA() );
#endif
}
```

***CPU transfer**

This is a method to write the transfer data (Display List) to the Display List FIFO of MB86290 Series by software. The operation procedure of this case is shown as follows. Repeat (1) through (4) for the times specified by "count".

(1) Acquires the Display List FIFO status
   - Calls the **GdcGetFIFOStatus** command and acquire the Display List FIFO status information.

(2) Checks the Display List FIFO status
   - Checks the empty entries of the Display List FIFO from the above status information. If FIFO is full, keep repeating (1) and (2) till open entries will be available.

(3) Transfers 4byte of data from the source address to the Display List FIFO

(4) Posts increment (+4) source address

[Example]

```
/* Start address of drawing control register field */
/* Please set a suitable value to the following ########
   according to the environment of use*/
#define DRAWBASE            0x########
#define FIFO_FULL           0x2

#ifdef GDC_MB86290A
/* for MB86290A */
#define WRITE_FIFO(i)       ( *(volatile GDC_ULONG*)(DRAWBASE+0x4a0) = (i) )
#else
/* for MB86291 or later */
#define WRITE_FIFO(i)       ( *(volatile GDC_ULONG*)(DRAWBASE+0x8400) = (i) )
#endif

void GdcFlushDisplayList(GDC_ULONG *src, GDC_ULONG count){
    int    i;

    for(i = 0; i < count; i++){
        /* If FIFO is full, wait until open entry will be available */
        while(GdcGetFIFOStatus() & FIFO_FULL);

        /* Transfers data to the FIFO */
        WRITE_FIFO(*src++);
    }

}
```

6.1.2 XGdcSwitchDLBuf [Changes DL Buffer]

Interface

GDC_BOOL **XGdcSwitchDLBuf** (GDC_CTX **drvctx*)

Arguments

drvctx	Pointer to Context

Return value

GDC_TRUE	Complete
GDC_FALSE	Incomplete

Error code

GDC_ERR_SWITCH_DL_BUF_FAILED

The **XGdcSwitchDLBuf** command has finished abnormally

Target GDC

All

Called by

The command for drawing control (The command which generates a Display List)

Description

When the Display List to generate is larger than the availability of the current DL Buffer, the command for drawing control calls this command, in order to change the current DL Buffer to another DL Buffer. The contents which this command should perform are searching DL Buffer which can be used and changing the current DL Buffer to applicable DL Buffer (a change uses the **XGdcSetDLBuf** command). When each other DL Buffer is in the state (for example, the waiting Display List for transmission is stored in DL Buffer) which cannot be used, change waiting and DL Buffer until it will be in the state which can be used within this command. For a certain reason, the change of DL Buffer be impossible, when interrupt processing, set up error code **GDC_ERR_SWITCH_DLBUF_FAILED** (the **XGdcSetErrCode** command is used), and return **GDC_FALSE**. Thereby, the command for drawing control of a calling by interrupts processing.

Notes

- When the generated Display List needs to be saved, it is necessary to mount so that applicable DL Buffer may not be chosen with this command.
- By calling this command, a Display List is outputted ranging over two or more DL Buffers. In this case, in application, since two or more DL Buffers must be transmitted to the Graphics Controller, it is necessary to mount the mechanism which should transmit which DL Buffer to the Graphics Controller in which turn, or is detected from application. Moreover, must

be transmit continuously the Display List generated ranging over two or more DL Buffers to the Graphics Controller.

- When this command became to unusual end, application needs to cancel the Display List generated to the middle.

6.1.3 GdcWait [Waiting routine]

Interface

void **GdcWait** (GDC_ULONG *msec*)

Arguments

msec Waiting time (millisecond)

Return value

None

Target GDC

All

Called by

The **GdcInitialize** command

Description

Return after the time progress specified by *msec*.

After initializing setting of the Graphics Controller in the **GdcInitialize** command, this command is called in order to consume time required for until stable of operation.

Even if it consumes more than the time specified by *msec*, there is no problem.